



Máster Universitario en Computación Gráfica, Realidad Virtual y Simulación

TRABAJO FIN DE MÁSTER

Desarrollo de una aplicación de realidad mixta con interacción por voz para la operación de vehículos submarinos ROV

Autora:

Olaia Picabea Sainz

Tutores:

Juan Jesús Ruiz Toscano

Thalia Alicia Morel Otazú

Madrid, 22 de Septiembre de 2026

Resumen

En entornos submarinos, donde el acceso humano directo es difícil, peligroso y costoso, la inspección del suelo marino y el mantenimiento de parques eólicos (entre otros) se realizan habitualmente mediante vehículos submarinos operados remotamente (ROV). La utilización de dichos robots exige al operario una atención mantenida en el tiempo y una toma de decisiones en tiempo real, mientras utiliza un mando de control y supervisa el vídeo y los datos de telemetría en una pantalla de un ordenador. Esta configuración obliga al operario a repartir su atención entre dos sistemas físicamente separados, y esto puede afectar a la seguridad y usabilidad de la operación, especialmente en condiciones no ideales, como la baja visibilidad por el brillo del entorno.

Este Trabajo de Fin de Máster presenta el diseño, implementación y evaluación de una experiencia de realidad mixta para Meta Quest que traslada la información del ROV a una pantalla virtual dentro del campo de visión del operario, permitiendo además cambiar entre realidad virtual y realidad aumentada (*passthrough*). Asimismo, la aplicación incluye un sistema de interacción por voz basado en detección de una *wake word*, y la escucha, transcripción y clasificación de comandos, que permite al operario controlar la disposición de la pantalla virtual sin necesidad de soltar el mando del ROV.

El sistema ha sido evaluado mediante experimentos de sesiones repetidas, y se ha medido el comportamiento de la aplicación calculando el tiempo de respuesta, la precisión de reconocimiento de comandos, la latencia de comunicación entre procesos, el consumo de recursos y la estabilidad del *framerate*.

Índice

1. Introducción **1**
- 1.1. Introducción . . . 1
- 1.2. Planteamiento del problema . . . 3
- 1.3. Objetivos . . . 4
  - 1.3.1. Objetivo general . . . 4
  - 1.3.2. Objetivos específicos . . . 4

2. Estado del Arte **5**
- 2.1. Realidad virtual y aumentada aplicada a la teleoperación de ROVs . . . 5
- 2.2. Realidad mixta y *frameworks* XR . . . 6
  - 2.2.1. Motor de desarrollo . . . 6
  - 2.2.2. Elección de SDK . . . 8
- 2.3. Captura y duplicación de pantalla / *streaming* de escritorio a VR . . . 8
  - 2.3.1. APIs de captura en Windows . . . 9
  - 2.3.2. Transferencia de *frames* GPU-GPU . . . 10
  - 2.3.3. Productos existentes de conexión PC Meta Quest . . . 12
  - 2.3.4. Conclusiones . . . 13
- 2.4. Interacción por voz en entornos inmersivos . . . 14
  - 2.4.1. Activación de sistemas de interacción por voz: *wake word* frente a activación manual . . . 14
  - 2.4.2. Arquitectura de procesamiento: local en el dispositivo frente a servidor externo . . . 16
  - 2.4.3. Modelos de reconocimiento automático del habla . . . 16
  - 2.4.4. Interpretación de comandos: comparación textual (*fuzzy matching*) frente a comprensión del lenguaje natural (NLU) . . . 17
  - 2.4.5. Síntesis y posicionamiento del trabajo . . . 18
- 2.5. La multimodalidad como respuesta a una restricción física . . . 18
- 2.6. Arquitecturas de comunicación entre procesos . . . 19
  - 2.6.1. Sockets TCP frente a otras alternativas de comunicación . . . 19
  - 2.6.2. TCP vs UDP . . . 20
  - 2.6.3. Operaciones de entrada y salida bloqueantes y ejecución concurrente . . . 21
  - 2.6.4. Conclusiones . . . 22
- 2.7. Conclusiones . . . 22

3. Desarrollo **24**
- 3.1. Arquitectura general . . . 24
  - 3.1.1. Aplicación de Unity XR . . . 24
  - 3.1.2. Plugin nativo StreamPantalla . . . 25
  - 3.1.3. Gestión de voz VoiceService . . . 25
  - 3.1.4. Comunicación entre procesos . . . 26
- 3.2. Justificación de las decisiones arquitectónicas . . . 27

3.3. Flujo de datos de extremo a extremo . . . 27
3.3.1. Flujo principal . . . 28
3.3.2. Flujo con *wake word* descartada por falso positivo . . . 29
3.3.3. Flujo con comando no reconocido . . . 30
3.4. Despliegue físico . . . 32
3.5. Restricciones del diseño . . . 32
3.6. Desarrollo detallado . . . 33
3.6.1. StreamPantalla . . . 33
3.6.2. VoiceService . . . 36
3.6.3. Unity XR . . . 42
3.6.4. Empaquetado y despliegue . . . 45
3.6.5. Obtención de datos para los resultados . . . 45

4. Resultados **53**
4.1. Introducción . . . 53
4.2. Tiempo total del *pipeline* de reconocimiento de comandos . . . 54
4.3. Precisión de reconocimiento de comandos . . . 56
4.4. Tiempo de escucha y procesamiento del comando . . . 57
4.5. Latencia de comunicación entre Python y Unity . . . 57
4.6. Consumo de recursos . . . 58
4.7. Rendimiento - FPS . . . 62
4.8. Limitaciones de las pruebas . . . 63

5. Conclusiones **64**
5.1. Conclusiones . . . 64
5.2. Trabajo Futuro . . . 65

6. Bibliografía **67**

A. Anexos **73**
A.0.1. Listado de comandos de voz disponibles . . . 73
A.0.2. Ejemplo del archivo de configuración config.json . . . 84
A.0.3. Sesiónes para obtener los resultados . . . 84

Índice de figuras

1.1. Imagen real de la experiencia de XR desarrollada. 1

3.1. Arquitectura general del proyecto: comunicación entre Unity, el módulo de captura de pantalla y el *backend* de control de voz. 24
3.2. Flujo de datos de extremo a extremo principal entre los procesos independientes de Python y Unity. 29
3.3. Flujo de datos de extremo a extremo para casos en los que se descarta la *wake word* por falso positivo. 30
3.4. Flujo de datos de extremo a extremo para casos en los que no se encuentra el comando. 31
3.5. Diagrama de despliegue físico: distribución del software sobre el *hardware*. 32

4.1. Esquema de detección de resultados posicionados en la ejecución. . . . 53
4.2. Representación visual de los tiempos medios del *pipeline* por modo de activación. 55
4.3. Representación visual de los tiempos medios del *pipeline* por sesión. . . 55
4.4. Porcentajes del tiempo de cada fase. 57
4.5. Latencia media por comandos 58
4.6. Latencia media por sesión 59
4.7. Consumo temporal de CPU en Python para cada sesión. 59
4.8. Consumo temporal de memoria en Python para cada sesión. 60
4.9. Consumo temporal de CPU en Unity para cada sesión. 61
4.10. Consumo temporal de memoria en Unity para cada sesión. 61
4.11. Evolución de FPS en el tiempo. 62

Índice de cuadros

3.1. Resumen de los archivos de registro generados durante la ejecución . . 46

4.1. Estadísticas del tiempo total del *pipeline* según el modo de activación. . 54
4.2. Tiempo medio del *pipeline* por sesión. 55
4.3. Tabla comparativa entre la transcripción exacta y el índice de acierto. . 56
4.4. Latencia media por sesión. 58
4.5. Media de FPS por sesión. 63

1. Introducción

1.1. Introducción

Hay lugares, como los entornos submarinos, donde el acceso humano directo es difícil, peligroso y costoso, y donde se necesita infraestructura especializada. Como solución, existen los vehículos submarinos operados remotamente (ROV), que permiten obtener datos e imágenes del fondo marino operando el dispositivo desde fuera del agua, desde un puesto de operación. Este tipo de vehículo se utiliza habitualmente en tareas como la inspección del suelo marino, el mantenimiento de parques eólicos marinos, la supervisión de tuberías marinas o la inspección de infraestructuras portuarias, entre otras.

Este tipo de operaciones es exigente para el operario porque requiere atención sostenida, además de precisión y toma de decisiones en tiempo real. Asimismo, las condiciones de trabajo no siempre son ideales, por ejemplo, por fenómenos atmosféricos. Los puestos de mando normalmente contienen un ordenador para visualizar las imágenes e información, además de un mando para controlar el ROV. Esta configuración, junto con las limitaciones de las condiciones de trabajo mencionadas, empeora la experiencia de usuario y dificulta el manejo correcto del vehículo. No solo afecta a la experiencia de usuario, sino que también puede traducirse en tiempos de reacción más lentos o en la posibilidad de pasar por alto anomalías durante la inspección.

En este proyecto se presenta una experiencia de realidad mixta que da respuesta a este problema, donde el operario puede visualizar en una pantalla virtual la imagen y los datos del vehículo en un entorno inmersivo (ver imagen 1.1), sin necesidad de recurrir a un ordenador separado. El operario consigue superar las barreras que impone el uso de un PC tradicional, como la baja visibilidad por el brillo del entorno o las distracciones de elementos externos, como se explica en la sección 1.2.

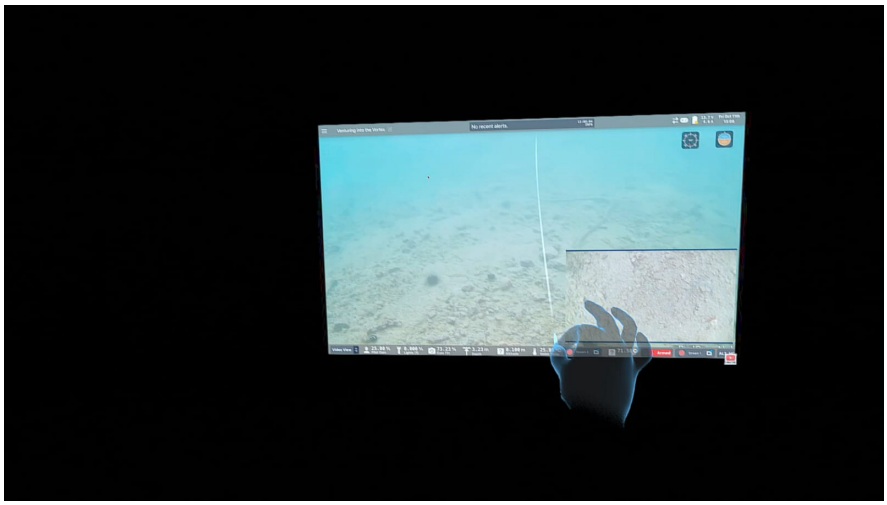


Figura 1.1: Imagen real de la experiencia de XR desarrollada.

Una de las características de la experiencia es que se trata de realidad mixta, lo que permite pasar de realidad virtual a realidad aumentada con un solo comando. La razón de ello es que el operario necesita mantener consciencia de su entorno físico real, pero a veces necesita la concentración máxima que le proporciona la realidad virtual. Por ello, la aplicación permite activar y desactivar el *passthrough* con un único comando.

Además de la visualización, el proyecto integra un sistema de interacción por voz basado en comandos específicos que permiten al operario controlar la posición de la pantalla o la gestión del *passthrough*, sin tener que soltar el mando del ROV.

Por último, este proyecto ha permitido aplicar lo aprendido en el Máster Universitario en Computación Gráfica, Realidad Virtual y Simulación. Entre ello, destaca la utilización de Unity como motor de desarrollo, la aplicación de principios de experiencia de usuario, el uso de lenguajes de bajo nivel como C++ y la utilización de modelos de inteligencia artificial aplicados para el reconocimiento de comandos de voz del operario.

La estructura general del documento consta de los capítulos Introducción (capítulo 1), Estado del Arte (capítulo 2), Desarrollo (capítulo 3), Resultados (capítulo 4) y Conclusiones (capítulo 5).

1.2. Planteamiento del problema

La operación de un vehículo submarino operado remotamente necesita que un operario maneje el vehículo mediante un mando de control físico mientras supervisa el vídeo captado por sus cámaras, los datos de telemetría y el estado del vehículo. Actualmente, esta supervisión se hace con un ordenador junto al puesto de mando en el que se ejecuta Cockpit, un software de control que muestra la información necesaria. Este planteamiento obliga al operario a repartir su atención entre la pantalla del ordenador y el mando. Además, los puestos de mando suelen estar cerca del puerto y al aire libre, por lo que el brillo del sol o los fenómenos climatológicos pueden afectar a la correcta visualización de la pantalla.

El planteamiento actual tiene dos problemas. Por un lado, tener dos sistemas físicamente separados puede afectar a la usabilidad y la seguridad de la operación, ya que el operario puede verse distraído por factores externos o puede tener problemas para visualizar la pantalla. Por otro lado, si el operario quiere cambiar la visualización, tiene que recurrir a un teclado y ratón, lo cual le obliga a soltar el mando del ROV.

Este Trabajo de Fin de Máster utiliza la realidad mixta para crear un entorno inmersivo donde se traslada el contenido de la pantalla del ordenador a una pantalla virtual dentro del campo de visión del operario, evitando a que este tenga que dirigir la mirada hacia un ordenador. Además, la aplicación desarrollada ofrece la interacción mediante comandos de voz para que el usuario pueda interactuar con la aplicación sin necesidad de soltar el mando de control. Es importante mencionar que se ha optado por duplicar la pantalla y no se ha integrado Cockpit directamente por las restricciones de tiempo del proyecto y la falta de acceso físico al ROV durante el desarrollo.

En resumen, este proyecto da solución a los problemas planteados mediante el diseño e implementación de una aplicación de realidad mixta para Meta Quest.

1.3. Objetivos

1.3.1. Objetivo general

El objetivo principal de este Trabajo de Fin de Máster es diseñar e implementar una aplicación de realidad mixta para Meta Quest que permita a un operario de un ROV visualizar, de una manera inmersiva, el vídeo y los datos de telemetría obtenidos en la aplicación Cockpit, además de permitirle interactuar con dicha información mediante comandos de voz, sin necesidad de soltar el mando de control.

1.3.2. Objetivos específicos

Los objetivos concretos del proyecto se pueden ver en el siguiente listado:

1. Diseñar y desarrollar una aplicación de realidad mixta que permita al operario visualizar la información del ROV (vídeo, datos telemétricos y estado) sin necesidad de recurrir a una pantalla externa.
2. Incorporar un sistema de interacción por voz que permita al usuario controlar la visualización de dicha información sin utilizar las manos ni los mandos de los visores.
3. Diseñar e implementar un sistema de evaluación para medir y cuantificar el comportamiento de la aplicación.
4. Evaluar el rendimiento del sistema en tiempos de respuesta, precisión de reconocimiento, eficiencia de uso de recursos, entre otros.
5. Analizar los resultados obtenidos, encontrar las limitaciones, desarrollar las conclusiones y analizar posible trabajo futuro.

2. Estado del Arte

Para crear una aplicación con un buen rendimiento y una baja latencia, es importante investigar, antes de implementar, las diferentes tecnologías disponibles actualmente. Además, conviene identificar qué aporta esta solución respecto a las existentes.

En este capítulo se explicarán dichas tecnologías que se estudiaron al inicio del proyecto, lo que permitirá tener un contexto tecnológico para el capítulo de desarrollo 3.

2.1. Realidad virtual y aumentada aplicada a la teleoperación de ROVs

Antes de analizar tecnologías concretas, el primer paso fue investigar soluciones actuales de realidad virtual y aumentada aplicadas específicamente a ROVs.

En el ámbito académico, el proyecto TWINBOT presenta una aplicación construida en Unity para las gafas HTC Vive que sirve para pilotar vehículos autónomos submarinos (AUV) cooperativos [1]. Su objetivo es obtener una nueva interfaz para reducir la complejidad para el operador que debe gestionar dos robots diferentes simultáneamente. Duplicar interfaces tradicionales como el ratón, teclado o pantallas no es una solución realista por su incremento de dificultad [1], por tanto, proponen sustituirlo por una única interfaz de realidad virtual. Esta aplicación se basa en el control del robot mediante los mandos del visor, pero no permite al usuario comunicarse con la aplicación mediante voz. Además, reconstruye la interfaz desde cero con un servidor de simulación propio, sin reutilizar el software del robot para obtener la imagen real o sus datos.

Una solución más ligada al uso de voz, aunque en el ámbito terrestre, es un proyecto presentado en el artículo LLM-Driven Augmented Reality Puppeteer: Controller-Free Voice-Commanded Robot Teleoperation [2]. La aplicación presentada combina un modelo de lenguaje, realidad aumentada y comandos de voz para teleoperar un robot sin un mando físico. El usuario interactúa con una réplica virtual del robot en la experiencia mientras da órdenes por voz y se ejecutan los correspondientes comandos [2].

Revisando los proyectos mencionados en este apartado, se observa que existen trabajos que combinan voz y realidad aumentada o virtual para la teleoperación de robots [2]. Sin embargo, esta solución sustituye el control físico, y en el caso de los ROV los mandos siguen siendo el método principal para pilotarlos por su complejidad y necesidad de precisión. Este trabajo mantiene los mandos como método para operar el ROV y añade la voz para tareas secundarias, integrándose directamente con el software de control ya existente en lugar de sustituirlo.

2.2. Realidad mixta y *frameworks* XR

Una vez identificado el hueco que cubre este proyecto, el siguiente paso es comparar las tecnologías disponibles y encontrar cuáles permiten construir una solución con estas características.

2.2.1. Motor de desarrollo

La selección del motor de desarrollo es una de las decisiones más relevantes para el desarrollo de la aplicación, ya que determina las limitaciones de la implementación y las posibilidades de integración con componentes externos. Entre las diferentes alternativas existentes, Unity y Unreal Engine destacan en el desarrollo de aplicaciones de realidad mixta.

En este contexto, la elección del motor no solo se basa en su capacidad para crear una aplicación de realidad mixta general, sino principalmente en su capacidad para integrarse con componentes externos. En este proyecto específico se necesita poder utilizar código nativo en C++ para desarrollar un sistema de captura de pantalla, por lo que este será el criterio fundamental a analizar en este apartado.

Integración de código nativo en C++

Tanto Unity como Unreal Engine permiten integrar código nativo en C++ mediante *plugins*, aunque el sistema cambia dependiendo del motor. En Unreal, el *plugin* se incorpora al proyecto y se desarrolla una interfaz en C++ encargada de exponer las funciones del DLL que quedan accesibles desde Blueprints. Este proceso obliga al desarrollador a configurar el módulo correspondiente y especificar las dependencias de la biblioteca mediante archivos de configuración del proyecto (`Build.cs`) [3, 4]. En Unity, en cambio, el *plugin* compilado se incorpora como un recurso del proyecto (teniendo en cuenta la plataforma y arquitectura para la que ha sido compilada) y sus funciones se declaran directamente en un *script* de C# utilizando el atributo `DllImport` y sin configuración adicional [5, 6].

Esta diferencia se debe a que Unreal Engine está desarrollado principalmente en C++ y proporciona un entorno de desarrollo orientado a ese lenguaje. Unity, por el contrario, utiliza C# como lenguaje principal [7]. Esta diferencia hace que Unreal Engine proporcione una integración más directa, mientras que Unity proporciona una separación más clara entre el código base de la aplicación y los *plugins* nativos.

Aunque la arquitectura de Unreal Engine resulta más ventajosa para proyectos con *plugins* nativos complejos y extensos, este proyecto no se beneficiaría de dichas ventajas, ya que sus necesidades se limitan a un conjunto reducido de funcionalidades. En conclusión, el mecanismo de integración de *plugins* de Unity resulta suficiente y presenta menos complejidad de integración para el desarrollador.

Motor para desarrollo de XR

Aunque la posibilidad de incluir *plugins* nativos ha sido el criterio principal a la hora de elegir el motor gráfico, es importante tener en cuenta la capacidad del motor para desarrollar experiencias de realidad mixta. Existen varios estudios donde se comparan Unity y Unreal Engine para realizar aplicaciones de XR. Por un lado, [8] presenta una comparación entre los dos motores (Unreal Engine 5.3 y Unity 2023.2) enfocada en la fidelidad del *rendering*, la eficiencia computacional, compatibilidad entre plataformas y el flujo de trabajo. No obstante, [9] analiza con profundidad las capacidades de Unreal Engine y lo compara con Unity en un nivel técnico y narrativo. Cabe destacar que las versiones de Unreal Engine y Unity comparadas en los artículos no son las actuales, y por ende, puede ser que los datos no coincidan con la realidad. Aun así, aunque las versiones más recientes de ambos motores hayan introducido mejoras de rendimiento, las diferencias estructurales que se mencionan en los estudios se mantienen como referencia válida para justificar la elección del motor.

Uno de los factores fundamentales de un motor para el desarrollo de XR es el rendimiento. Resumiendo lo que se menciona en los artículos [8] y [9], Unreal ofrece mayor fidelidad visual a cambio de un mayor coste computacional (mayor uso de GPU, CPU y VRAM). Por el contrario, Unity es más ligero en recursos, aunque con menor calidad gráfica en escenas complejas. Dado que la experiencia se centra en crear una pantalla virtual sobre el entorno real (*passthrough*) y no en renderizar escenas complejas, el menor coste computacional de Unity resulta más adecuado para mantener un buen rendimiento.

Por otro lado, la curva de aprendizaje varía dependiendo del motor. Por ejemplo, en [8] se menciona que las horas estimadas para prototipar son de 15 a 20 horas en Unreal contra 8 a 12 horas en Unity. También se menciona que Unity es considerado más accesible para principiantes gracias a su interfaz intuitiva y el uso de .NET y C#. En conclusión, la curva de aprendizaje de Unreal se ha demostrado ser más pronunciada que la de Unity [8, 9]. Al tratarse de un proyecto individual y con tiempo limitado, una curva de aprendizaje más accesible como la Unity facilita el trabajo.

Asimismo, los requisitos de *hardware* son de vital importancia, ya que afectan directamente la viabilidad de utilizar un motor específico [8]. Como se menciona en el artículo, Unreal Engine 5.5, por ejemplo, requiere una tarjeta gráfica compatible con DirectX 11/12, mínimo 8 GB de VRAM, 32 GB de RAM y CPU de 2,5 GHz o superior. Sin embargo, Unity 6 tiene requisitos considerablemente menores: en Windows cualquier GPU es válida si es compatible con DirectX 10, 11 o 12, en dispositivos móviles el mínimo es de 1 GB de RAM, y en sistemas embebidos con Linux 1 GB de RAM, CPU Dual-Core x64 o ARM64 y GPU compatible con OpenGL ES3 o Vulkan 1.1 [8].

En el desarrollo de Virtual Museum Project explicado en [8], se empleó el *XR Interaction Toolkit* de Unity, que ofrece componentes como el *XR Ray Interactor* y el *XR Direct Interactor* para interacciones realistas con los elementos del espacio. En dicho proyecto Unity fue elegido por su arquitectura ligera, facilidad de uso y amplias herramientas de XR, ideal para un proyecto centrado en interactividad más que en gráficos avanzados [8].

En conclusión, Unreal Engine se ha demostrado ser adecuado para proyectos donde la calidad visual es indispensable, mientras que Unity se ha adoptado en aplicaciones que priorizan la accesibilidad y el desarrollo más rápido [9].

En conjunto, la menor complejidad del sistema de *plugins* nativos de Unity frente a Unreal Engine y su idoneidad para aplicaciones que priorizan la accesibilidad y el desarrollo rápido frente a la calidad gráfica avanzada hacen que Unity sea el motor adecuado para este proyecto.

2.2.2. Elección de SDK

Una vez elegido el motor de desarrollo, la siguiente decisión a tomar fue qué SDK utilizar. OpenXR se ha establecido como el estándar para el desarrollo de aplicaciones de realidad virtual multiplataforma, por lo que se consideró necesario analizarlo para determinar si resultaba adecuado para las necesidades de este proyecto.

Antes de OpenXR, cada fabricante de *hardware* de realidad virtual exigía desarrollar con su SDK propio. Esto obligaba a adaptar el código para cada visor y complicaba el desarrollo de aplicaciones, pero la compañía Khronos [10] presentó OpenXR. Este es un estándar abierto y multiplataforma que permite desarrollar una única vez y ejecutarlo en diferentes dispositivos. Prueba del éxito de este estándar es que a fecha de redacción de este trabajo, fabricantes como Meta, HTC, Microsoft y Valve cuentan con soporte para OpenXR [10].

Gracias a la adopción de OpenXR por parte de los fabricantes, elegir este estándar frente a un SDK propio garantiza la portabilidad y flexibilidad de la aplicación, lo cual es importante si el proyecto se quisiera extender a otros dispositivos.

Además, Meta ofrece una extensión de OpenXR, llamada Meta SDK. Esta capa extra da acceso a extensiones específicas del *hardware* de Meta [11], como el `XR_FB_passthrough`, que permite mostrar al usuario su entorno físico real. Esta funcionalidad específica no está incluida con OpenXR puro porque depende directamente de las cámaras y el *hardware* específico de cada visor, por lo que no existe una implementación común entre fabricantes. Este proyecto se ha planteado para utilizar dispositivos de Meta Quest con *passthrough*, por tanto, se ha utilizado Meta SDK por encima de OpenXR.

2.3. Captura y duplicación de pantalla / *streaming* de escritorio a VR

Una característica del proyecto consiste en tener un plano flotante en la escena donde se duplica la pantalla del ordenador al que está conectado. En otras palabras, la meta es capturar la pantalla de Windows y llevarla a una textura en Unity. Windows ofrece diferentes mecanismos para acceder al contenido de la pantalla, como por ejemplo GDI (interfaz para que las aplicaciones obtengan los gráficos sin acceder al *hardware* [12]) y DXGI (API orientada al acceso de imágenes del escritorio con acceso a la GPU).

Aun así, no todas las APIs dan la imagen de la misma manera, ni con las mismas opciones de procesamiento.

2.3.1. APIs de captura en Windows

Existen diferentes alternativas para acceder a los gráficos de un sistema Windows. En este apartado se explicarán las diferentes opciones, así como sus ventajas y desventajas.

GDI

Como ya se ha mencionado brevemente, GDI (Graphics Device Interface) es una interfaz que permite a las aplicaciones utilizar gráficos y texto formateado tanto en vídeo como en impresoras. Esta herramienta hace que las aplicaciones no accedan al *hardware* de gráficos y utilicen GDI como intermediario. GDI es la interfaz utilizada para dibujar en un *device context*, es decir, en la estructura que define un conjunto de objetos gráficos y sus atributos [13]. Entre esos objetos gráficos están los *bitmaps*, que se utilizan para crear, manipular y guardar imágenes en el disco [14]. Estos contextos son importantes porque la función `BitBlt` recibe dos *device contexts* como parámetros, uno de origen y el otro de destino. Esta función de GDI realiza una transferencia de los datos de color correspondientes a un rectángulo de píxeles desde un *device context* de origen a uno de destino [15].

GDI está basado en operaciones de copia entre *device contexts*. Por tanto, la ruta de datos y las copias son un factor a considerar cuando la captura se realiza repetidamente y a alta resolución.

Desktop Duplication API

El Desktop Duplication API es un servicio que se creó con el fin de proporcionar acceso al contenido del escritorio en escenarios de colaboración y acceso remoto. La documentación afirma que la API permite acceder a actualizaciones del escritorio *frame* a *frame*, y como estas actualizaciones se proporcionan mediante una superficie DXGI, las aplicaciones que la implementan pueden utilizar la capacidad total de la GPU para procesar la imagen [16].

DXGI (DirectX Graphics Infrastructure) permite trabajar con adaptadores gráficos, sus salidas y los recursos asociados a la infraestructura gráfica de DirectX, y es la infraestructura sobre la que se apoya Desktop Duplication API [17]. A diferencia de GDI, los *frames* obtenidos mediante Desktop Duplication son descritos como imágenes colocadas en memoria de GPU y contienen metadatos que detallan qué regiones de la pantalla se han modificado o desplazado desde la última actualización [18]. Esto hace que se puedan diseñar sistemas aprovechando la aceleración del *hardware* y evitar procesar la imagen completa en cada actualización.

Sin embargo, este sistema presenta algunas limitaciones. Por ejemplo, cada interfaz de duplicación representa un solo monitor (*output*), y para duplicar múltiples

escritorios se debe gestionar una interfaz por cada uno, y Microsoft establece que, por defecto, solo cuatro procesos pueden utilizar simultáneamente una interfaz de Desktop Duplication [19]. Además, la interfaz puede quedar inválida en cualquier momento de la ejecución por determinados cambios de escritorio, y si eso pasa, debe reconstruirse [20].

Windows.Graphics.Capture

Windows.Graphics.Capture es un conjunto de APIs de Windows que se emplea para capturar contenido de una pantalla o ventana de la aplicación para crear imágenes, vídeos o experiencias interactivas [21]. El flujo de captura se divide en un *picker* que permite al usuario seleccionar qué pantalla o ventana capturar, una sesión que controla la captura sobre ese elemento y un *frame pool* que almacena los *frames* obtenidos durante la sesión [21, 22, 23, 24].

Este sistema moderno tiene varias ventajas, como por ejemplo la posibilidad de capturar tanto ventanas de aplicación como pantallas completas; un *picker* de selección proporcionado por el propio sistema, que facilita el desarrollo; su integración nativa con Direct3D; y la incorporación de mecanismos propios para gestionar detalles como el cursor [23].

Aun así, hay varias limitaciones a tener en cuenta. Por ejemplo, algunas funcionalidades de Windows.Graphics.Capture dependen de versiones específicas de Windows (el *picker*, por ejemplo, empezó a estar disponible a partir de la versión 1803 de Windows 10 [22]). Además, este sistema no da control total sobre la captura, ya que Windows lo gestiona internamente (por ejemplo, para elegir qué ventana capturar hay que usar GraphicsCapturePicker) y hace que haya menos capacidad de personalización. Asimismo, cuando se utiliza este sistema, Windows dibuja automáticamente un borde amarillo para avisar al usuario de ello y la única manera de quitarlo es que el usuario dé un permiso especial [25].

2.3.2. Transferencia de *frames* GPU-GPU

Transferencia CPU vs. GPU y `CopyResource`

Una vez que se captura la pantalla, se debe decidir cómo llevar esos datos hasta el motor gráfico. Primero hay que distinguir entre dos formas de mover contenido gráfico entre dos puntos del sistema. La primera consiste en pasar los datos por memoria accesible por la CPU (GPU → copiar memoria accesible por CPU → CPU → volver a subir a GPU). La segunda manera, en cambio, mantiene los datos dentro de la GPU en todo momento (GPU → operación gráfica → GPU). La transferencia por CPU no es necesariamente negativa, pero la decisión de cómo mover el contenido es un factor relevante a tener en cuenta al diseñar un sistema de captura y presentación en tiempo real.

Existen recursos de programación para transferir los datos utilizando la GPU. En la documentación de Microsoft se explica que la función `ID3D11DeviceContext::`

`CopyResource` se encarga de copiar el contenido completo de un recurso de origen a un destino utilizando la GPU, siempre y cuando el origen y el destino sean del mismo tipo, tengan las mismas dimensiones y compartan un formato de DXGI compatible [26]. Como esta aplicación necesita transferir *frames* en tiempo real y de forma repetida, evitar el paso por la CPU es algo relevante para mantener un buen rendimiento.

Plugins nativos de Unity

En este proyecto específico uno de los objetivos es duplicar la pantalla y enseñarla en la experiencia de VR, lo que significa que el *plugin* nativo de captura tiene que poder acceder al sistema de renderizado de Unity.

El Native Plugin API de Unity permite acceder a funcionalidades del editor y del motor desde *plugins* nativos. Entre las interfaces disponibles, está `IUnityGraphics`, que da acceso a funcionalidades gráficas de Unity y permite al *plugin* identificar y trabajar con el *renderer* activo para sincronizarse con el sistema de renderizado [27, 28].

Otra herramienta que ofrece Unity es la función `Texture.GetNativeTexturePtr`, que permite obtener el puntero a una textura (para dispositivos de tipo Direct3D, el puntero devuelto corresponde al tipo ID3D11Resource). Unity recomienda obtener este puntero únicamente durante la inicialización, ya que llamar a esta función con el renderizado multihilo activo puede provocar una sincronización costosa con el hilo de renderizado [29].

Recursos compartidos y sincronización

La alternativa a copiar explícitamente el contenido de un recurso es la utilización de *shared textures*, un recurso compartido al que pueden acceder diferentes dispositivos o componentes compatibles. Este recurso se puede compartir no solo entre distintos dispositivos de una misma API, sino también entre API gráficas de Windows diferentes entre sí, lo que permite construir aplicaciones que combinan varias tecnologías gráficas de Windows sin renunciar a la compatibilidad entre ellas [30]. Aunque compartir un recurso en vez de crear copias sea una ventaja, esto introduce la necesidad de coordinar el acceso de ambos dispositivos a ese recurso compartido.

Uno de los mecanismos de sincronización para recursos compartidos es el `IDXGIKeyedMutex`, que permite gestionar el acceso exclusivo a un recurso compartido mediante claves [31]. Cada dispositivo adquiere el acceso al recurso con `AcquireSync` y lo libera con `ReleaseSync` cuanto termina de utilizarlo [31, 32, 33].

En este proyecto se ha elegido hacer la copia directa GPU-GPU mediante `CopyResource` y se ha descartado la opción de los recursos compartidos, ya que resulta suficiente para las necesidades del proyecto y se evita la complejidad de la sincronización.

2.3.3. Productos existentes de conexión PC Meta Quest

Este apartado examinará el panorama actual de soluciones que conectan un PC con un visor Meta Quest. Para ello, se comparan diferentes productos utilizando los criterios de la plataforma de visor soportada, si la comunicación se produce entre PC y Meta Quest, el tipo de conexión y si ofrecen *streaming* de escritorio.

Virtual Desktop

Virtual Desktop es una aplicación que permite utilizar un PC desde un visor Meta Quest y realizar *streaming* de contenido PCVR hacia el visor. Su documentación oficial menciona específicamente el soporte con distintos dispositivos Quest, y ofrece *streaming* de escritorio además de *streaming* de juegos PCVR, para lo cual exige un PC compatible con VR ejecutando Windows. En cuanto a los requisitos de red, la documentación señala que la conexión tiene que ser cableada del PC a un *router* mínimo con estándar 802.11ac y el uso obligatorio de la banda de 5 GHz. Para el juego remoto a través de internet se requiere habilitar UPnP o redirigir manualmente varios puertos TCP [34].

Meta Quest Link / Air Link

Link es la solución estándar del ecosistema Meta para conectar un visor Quest a un PC. Por un lado, está Meta Quest Link, que conecta los dispositivos mediante cable USB 3.0 desde el PC hasta un puerto USB-C del visor [35]. Por otro lado, está Air Link, su equivalente inalámbrico. Los requisitos técnicos del PC son comunes para ambas opciones: procesador mínimo Intel i5-4590 o AMD Ryzen 5 1500X, 8 GB de RAM como mínimo, Windows 10 u 11 y una lista de GPU NVIDIA y AMD compatibles [36].

ALVR (Air Light VR)

ALVR es un proyecto de código abierto que permite transmitir juegos de PCVR desde un PC hasta un visor a través de WiFi [37]. A diferencia de las soluciones mencionadas, ALVR documenta con un alto nivel de detalle su arquitectura. SteamVR renderiza el contenido y un driver de ALVR captura el *frame* mediante Direct3D 11 en Windows, donde se le aplica corrección de color. La codificación se realiza por *hardware* siempre que el sistema lo permite (ALVR también soporta codificación por software con x264, aunque con peor rendimiento) [38] y el resultado se transporta por red mediante un *socket* de control (TCP) y uno de datos (UDP o TCP). Cuando ALVR detecta una pérdida de paquetes, solicita un nuevo fotograma IDR al codificador [38]. Una vez en el visor, la decodificación se realiza mediante MediaCodec y se aplican transformaciones finales antes de entregar el *frame* a la ejecución de VR [38].

Bigscreen Remote Desktop

Dentro de la aplicación Bigscreen hay una función de Remote Desktop orientada a visores de Meta Quest. Según su documentación, el visor y el PC deben estar en la misma red local con el visor conectado por WiFi en la banda 5 GHz. Se puede considerar como una solución de PC a red local y de red local a Quest, y orientada al escritorio remoto más que al *streaming* de contenido [39].

Immersed

Immersed está orientado a un escenario distinto a las soluciones anteriores, ya que permite utilizar el visor como pantalla o pantallas de un PC, con hasta cinco monitores simultáneos, y su fin es la productividad. En su documentación no se menciona el *streaming* de contenido PCVR [40].

2.3.4. Conclusiones

Conclusiones sobre las APIs de captura de imagen

Respecto a las APIs de captura de Windows, se pueden obtener tres conclusiones principales. Por un lado, GDI proporciona un sistema clásico de captura basado en *device contexts* y *bitmaps* y está ampliamente disponible, pero pensado originalmente para operaciones gráficas generales y no específicamente para la captura repetida de pantalla. Por otro lado, Desktop Duplication proporciona los *frames* del escritorio a través de DXGI en GPU y con acceso a metadatos, lo que hace que la solución esté más orientada a escenarios de acceso remoto con actualizaciones de *frames* constantes. Por último, Windows.Graphics.Capture ofrece un modelo más moderno, integrado en el sistema operativo y capaz de capturar tanto pantalla como ventanas concretas, pero con un *picker* propio del sistema y con marcas visuales como el borde amarillo. Para este proyecto, GDI queda descartado porque no ofrece acceso a los *frames* en memoria de GPU y por eso reduce el rendimiento cuando una captura se tiene que hacer repetidamente y con alta resolución. Windows.Graphics.Capture, aunque ofrece integración con Direct3D, exige el uso de un *picker* para selección manual de la pantalla a duplicar y dibuja un borde amarillo en la captura. Por estos motivos, se ha elegido Desktop Duplication API como sistema de captura, ya que permite obtener los *frames* directamente en memoria GPU sin interfaces propias del sistema, y esto favorece un flujo de transferencia GPU-GPU que aprovecha su capacidad para maximizar la velocidad de la captura.

Conclusiones sobre la transferencia gráfica

En sistemas con exigencias de tiempo real hay que tener en cuenta las transferencias de datos y si pasan por CPU o no. D3D11 permite realizar copias directamente en GPU mediante `CopyResource`, pero también existen sistemas de recursos compartidos

entre dispositivos como alternativa,que requieren mecanismos adicionales de sincronización, como el que ofrece DXGI mediante `IDXGIKeyedMutex`. Para este proyecto, se ha optado por la copia directa en lugar de recursos compartidas, ya que resulta suficiente para sus necesidades y evita la complejidad de la sincronización.

Conclusiones sobre las soluciones existentes

En cuanto a soluciones existentes, cabe mencionar que ya existen aplicaciones que conectan un PC a un visor. Destacan entre ellas las aplicaciones que permiten hacer un *streaming* de vídeo entre un PC físicamente separado del visor sobre una red. El proceso que más o menos comparten las soluciones comerciales es el siguiente: PC → captura → *frame* → codificación → vídeo comprimido → red → visor → decodificación → *frame* en VR. Sin embargo, este proceso está pensado para comunicar un ordenador que genera el contenido con un dispositivo externo que lo recibe y presenta. Cuando el ordenador que genera el contenido y el sistema que ejecuta la experiencia VR forman parte del mismo equipo físico, como ocurre con una aplicación de Unity para PC que se ejecuta en el mismo PC al que está conectado el visor mediante Meta Quest Link o Air Link, desaparece la necesidad de establecer una conexión entre los dos dispositivos basada en la red, y con ella, parte de la complejidad (codificación, transmisión y decodificación del vídeo).

2.4. Interacción por voz en entornos inmersivos

La interacción por voz se ha convertido en una técnica muy utilizada en sistemas comerciales como Alexa o Siri. Asimismo, es habitual encontrarla en entornos de realidad virtual, donde las manos pueden estar ocupadas por controladores o gestos para interactuar con el entorno. Esta funcionalidad resulta necesaria en este proyecto, ya que los mandos se utilizarán para controlar el ROV y son independientes del entorno de realidad virtual. Sin embargo, incorporar la voz como canal de interacción plantea varias decisiones técnicas: cómo determinar el momento en que el sistema debe empezar a escuchar, dónde ejecutar el procesamiento del audio teniendo en cuenta las librerías y limitaciones del *hardware*, qué modelo de transcripción de voz utilizar y cómo interpretar el resultado para convertirlo a una acción concreta de la aplicación. Para resolver estas decisiones, este proyecto incorpora una aplicación externa desarrollada en Python encargada de la gestión de la voz, que se comunica con Unity. El siguiente bloque revisa el trabajo previo relacionado con este tipo de sistemas y explica las decisiones que se han tomado.

2.4.1. Activación de sistemas de interacción por voz: *wake word* frente a activación manual

Los asistentes comerciales, a veces, utilizan un patrón de detección de dos etapas. La primera consiste en un detector ligero de *wake word* siempre activo, cuya función es detectar si dicha clave se ha dicho o no. A esto se añade una segunda capa, un modelo o

programa pesado que solo se debe ejecutar tras detectar la *wake word*. Esta arquitectura se debe a que es computacionalmente inviable mantener un modelo grande escuchando sin interrupción [41], por tanto, se utiliza uno muy pequeño para esa primera acción [42]. Aunque estas técnicas se utilicen en productos modernos como Amazon Echo [41], el equilibrio entre precisión y coste computacional es un campo de investigación activo [43]. Algunas arquitecturas van más allá e incluyen una segunda capa de detección, a menudo un servidor, que revisa y confirma el primer modelo. Gracias a esta técnica se reduce la cantidad de falsos positivos [44]. En resumen, el primer modelo prioriza la velocidad y bajo consumo frente a la precisión, mientras que el segundo modelo refina la decisión inicial antes de continuar con la ejecución.

Existen varios servicios para integrar *wake words* en aplicaciones locales. Entre ellos se encuentran Picovoice Porcupine [45] y openWakeWord [46], dos librerías que permiten entrenar modelos descargables y utilizarlos en entornos *offline*. La diferencia entre ambas es que las plataformas comerciales como Picovoice requieren registro y una clave de acceso vinculada a una cuenta de terceros, mientras que *frameworks* de código abierto como openWakeWord no necesitan ninguna dependencia externa de cuenta o licencia. En este proyecto se ha optado por openWakeWord por la ausencia de dichas condiciones.

Como se ha mencionado, una de las características para mantener la robustez es la detección de falsos positivos. La literatura analizada identifica dos causas principales.

Por un lado, están los falsos positivos por palabras o sonidos fonéticamente similares a la *wake word*. Este estudio sobre el fenómeno FakeWake analiza sistemáticamente sonidos que no son la *wake word* real, pero que le son acústicamente cercanos. En su proceso consiguen reproducir los falsos positivos, y crean un sistema que produce *fuzzy words*, es decir, las palabras o frases que hacen que los sistemas se activen con un falso positivo. Finalmente, proponen remedios para mitigar el efecto FakeWake y consiguen crear modelos con mejor rendimiento que los originales [47].

La segunda causa de las activaciones falsas es la retroalimentación acústica, donde el propio audio de salida (una respuesta hablada del sistema, por ejemplo) es captado por el micrófono y reinterpretado como una nueva activación. Este fenómeno está documentado en la literatura de patentes de la industria de asistentes de voz, que describe mecanismos de cancelación de eco acústico (AEC) dirigidos a prevenir que un dispositivo se autoactive con su propia salida de audio [48]. Además, otro estudio observacional de 8 semanas analiza la tecnología Amazon Echo y confirma que la tasa de falsos positivos por voz humana varía según la *wake word* elegida y según el *hardware* del dispositivo [49].

Además de los falsos positivos, también se han contemplado los falsos negativos, donde el sistema no se activa aunque se haya pronunciado la *wake word*. Como estos falsos negativos constituyen un problema estructural, el diseño debe contemplar mecanismos de activación alternativos en vez de depender solo de la voz. En este proyecto esto se resuelve mediante *hand tracking*. El usuario puede activar manualmente la escucha de comandos y así evitar que un fallo en la detección de la *wake word* bloquee toda la interacción.

En cuanto al entrenamiento de modelos personalizados de detección de *wake*

word, la técnica más consolidada para obtener los datos de entrenamiento es el uso de datos sintéticos generados mediante sistemas de texto a voz (TTS), en lugar de grabaciones humanas reales. Esta técnica ya se ha aplicado previamente en la literatura de *keyword spotting* [50]. Estas técnicas respaldan la viabilidad de entrenar modelos de *wake word* personalizados sin depender de grabaciones humanas propias, lo cual es fundamental para crear un sistema con una *wake word* personalizada, como es el caso de este proyecto.

2.4.2. Arquitectura de procesamiento: local en el dispositivo frente a servidor externo

Como ya se ha mencionado, este proyecto se basa en OpenXR y Meta SDK, que ya ofrecen un sistema de detección de voz que delega el procesamiento a Wit.ai (servicio en la nube) [51]. Esta solución requiere conexión a internet para funcionar y obliga al desarrollador y al usuario a aceptar las condiciones de tratamiento de datos de la plataforma [52, 53].

Cabe señalar que esta arquitectura donde el dispositivo ligero envía un audio a un servidor externo para su procesamiento no es exclusiva de soluciones en la nube. Este mismo patrón se utiliza en sistemas que delegan a un servidor en la red local. Esta diferencia de distribución puede afectar a la latencia, como se analiza en artículos sobre *edge computing*. Ya en 2017 [54] se evaluaron experimentalmente ambas configuraciones y mostraron que delegar tareas de procesamiento a un servidor fuera de la red local produce latencias de entre 2 y 5 segundos, mientras que un servidor dentro de la red local logra completar el reconocimiento de voz en aproximadamente 200 ms. Cabe mencionar que el mismo estudio reporta que delegar el reconocimiento de voz a un servidor externo consume aproximadamente un 5 % de CPU en el dispositivo cliente, frente a un 20 % si el reconocimiento se ejecuta de forma local en un dispositivo con recursos limitados.

Dado que en este proyecto ambos procesos se ejecutan en la misma máquina, se ha optado por un procesamiento completamente local, evitando así la latencia de red y la dependencia de conexión a internet.

2.4.3. Modelos de reconocimiento automático del habla

Whisper es un modelo Transformer entrenado en múltiples tareas como el reconocimiento multilingüe, traducción de voz o la identificación de idioma. Este modelo está entrenado sobre 680000 horas de audio multilingüe, y gracias a ello el modelo funciona razonablemente bien en entornos de *zero-shot* sin necesidad de ajuste, es decir, cuando procesa datos no vistos en la fase de entrenamiento [55]. Whisper es una herramienta aplicada en entornos de realidad extendida donde se requiere transcripción precisa en tiempo real, como por ejemplo en este artículo sobre asistencia lingüística en entornos inmersivos [56].

Frente a servicios de reconocimiento de voz en la nube, la diferencia principal es que Whisper puede ejecutarse de forma local y sin conexión a internet, sin limitaciones

de uso ni suscripciones a plataformas. La documentación de alternativas ligeras de ejecución *offline* como Vosk confirma la existencia de modelos portátiles de apenas 50 MB frente a modelos de servidor considerablemente más grandes [57]. Sin embargo, Whisper prioriza la robustez a costa de un mayor coste computacional.

Existe además una reimplementación más ligera de Whisper llamada Faster-Whisper [58] y desarrollada por SYSTRAN sobre el motor de inferencia CTranslate2, y ha reportado mejoras de hasta 4 veces en velocidad con un consumo de memoria menor, manteniendo una precisión equivalente a la implementación original. Esta diferencia de rendimiento sin pérdida de precisión ha hecho que Faster-Whisper haya sido el motor de reconocimiento de voz elegido para esta implementación.

2.4.4. Interpretación de comandos: comparación textual (*fuzzy matching*) frente a comprensión del lenguaje natural (NLU)

El fuzzy string matching, implementado en librerías como RapidFuzz [59] mediante métricas basadas en la distancia de Levenshtein y variantes como WRatio [60], mide la similitud entre dos textos sin ningún tipo de comprensión semántica. Este enfoque no requiere entrenamiento previo, es determinista y el coste computacional es mínimo.

Los sistemas de comprensión de lenguaje natural utilizados en sistemas como Alexa o Siri realizan clasificación de intención, es decir, identifican la acción que corresponde a una instrucción del usuario aunque esta varíe de forma superficial respecto a los ejemplos de entrenamiento [61]. Por ejemplo, este artículo sobre la comprensión del lenguaje natural pone como ejemplo la instrucción 'Book me a flight from Denver to New Jersey', del que el sistema de NLU extraería la acción 'book_flight' junto con los valores de Denver como origen y New Jersey como destino [62]. Esta extracción semántica no puede ser realizada por *fuzzy matching*.

Sin embargo, esta mayor flexibilidad que los sistemas de NLU ofrecen tiene un coste: estos sistemas requieren conjuntos de entrenamiento etiquetados, y en caso de arquitecturas basadas en modelos *transformers*, presentan limitaciones documentadas de rendimiento en dispositivos embebidos con recursos limitados. En el artículo [61] los investigadores evalúan el despliegue de estos modelos en plataformas embebidas de bajo coste, y demuestran que su ejecución es viable en determinadas configuraciones, aunque a costa de una complejidad de despliegue y un consumo de recursos notablemente mayor que el de los modelos ligeros evaluados en el estudio. En entornos donde el listado de comandos es cerrado y conocido de antemano, esta diferencia de coste resulta determinante frente a la flexibilidad adicional que aportaría un sistema de NLU. Por ello, se ha optado por *fuzzy matching* mediante RapidFuzz como mecanismo de interpretación de comandos.

2.4.5. Síntesis y posicionamiento del trabajo

La revisión anterior descarta soluciones comerciales de interacción por voz en VR (Voice SDK de Meta, por ejemplo), ya que estas dependen de infraestructura en la nube de terceros y de políticas de tratamiento de datos de dichos proveedores. Al mismo tiempo, la información disponible sobre *edge computing* muestra que trasladar ese procesamiento desde un servidor en la nube a una red local es una alternativa viable que reduce drásticamente la latencia sin renunciar a la capacidad computacional de un servidor externo. En este proyecto, al ejecutarse el proceso de reconocimiento de voz en la misma máquina que Unity, se elimina por completo tanto la dependencia de la nube como la latencia de red. De una manera similar, la literatura de *keyword spotting* confirma que es posible construir modelos de activación por voz robustos sin depender de soluciones comerciales con licencia, por lo que este proyecto utiliza openWakeWord, aunque estos sistemas heredan problemas estructurales de falsos positivos y de retroalimentación acústica que deben tenerse en cuenta en el diseño de la interacción. Por último, para el reconocimiento de comandos desde un listado cerrado, los artículos y la documentación respaldan que un enfoque de comparación textual ligero es preferible a soluciones NLU basadas en *transformers*, ya que el objetivo es mantener el sistema ejecutándose en local con recursos moderados. Por ello, este proyecto utiliza RapidFuzz para la interpretación de comandos.

Entre los trabajos que se han buscado y estudiado, no se ha encontrado en el ámbito de las aplicaciones de teleoperación en realidad virtual una arquitectura de interacción por voz que combine procesamiento local, activación mediante modelos de *wake word* de código abierto y reconocimiento de comandos adaptado a un listado cerrado.

2.5. La multimodalidad como respuesta a una restricción física

Un requisito del proyecto es que la interacción principal sea accesible sin necesidad de obligar al usuario a interactuar con las manos, ya que este se encuentra con las manos ocupadas controlando el mando del ROV durante la mayor parte de la experiencia. Normalmente, las interacciones en realidad virtual se gestionan con mandos o *hand tracking*, pero en este proyecto específico no se ha podido depender de ellos como mecanismo principal, aunque el *hand tracking* sí se utiliza de forma puntual como mecanismo secundario. Esta restricción exige buscar alternativas de interacción *hands-free*. Un estudio sistemático analizó 80 estudios para identificar qué interfaces *hands-free* se utilizan, para qué, qué métricas utilizan para su evaluación y los resultados de dichas evaluaciones. Este estudio compara las modalidades de voz, seguimiento ocular, seguimiento de cabeza, interfaces cerebro-PC, expresión facial, posición corporal, movimiento de pies y actividad muscular, y concluyó que la voz es la modalidad más estudiada y una de las que mejores resultados presenta, especialmente para tareas de control del sistema y entrada simbólica [63]. Esa doble condición de control de sistema (activar o desactivar funciones) y entrada simbólica (comandos discretos, no

continuos) coincide con el tipo de interacción de este proyecto, por lo que se ha elegido la voz como modo de interacción principal en respuesta a esta restricción física.

2.6. Arquitecturas de comunicación entre procesos

La comunicación entre diferentes procesos de la aplicación es una decisión importante de arquitectura, ya que puede tener implicaciones directas sobre el rendimiento, la complejidad de implementación y la escalabilidad futura. La comunicación entre procesos (*Inter-Process Communication*, IPC) es un área ampliamente estudiada en sistemas operativos. En estos estudios se presentan desde soluciones de bajo nivel orientadas a un único *host*, como los *pipes* o la memoria compartida, hasta protocolos de alto nivel diseñados para entornos de red como gRPC o REST sobre HTTP [64].

A continuación se comparan los principales mecanismos de IPC disponibles para determinar cuál resulta más adecuado para este proyecto.

2.6.1. Sockets TCP frente a otras alternativas de comunicación

Según el libro [64], hay dos modelos fundamentales de comunicación entre procesos: memoria compartida y transmisión de mensajes. Cuando se comparte la memoria, los procesos pueden intercambiar información escribiendo y leyendo en la región de la memoria compartida, sin pasar por el *kernel* en cada operación. Sin embargo, el proceso de comunicación por mensajes no hace uso de una región de memoria compartida entre los procesos, es decir, los mensajes se intercambian directamente entre ambos. Ambos sistemas son habituales en sistemas operativos, y la elección depende de la aplicación individual.

El problema de productor-consumidor se soluciona con la comunicación por memoria compartida. Este problema consiste en un productor de datos (el proceso que manda los datos) y un consumidor (el proceso que obtiene los datos). Para que los dos procesos puedan ejecutarse simultáneamente se utiliza un *buffer* alojado en la memoria que puede ser rellenado por el productor mientras se vacía por el consumidor. Sin embargo, para que esto funcione correctamente, y para evitar condiciones de carrera, los dos procesos deben tener una fase de sincronización explícita, para que el consumidor no intente acceder a un elemento que todavía no se ha producido. Este sistema puede ser más rápido que el traspaso de mensajes, ya que este último está habitualmente implementado utilizando llamadas al sistema y por tanto consume más tiempo. Por otro lado, en sistemas de memoria compartida las llamadas al sistema solo se necesitan para consolidar las regiones de la memoria que se utilizarán [64].

Los sistemas de traspaso de mensajes permiten a los procesos comunicarse y sincronizar sus acciones sin compartir espacio en la memoria. Estos sistemas son especialmente útiles en entornos donde los procesos se ubican en diferentes ordenadores conectados a la misma red. Entre los diferentes sistemas de este estilo están los *pipes*, *sockets* y los *Remote Procedure Calls* (RPC) [64].

Un *pipe* es en otras palabras, un canal que permite a dos procesos comunicarse. Aunque un sistema de *pipes* es uno de los más sencillos, también tiene sus limitaciones. En [64] se analizan esas limitaciones centrándose en dos tipos de sistemas de *pipes*: los *pipes* simples y los *named pipes*. La comunicación en los *ordinary pipes* o *pipes* simples es la arquitectura de productor-consumidor original: un proceso escribe en un extremo del *pipe* (*write end*) y el otro lee la información desde el otro extremo (*read end*). Su limitación es que estos *pipes* son unidireccionales. Si se quisiera un sistema bidireccional, harían falta dos *pipes*, cada uno mandando información en una dirección. Además, en Windows y UNIX estos *pipes* se utilizan en relaciones padre-hijo unidireccionales. Esto significa que ambos procesos tienen que estar en la misma máquina. No obstante, los *named pipes* presentan una solución para cuando dos procesos se comunican entre sí en una situación bidireccional y fuera de la estructura padre-hijo. Asimismo, otra de las diferencias con los *pipes* simples es que muchos procesos pueden utilizar los *named pipes* para la comunicación. Aun así, los procesos tienen que estar ejecutándose en la misma máquina.

Para poder comunicar procesos en diferentes máquinas se utilizan los *sockets* [64]. Estos ofrecen una comunicación bidireccional que permite a los dos procesos tanto mandar como recibir información. Esto es un punto clave para este proyecto, ya que aunque los procesos se ejecutan en la misma máquina, el uso de *sockets* hace que en algún momento se puedan trasladar a la nube o a otra máquina haciendo cambios mínimos.

La última técnica tradicional es el RPC, que permite invocar una función en otro proceso o máquina como si fuera local [64]. La diferencia principal frente a los mensajes enviados por *sockets* o *pipes* es que en este sistema los datos que se trasladan tienen una estructura y dejan de ser bloques de datos. Cada mensaje es enviado a un receptor RPC que escucha un puerto en el sistema remoto, y cada mensaje tiene un identificador que especifica la función a ejecutar y los parámetros a pasar. La función luego se ejecuta y el resultado se manda al proceso que lo ha solicitado.

Existen protocolos más modernos como gRPC o REST sobre HTTP, pero su complejidad adicional no está justificada para las necesidades de este proyecto, ya que la comunicación entre los procesos se hace enviando mensajes cortos en formato JSON. Por este mismo motivo, tampoco hacen falta sistemas como RPC ni memoria compartida. No obstante, la comunicación bidireccional y la flexibilidad de poder trasladar el sistema a la nube en un futuro han hecho que los *sockets* se hayan convertido en la opción más completa para este desarrollo.

2.6.2. TCP vs UDP

En la comunicación por *sockets*, los protocolos más utilizados son TCP (*Transmission Control Protocol*) y UDP (*User Datagram Protocol*). Por un lado, TCP es un protocolo de transporte orientado a la conexión y a la fiabilidad, ya que antes de intercambiar datos, ambas aplicaciones establecen la conexión mediante un proceso de *handshake*. Ya con la conexión abierta, TCP garantiza que los datos lleguen completos y en el orden correcto, retransmitiendo cualquier bloque que se pierda [65]. Este comportamiento es fundamental en este proyecto, ya que no se puede permitir que co-

mandos de voz se pierdan en la conexión, o que se procesen comandos pero su función correspondiente en Unity no se ejecute debido a errores en la conexión.

Por el contrario, UDP es un protocolo de transporte sin *handshake* inicial, y no garantiza la entrega ni el orden de los mensajes. Aunque UDP sea útil en aplicaciones donde la latencia es más importante que la precisión, en este proyecto se ha descartado por la necesidad de una transmisión fiable de datos entre Unity y Python [65].

2.6.3. Operaciones de entrada y salida bloqueantes y ejecución concurrente

Las aplicaciones de realidad virtual presentan requisitos de rendimiento bastante exigentes, ya que en estos sistemas es muy importante mantener una correspondencia temporal adecuada entre la interacción del usuario, la actualización de la simulación y la representación de la escena. El tiempo disponible para generar cada fotograma está condicionado por la frecuencia de refresco del dispositivo de visualización. Como se menciona en el manual de Unity, por ejemplo, una frecuencia de actualización de 90 Hz proporciona aproximadamente 11,1 ms para cada fotograma. Si Unity no entrega el fotograma a tiempo para la siguiente actualización de pantalla, el sistema puede mostrar el fotograma anterior o aplicar técnicas de reproyección dependiendo de la posición de la cabeza, pudiendo producir *judder* y una degradación significativa de la calidad de la experiencia [66].

Esta restricción temporal hace especialmente relevante el tratamiento de las operaciones de entrada y salida. En la comunicación mediante *sockets*, las operaciones de recepción síncrona pueden permanecer esperando hasta que existan datos disponibles. La documentación oficial de .NET confirma que cuando no hay datos para leer, el método `Socket.Receive()` permanece bloqueado hasta que llegan datos (a no ser que se haya configurado un tiempo de espera o el *socket* funcione en modo no bloqueante) [67]. Por tanto, la duración de una operación de recepción síncrona no está necesariamente determinada por el propio proceso, sino que puede depender de la disponibilidad de los datos en el *socket*.

Es fundamental analizar esta característica cuando las operaciones de entrada y salida se ejecutan en el hilo responsable de la actualización de la aplicación. En otras palabras, es importante examinar si una operación que permanece esperando datos puede impedir temporalmente que dicho hilo continúe con las tareas que debe realizar para que la aplicación funcione con fluidez. En el caso de Unity, esta cuestión es especialmente relevante debido a la organización de su ejecución en diferentes hilos. La documentación menciona que hay un hilo de simulación encargado principalmente de ejecutar *scripts*, sonido, inteligencia artificial, etc., mientras que el hilo de renderizado se ocupa de enviar las órdenes gráficas al controlador. Unity sincroniza ambos hilos para reducir la latencia y permitir la ejecución paralela de determinadas tareas [66]. En consecuencia, añadir una operación que pueda bloquear el flujo de ejecución de la simulación puede afectar al tiempo necesario para procesar un fotograma, y así, empeorar la experiencia de usuario.

Además del problema asociado al bloqueo, el motor también presenta restricciones

relacionadas con el acceso de su API. La documentación oficial de Unity menciona que la mayor parte de su API no es *thread-safe*, y que salvo casos específicos, no se debe utilizar fuera del hilo principal. Unity proporciona mecanismos para ejecutar operaciones en hilos secundarios y después regresar al hilo principal [68]. Asimismo, la documentación también muestra que determinadas operaciones del motor tienen que ejecutarse en el hilo principal, y advierte que el uso inadecuado de la API en sistemas con más de un hilo puede provocar errores [68]. Por todo ello se recomienda establecer una separación conceptual entre la comunicación externa al motor y los procesos internos del motor.

Toda esta información se puede resumir en dos requisitos a tomar en cuenta en el diseño del sistema. Por un lado, las operaciones de recepción que pueden permanecer bloqueadas no deberían influir en la ejecución del flujo principal y el refresco de fotogramas. Por otro lado, los datos recibidos tienen que transferirse de forma controlada al contexto de ejecución donde se pueden ejecutar las funciones de la API de Unity.

La solución tomada respecto a la gestión de hilos y procesos de esta aplicación se desarrollará en el capítulo 3.

2.6.4. Conclusiones

Entre los mecanismos para la comunicación que se han analizado en este apartado, los *sockets* TCP han resultado la opción más adecuada. A diferencia de los *pipes* o la memoria compartida, los *sockets* permiten comunicar procesos que podrían ejecutarse en máquinas distintas, lo cual da flexibilidad por si en un futuro se quisiera migrar el proceso de gestión de voz a la nube. TCP, frente a UDP, garantiza que los comandos no se pierdan ni lleguen desordenados, un requisito imprescindible en este proyecto. Por último, se han presentado las restricciones de la API de Unity respecto al uso de hilos, pero esto se resolverá en la implementación.

2.7. Conclusiones

Las tecnologías que se han elegido no son las más sofisticadas comparadas con otras de su mismo ámbito, pero contribuyen a la prioridad de diseño de funcionar de forma local sin depender de conectividad externa ni servicios de terceros. Esta prioridad se refleja en las decisiones tomadas a lo largo de este trabajo, desde la duplicación de pantalla y la comunicación entre procesos hasta la interacción multimodal por voz y *hand tracking*. En conjunto, estas decisiones dan forma a un flujo que permite al operario visualizar Cockpit dentro de una experiencia de realidad mixta sin perder el control manual del ROV. Además, la combinación de tecnologías de este proyecto no aparece resuelta en la literatura ni en los productos comerciales que se han revisado, lo cual demuestra la aportación de este trabajo dentro de su ámbito.

3. Desarrollo

Hasta ahora se han presentado tanto los objetivos del proyecto como las decisiones de arquitectura y tecnologías que se han tomado. Aun así, todavía no se ha explicado con detalle cómo se han implementado las soluciones con dichas herramientas. En este apartado se definirá la arquitectura general del proyecto, y se detallará cada bloque para entender su desarrollo, sus limitaciones y los cambios de diseño que se han ido incorporando durante el proceso.

3.1. Arquitectura general

El proyecto podría dividirse en cuatro grandes bloques: la aplicación de realidad virtual en Unity, el *plugin* nativo de captura de pantalla, el programa en Python para el control de voz y la capa de comunicación TCP que une la gestión de voz con Unity.

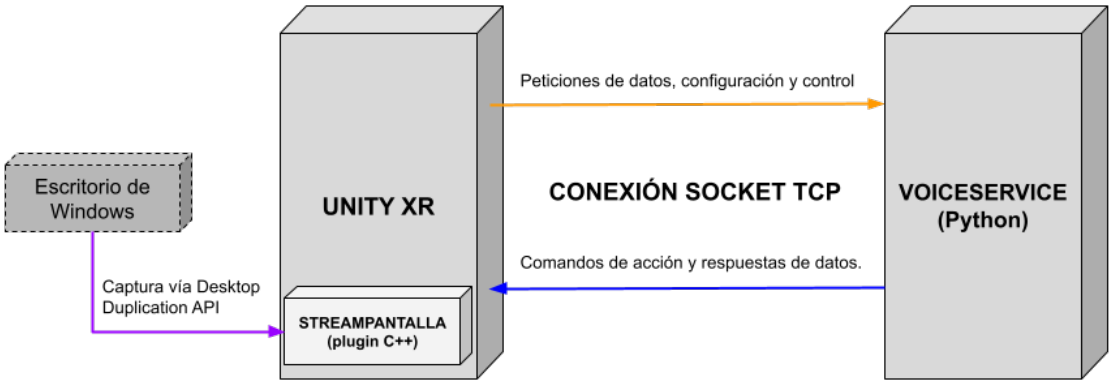


Figura 3.1: Arquitectura general del proyecto: comunicación entre Unity, el módulo de captura de pantalla y el *backend* de control de voz.

En los siguientes subapartados se dará un enfoque general de cada bloque del proyecto.

3.1.1. Aplicación de Unity XR

Este es el componente central de la aplicación y es responsable del renderizado de la escena, de la gestión de los cambios de paneles (inicio, configuración, *streaming* y lista de comandos) de la pantalla principal mediante un controlador de estados, de la integración de *passthrough* y de la gestión del *hand tracking* para las interacciones.

Las tecnologías utilizadas para esta parte del desarrollo han sido, además de Unity, XR Interaction Toolkit para la lógica de interacción (*poke*, *grab*, etc.) y el *hand*

tracking de Meta OpenXR para el seguimiento de manos, que además da los datos a XR Interaction Toolkit para que este interprete los gestos. Asimismo, se ha utilizado XR Origin + OpenXR para el soporte del *passthrough*, la gestión de la cámara y el seguimiento del visor, y Universal Render Pipeline (URP) como *pipeline* de renderizado. A su vez, se ha configurado explícitamente Direct3D 11 como Graphics API para Windows en los ajustes del proyecto, requisito necesario para la compatibilidad con StreamPantalla, el *plugin* nativo para la captura de pantalla.

Como ya se ha mencionado, la interacción principal del proyecto es la comunicación por voz. El usuario puede, entre otros, abrir o cerrar la configuración, aumentar o disminuir el tamaño de la pantalla, o activar o desactivar el *passthrough*, alternando así entre un entorno inmersivo (VR) y uno de realidad mixta (MR) dentro de la misma aplicación.

Esta aplicación de Unity está configurada para ser compilada y ejecutada en Windows, dado que el *plugin* de captura de pantalla tiene que ser ejecutado en la máquina que contiene dichas pantallas. Aunque el programa corre en Windows, la aplicación Meta Quest Link permite visualizar el contenido en el visor conectado al PC.

3.1.2. Plugin nativo StreamPantalla

El *plugin* nativo StreamPantalla es un módulo integrado dentro de la aplicación de Unity, pero implementado en C++, cuya responsabilidad es capturar el escritorio de Windows en cada fotograma mediante Desktop Duplication API y pasárselo a Unity en forma de textura.

Hay dos razones principales por las que el *plugin* está implementado en C++ y no directamente en C# dentro de Unity. Por un lado, la API Desktop Duplication es nativa de Windows, está basada en DXGI y Direct3D 11, y Microsoft no ofrece un equivalente de alto nivel proporcionado directamente para C#. No obstante, se puede acceder a ella desde C# mediante librerías que actúan como capa intermedia. Por otro lado, la implementación en C++ permite acceder directamente a los recursos de Direct3D 11 y realizar operaciones como la copia de los recursos directamente en la GPU. En conclusión, C++ facilita la integración con el sistema de renderizado de Unity y evita capas intermedias adicionales.

3.1.3. Gestión de voz VoiceService

VoiceService es el *backend* independiente programado en Python que gestiona la voz. Este sistema se encarga de capturar el audio del micrófono de forma continua, de detectar la *wake word*, de grabar y transcribir lo escuchado y de resolver el comando pronunciado. Las tecnologías que componen este sistema son openWakeWord para la detección de la *wake word*, Faster-Whisper para la transcripción de voz a texto y RapidFuzz para encontrar similitudes entre la transcripción obtenida y la lista de comandos.

El diseño de esta aplicación se basa en una arquitectura de dos capas. La primera capa es la de un modelo ligero para la detección de *wake word* (openWakeWord), que funciona de forma continua sin consumir muchos recursos. La segunda capa, que se activa solo si detecta la *wake word* (o detecta el *hand pose*, explicado en la sección 3.6.3), es la del modelo de transcripción de audio mucho más pesado. Este diseño que no mantiene el modelo de transcripción activo en todo momento permite reducir el coste computacional y hace que la experiencia se parezca más a las soluciones de reconocimiento de voz que hay comercializadas.

En cuanto a su compilación, la aplicación se empaqueta como un ejecutable independiente para que se facilite, en un futuro, un posible despliegue en una máquina o servidor distintos sin necesidad de cambiar el código fuente (solo haría falta ajustar los parámetros de conexión en el archivo de configuración).

3.1.4. Comunicación entre procesos

Los procesos de Unity y Python son dos procesos independientes, por tanto, necesitan un canal de comunicación para que puedan pasarse información (ver Figura 3.1). El rol de Unity es el de servidor que escucha las conexiones entrantes y espera a que un cliente se conecte, mientras que el rol de VoiceService es el de cliente, conectándose al servidor.

El formato de los mensajes que viajan de un extremo a otro se ha mantenido simple y funcional. La estructura que mejor encajaba para ello era la de mensajes en formato JSON, con campos como `type` para identificar la acción que se quiere ejecutar y campos extra para pasar información necesaria por si hiciera falta (ver código 3.1). Además de los comandos de voz y las peticiones de configuración, este canal se utiliza también para transmitir eventos de *logs* utilizados posteriormente para la evaluación de la aplicación.

```
...
self.client.send_json({
    "type": "configuration",
    "config": self.config.data
})
...
```

Extracto de código 3.1: Llamada a función send_json para enviar la configuración del sistema de Python a Unity

Este proyecto se ha implementado y probado en un entorno donde ambos procesos se encuentran en la misma máquina, pero se ha tenido en cuenta la posibilidad de que, en el futuro, el proceso de Python se ejecute en una máquina distinta, ya que los parámetros de conexión (puerto y servidor) se definen en un archivo de configuración.

Cabe mencionar que durante la implementación surgió un error de sincronización en el arranque, ya que si Python intentaba conectarse al *socket* antes de que Unity hubiera arrancado, el intento de conexión fallaba. Esto se resolvió mediante un bucle

de reintentos, en el que Python se intenta reconectar al *socket* cada segundo (hasta 30 segundos).

3.2. Justificación de las decisiones arquitectónicas

Antes de iniciar la implementación del proyecto y después de analizar las herramientas y soluciones disponibles, se tomaron unas decisiones arquitectónicas iniciales.

Para empezar, se diseñó una arquitectura multiproceso porque openWakeWord y Faster-Whisper proporcionan principalmente interfaces para utilizar en Python, así que si se quisiera integrar directamente en Unity, habría que implementar capas adicionales compatibles con Unity. Además, tener dos procesos independientes permite proteger la fluidez de la escena, es decir, si el proceso de gestión de voz estuviera dentro de Unity y por algún motivo este se cayera o fallara, afectaría al hilo principal de Unity y, por tanto, al *framerate* y a la experiencia de usuario. Sin embargo, el lado negativo de esta decisión es la complejidad añadida de tener que diseñar e implementar la capa de comunicación entre los procesos.

Como se ha explicado en la sección 2.3, se optó por Desktop Duplication API frente a GDI/BitBlt y Windows.Graphics.Capture como sistema para capturar la pantalla en StreamPantalla. La decisión se tomó porque permite obtener los *frames* directamente en la memoria de la GPU y favorece la transferencia GPU-GPU, a diferencia de GDI (basado en CPU) y de las limitaciones de personalización e indicadores visuales de Windows.Graphics.Capture.

La gestión de voz se implementó en Python porque openWakeWord y Faster-Whisper son librerías que se mantienen activamente. Además, reutilizar modelos ya entrenados y librerías probadas facilitó mucho el proceso y evitó implementaciones o reentrenamientos de modelos de detección de *wake word* y transcripción de comandos.

Como se ha explicado en la sección 2.6, se eligió *sockets* TCP para la comunicación entre procesos. Esta decisión se tomó para no limitar el proyecto y prever un futuro en el que el programa de Python se pudiera ejecutar en otra máquina o servidor. Además, TCP está soportado de forma nativa tanto en C# como en Python, y su fiabilidad orientada a la conexión garantiza que los mensajes no se pierdan ni lleguen desordenados. En cuanto al formato de los mensajes, el formato JSON es adecuado porque permite entender los atributos y sus valores casi sin necesidad de conocimientos específicos.

3.3. Flujo de datos de extremo a extremo

El objetivo de este apartado es dar un ejemplo de un caso completo, paso por paso, tomando como referencia el flujo de extremo a extremo del comando 'Aumenta el tamaño', representado en las figuras 3.2, 3.3 y 3.4. La figura del flujo principal (ver figura 3.2) presenta el caso de éxito, en el que la *wake word* se detecta correctamente y el comando se detecta al primer intento. La figura del falso positivo (ver figura 3.3)

representa el caso en el que el valor de RMS es menor que un valor definido y, por tanto, se descarta y no se continúa con la ejecución normal. Por último, tenemos el flujo en el que la *wake word* se detecta pero el sistema no entiende el comando que ha transcrito (ver figura 3.4), por tanto, se avisa a Unity y este se ocupa de reproducir el audio correspondiente para comunicárselo al usuario.

3.3.1. Flujo principal

El flujo comienza con el micrófono y con el modelo de openWakeWord activados, preparados para la detección de la *wake word*. Una vez se ha detectado, Python pide a Unity que ejecute la función `PlayAudio`, que se encarga de reproducir un audio que dice '¿Sí?'. Este audio hace entender al usuario que el sistema está preparado para escuchar. Una vez el usuario habla y se detecta el comando en el audio, Faster-Whisper lo transcribe y, con el porcentaje de similitud que devuelve RapidFuzz, se encuentra el comando correspondiente en el listado. Una vez identificado, se genera el JSON y se envía a Unity, para que este ejecute su función correspondiente y actualice la escena dependiendo de lo que se pida.

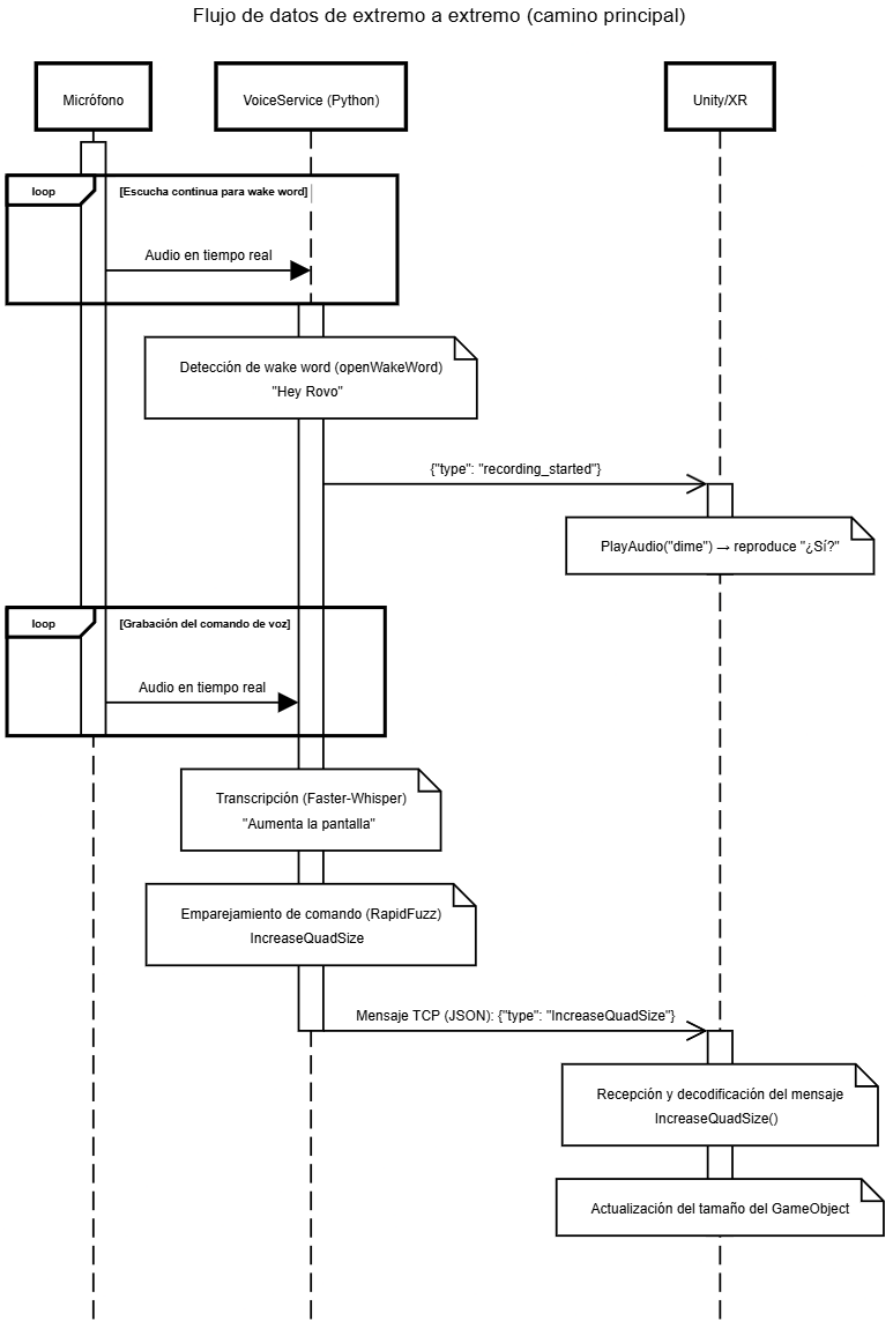


Figura 3.2: Flujo de datos de extremo a extremo principal entre los procesos independientes de Python y Unity.

3.3.2. Flujo con *wake word* descartada por falso positivo

Como ya se ha mencionado, la solución de openWakeWord no es precisa al cien por cien, por tanto, es importante explicar el flujo cuando la *wake word* es detectada por el sistema, pero se trata de un falso positivo.

Si la *wake word* es detectada, se mide su RMS y se compara con un valor definido en el código, en este caso, 30. Este se eligió probando diferentes valores hasta encontrar

uno que bloqueara los falsos positivos generados por ruido externo sin descartar, en la medida de lo posible, los casos positivos. Si el valor es menor que 30, la detección se descarta y se sigue escuchando. Sin embargo, si el RMS es mayor que 30, se continúa con el flujo normal visto en la figura 3.2.

Este valor se determinó de forma empírica, probando distintos umbrales hasta encontrar uno que bloqueara el ruido de fondo y las conversaciones ajenas sin descartar activaciones válidas.

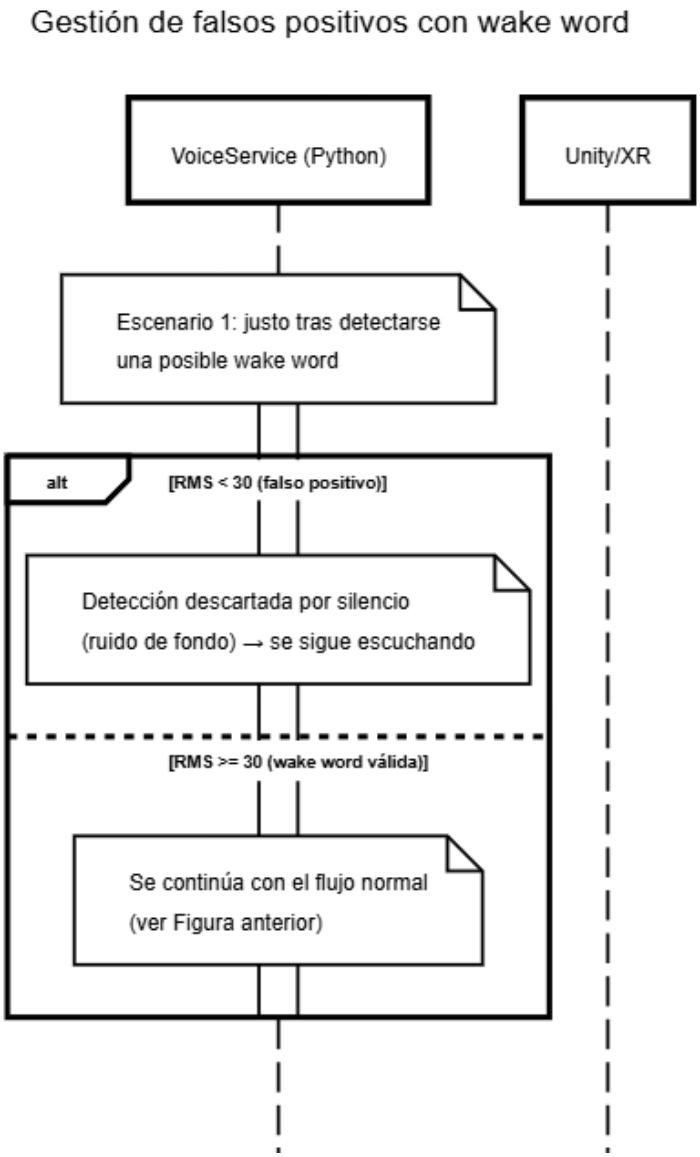


Figura 3.3: Flujo de datos de extremo a extremo para casos en los que se descarta la *wake word* por falso positivo.

3.3.3. Flujo con comando no reconocido

El último caso problemático que falta por analizar es el del comando no reconocido. Tras la transcripción, se configura el sistema para intentar detectar el comando

dos veces. Se implementó esta limitación porque cuando se empezó con el proceso de pruebas se encontraron casos en los que el sistema quedaba encerrado en un bucle en el que la aplicación intentaba detectar el comando una y otra vez. Para evitar casos en los que el usuario dice un comando y el sistema no lo encuentra, se implementó la limitación de dos intentos. En cada intento, si el comando se reconoce, se sigue el flujo principal, pero si no, se avisa a Unity para que este reproduzca un audio que avisará al usuario de que no se ha encontrado el comando correspondiente. En el primer intento, el audio que se reproduce es el que avisa al usuario de que no se ha entendido y que lo vuelva a repetir. Si es el segundo intento, el audio que se reproduce comunica que el comando no se ha encontrado. Después de reproducir cualquiera de los dos mensajes, se avisa a Python de que el audio ha terminado y que debe activar el micrófono otra vez. Si el comando no se reconoce en ninguno de los casos, se vuelve a la escucha activa de la *wake word* y el proceso vuelve a comenzar.

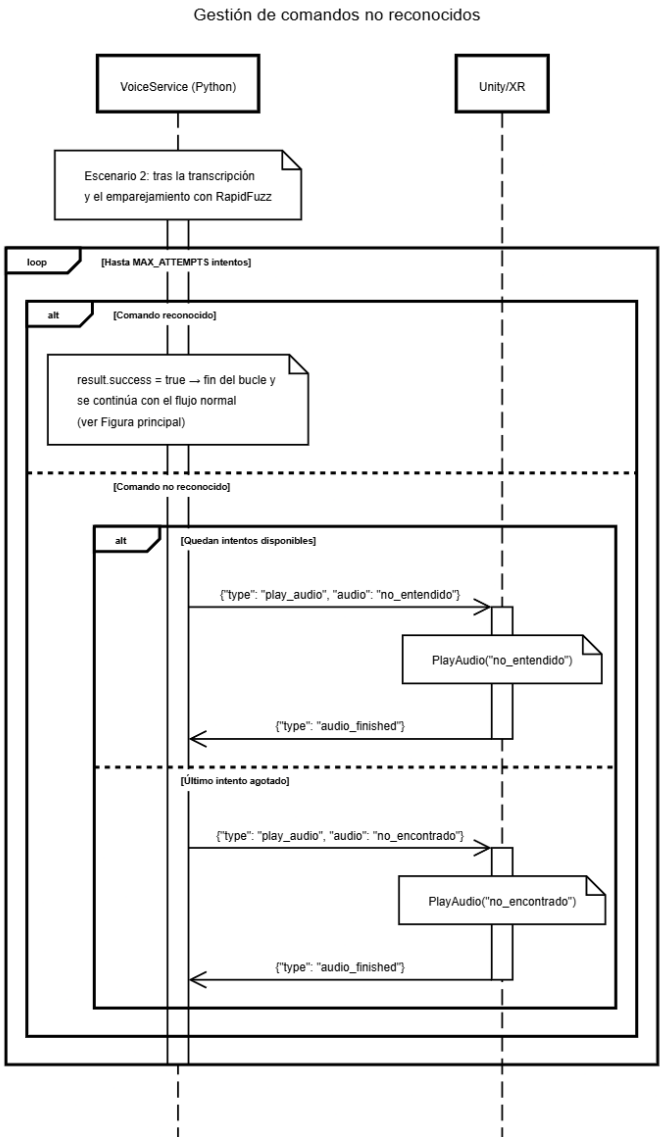


Figura 3.4: Flujo de datos de extremo a extremo para casos en los que no se encuentra el comando.

3.4. Despliegue físico

El despliegue físico de un sistema indica la máquina donde se ejecuta cada bloque de software. En este caso, se ha resumido en la figura 3.5.

Como se puede observar, todos los componentes desarrollados se ejecutan en un PC de Windows, mientras que el visor Meta Quest está conectado al PC mediante Meta Quest Link, ya sea por cable o de forma inalámbrica mediante Air Link.

Esta configuración implica que todo el sistema se ejecuta de manera local y sin depender de una conexión a la red. La comunicación entre Unity y VoiceService se mantiene en *localhost*, y la única conexión que sale del PC es la establecida con el visor a través de Meta Quest Link.

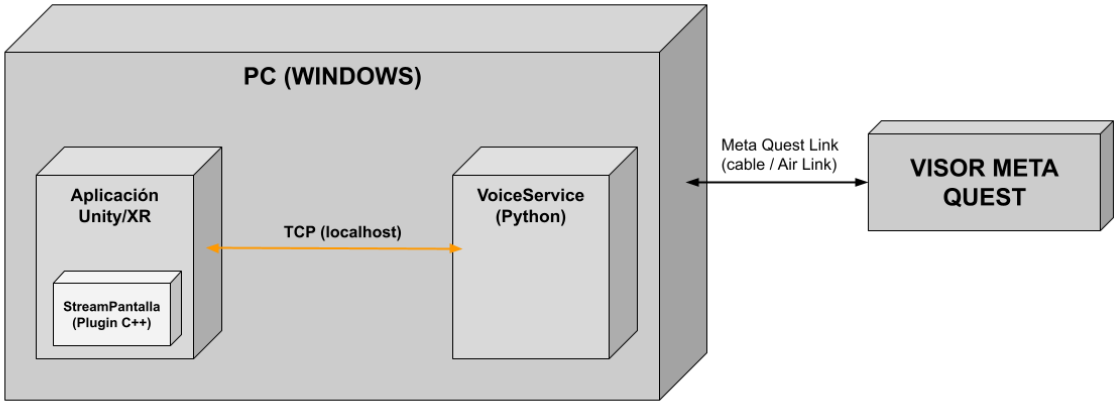


Figura 3.5: Diagrama de despliegue físico: distribución del software sobre el *hardware*.

3.5. Restricciones del diseño

El diseño actual presenta algunas restricciones que merece la pena explicar.

Por un lado, la implementación del proyecto está pensada para que funcione en Windows. Por ejemplo, Meta Quest Link tiene soporte exclusivo en Windows 10 y 11, y Desktop Duplication API es una tecnología nativa de Windows. Además, el visor debe ser compatible con OpenXR y Meta Quest Link.

En cuanto al entorno de uso, se asume un único operador a la vez. El sistema no está pensado para diferenciar hablantes ni para un uso multiusuario. Además, se asume un entorno relativamente silencioso o con ruido de fondo moderado. Si hay mucho ruido y el sistema no puede distinguir la voz del usuario, puede que los comandos no se transcriban correctamente y el sistema falle. Asimismo, los comandos de voz deben pronunciarse en español, ya que la lista de comandos se ha creado en dicho idioma. Por último, aunque se hayan tenido en cuenta una gran cantidad de frases relacionadas con cada comando, el sistema sigue dependiendo de una lista cerrada, y el usuario debe decir algo razonablemente parecido a una de las frases disponibles. Cabe destacar que

se ha incluido una ventana de comandos para que el usuario pueda tener a simple vista las frases (una opción por cada comando) y los comandos disponibles.

En conjunto, estos bloques conforman un sistema dividido en varios procesos que colaboran para ofrecer una experiencia controlada por voz. En los siguientes apartados se detalla la implementación de la aplicación de Unity XR, el *plugin* nativo StreamPantalla, el *backend* VoiceService, la capa de comunicación que conecta voz con Unity, el despliegue físico del sistema completo y por último los cambios de código llevados a cabo para recoger datos de la ejecución.

3.6. Desarrollo detallado

3.6.1. StreamPantalla

El primer paso para crear un *plugin* nativo para Unity fue crear un nuevo proyecto en Visual Studio 2022 de tipo Dynamic-link Library (DLL). Una vez creado, comenzó la implementación en el archivo `dllmain.cpp`.

Se comenzó por obtener las interfaces de Unity para acceder al *device* y al contexto de Direct3D 11 que Unity utiliza internamente. La carga de dichas interfaces se hace en la función `UnityPluginLoad`, función que Unity llama cuando se carga el *plugin*. Esta recibe un objeto de tipo IUnityInterfaces que permite obtener las interfaces de Unity disponibles, y se guarda su valor en una variable global. Después, se obtiene la interfaz de gráficos IUnityGraphics, se comprueba que el *renderer* activo de Unity sea D3D11 (explicado en el apartado 3.1.1), y, si es así, se obtiene la interfaz específica para D3D11. Para terminar, se consiguen el *device* y su contexto y se guardan en variables globales para que el resto del *plugin* pueda usarlos en cada *frame* sin tener que repetir la búsqueda cada vez.

```
extern "C" void UNITY_INTERFACE_EXPORT UNITY_INTERFACE_API
UnityPluginLoad(IUnityInterfaces* unityInterfaces)
{
    g_unityInterfaces = unityInterfaces; //guardar la referencia a las interfaces de
        Unity

    g_unityGraphics = unityInterfaces->Get<IUnityGraphics>(); //obtener la interfaz de
         graficos

    if (g_unityGraphics->GetRenderer() != kUnityGfxRendererD3D11) //comprobar que el
        renderer en Unity sea D3D11
        return;

    IUnityGraphicsD3D11* d3d11 = unityInterfaces->Get<IUnityGraphicsD3D11>(); //
        obtener la interfaz para D3D11

    g_unityDevice = d3d11->GetDevice(); //conseguir el device ID3D11Device

    g_unityDevice->GetImmediateContext(&g_unityContext); //guardar el conexto para
        luego ejecutar copias GPU-GPU
}
```

Extracto de código 3.2: Implementación simplificada de UnityPluginLoad

La idea general de la siguiente parte está implementada en la función `InitializeDesktopDuplication`. Su meta es partir del *device* de Direct3D 11 obtenido al inicio y llegar al objeto que permite capturar el escritorio. Para conseguirlo, hay que ir bajando de nivel en una cadena de conversiones. Para empezar, desde el *device* de Unity se llega al *device* de DXGI con QueryInterface (esto es posible porque Direct3D está construido por encima de DXGI), y desde ahí se continúa hasta conseguir el *adapter* que representa la GPU, el monitor conectado, y la interfaz IDXGIOutput1 que permite duplicar el escritorio (IDXGIOutput no tiene la función necesaria). Por último, se duplica el contenido pasándole como argumento `g_UnityDevice` para que lo utilice (el *device* de Unity que se ha inicializado al principio). Gracias a esto, se podrá copiar el *frame* capturado a la textura de Unity con una simple copia GPU-GPU. Es importante mencionar que esta cadena no ha sido una decisión de diseño, sino que se ha adaptado a cómo está construida la API de Desktop Duplication.

```
extern "C" __declspec(dllexport)
int InitializeDesktopDuplication()
{
    if (g_duplication != nullptr) //comprobar si el proceso de duplicacion ha se ha
        inicilizado
        return 1;

    IDXGIDevice* dxgiDevice = nullptr; //interfaz DXGI asociada al device ID3D11Device
         de Unity
    IDXGIAdapter* adapter = nullptr; //el adaptador (GPU) fisico
    IDXGIOutput* output = nullptr; //salida del monitor
    IDXGIOutput1* output1 = nullptr; //salida que expone DuplicateOutput, necesario
        para duplicar la pantalla

    HRESULT hr;

    hr = g_unityDevice->QueryInterface(__uuidof(IDXGIDevice),reinterpret_cast<void
        **>(&dxgiDevice)); //convertir el device de Unity (D3D11) a un device DXGI

    hr = dxgiDevice->GetAdapter(&adapter); //obtener el adaptador (GPU) desde el
        device DXGI

    hr = adapter->EnumOutputs(0, &output); //Obtener el primer monitor

    hr = output->QueryInterface( __uuidof(IDXGIOutput1), reinterpret_cast<void**>(&
        output1)); //convertir el IDXGIOutput a IDXGIOutput para tener acceso a
        DuplicateOutput

    hr = output1->DuplicateOutput(g_unityDevice, &g_duplication); //Crear la interfaz
        para la duplicacion de \textit{frames}

    return 1;
}
```

Extracto de código 3.3: Implementación simplificada de InitializeDesktopDuplication

La siguiente función que se implementó fue `SetTargetTexture`, la encargada de asignar el puntero a la textura de Unity en la variable global `g_unityTexture`. Esta función se llama desde Unity, pasándole como parámetro la dirección de la memoria donde está el recurso Direct3D 11 que hay detrás de una textura (Texture2D) del *quad* que representa la pantalla en Unity.

```
extern "C" void UNITY_INTERFACE_EXPORT UNITY_INTERFACE_API
SetTargetTexture(void* texture)
{
    g_unityTexture = reinterpret_cast<ID3D11Resource*>(texture);
}
```

Extracto de código 3.4: Implementación de SetTargetTexture

La función `OnRenderEvent` se encarga de obtener el siguiente *frame* disponible y de copiarlo en la textura que Unity ha pasado anteriormente y se ha guardado en la variable global `g_unityTexture`. El siguiente *frame* se obtiene con la función `AcquireNextFrame` con un tiempo de espera de 0, para que devuelva el *frame* disponible en ese instante. Después se analiza su resultado: primero, si no hay un *frame* disponible (DXGI_ERROR_WAIT_TIMEOUT), se termina la función sin errores. Por otro lado, si se obtiene DXGI_ERROR_ACCESS_LOST, significa que el acceso se ha perdido (cambio de usuario o modo de pantalla, *driver* reiniciado, etc.), se libera la duplicación y se marca que se necesita reiniciar desde el lado de Unity. Si todo va bien, se empieza el proceso de obtener la textura D3D11. `AcquireNextFrame` devuelve un IDXGIResource, pero para poder copiarlo con las funciones de Direct3D 11 se necesita convertirlo a un ID3D11Resource. Por último, se copia el *resource* a la textura de Unity dentro de la misma GPU.

```
static void UNITY_INTERFACE_API
OnRenderEvent()
{
    DXGI_OUTDUPL_FRAME_INFO frameInfo = {};

    IDXGIResource* desktopResource = nullptr;

    HRESULT hr = g_duplication->AcquireNextFrame(0, &frameInfo, &desktopResource); //
        obtener siguiente frame

    if (hr == DXGI_ERROR_WAIT_TIMEOUT) //Si no hay frame disponible en este instante,
        salir
    {
        return;
    }

    if (hr == DXGI_ERROR_ACCESS_LOST) //si el acceso se pierde, diberar la duplicacion
         y marcar que se necesita reiniciar desde Unity
    {
        ReleaseDesktopDuplication();
        g_needsDuplicationRestart = true;
        return;
    }

    ID3D11Resource* desktopTexture = nullptr;

    hr = desktopResource->QueryInterface(__uuidof(ID3D11Resource), reinterpret_cast<
        void**>(&desktopTexture)); //obtener la interfaz ID3D11Resource para poder
        utilizarlo con el conexto D3D11 de Unity.

    g_unityContext->CopyResource(g_unityTexture, desktopTexture); //copiar el
        contenido de la textura en la textura de Unity por GPU

}
```

Extracto de código 3.5: Implementación simplificada de OnRenderEvent

Es importante tener en cuenta casos en los que el sistema puede fallar, para así no afectar al flujo de la experiencia ni bloquear Unity. Por un lado, como se ha mencionado antes, cuando se empieza el proceso de duplicación de *frames* puede que `AcquireNextFrame` devuelva DXGI_ERROR_ACCESS_LOST, lo que significa que el acceso al flujo de *frames* se ha perdido (si cambia la resolución de la pantalla, si el *driver* de GPU se reinicia, etc.). Si esto pasa, el objeto de duplicación ya no sirve y hay que crear uno nuevo desde cero. Para controlar este caso está el *flag* `g_needsDuplicationRestart`, que si su valor es *true*, en la función `Update` de Unity se llama a la función `RestartDesktopDuplication`, que se encarga de descartar el objeto

de duplicación y de crear uno nuevo, como se explica en la sección 3.6.3.

Al final, el proyecto DLL compilado se insertó en la carpeta Assets/Plugins/x86_64 dentro del proyecto de Unity. De este modo, el *plugin* queda integrado y puede ser utilizado directamente.

3.6.2. VoiceService

En este apartado se va a detallar la implementación de la aplicación VoiceService. La explicación se va a dividir en las fases de desarrollo (entrenamiento de *wake word*, arranque e inicialización, etc.), y no tanto en los archivos independientes.

Entrenamiento de la *wake word* en Google Colab

Como openWakeWord es un modelo de inteligencia artificial, los ordenadores personales convencionales no son capaces de entrenarlo por limitaciones del *hardware*. Aun así, Google ofrece una plataforma llamada Google Colab, que permite a desarrolladores con una cuenta de Google utilizar servidores para entrenar o utilizar modelos disponibles en internet.

La meta de utilizar Colab para este proyecto fue entrenar openWakeWord para que pudiera detectar la frase 'Hey Rovo'. El repositorio de openWakeWord ofrece un archivo llamado `automatic_model_training_simple.ipynb`, que permite entrenar su modelo para un *wake word* específico [46]. Sin embargo, este archivo presenta errores relacionados con la compatibilidad entre paquetes como `piper-phonemize`, `torchaudio`, etc. y la versión de Python de Colab, y, por tanto, este *script* no se puede utilizar tal cual está. Pese a ello, existe un repositorio de la comunidad con un *notebook* parcheado y actualizado en 2026 (`train_wakeword.ipynb`) pensado para evitar los problemas de compatibilidad que da el archivo original [69]. Este archivo fue el que se utilizó para crear el modelo que detecta la *wake word* 'Hey Rovo'. Como el código estaba preparado para la ejecución, solamente se tuvo que cambiar la frase con la que el modelo tenía que ser entrenado, como se ve en código 3.6.

```
...
TARGET_PHRASE = ['Hey Rovo']
MODEL_NAME    = 'hey_rovo'
...
```

Extracto de código 3.6: Código editado en el archivo `train_wakeword.ipynb`

Una vez se terminó la ejecución, que duró unas horas, se descargó automáticamente el modelo entrenado, `hey_rovo.onnx`, listo para añadirlo a la carpeta 'models' en VoiceService.

Arranque e inicialización

El punto de inicio de esta aplicación está en el archivo `main.py`, en el que se instancia la clase VoiceService y se llama a su función `run`.

La clase VoiceService es el motor principal del sistema. Su constructor inicializa los atributos globales y el micrófono, carga la configuración y el modelo de *wake word*, y se conecta al *socket* TCP que ha activado Unity como servidor. Además, el constructor también se ocupa de inicializar la clase SpeechEngine, cuya responsabilidad es cargar el modelo de Faster-Whisper con los valores de tipo de modelo (`'model'`), dispositivo (`'device'`), idioma (`'language'`) y precisión numérica de la inferencia (`'compute_type'`) definidos en la configuración (para acceder al JSON completo de la configuración, ver anexo A.2). Asimismo, se inicializa la clase CommandClassifier, que en su constructor carga el archivo de comandos `commands.json` y guarda el umbral de similitud (`'threshold'`) que se ha pasado al constructor como parámetro. Este umbral se utilizará para decidir si una transcripción coincide con algún comando. Por último, se inicializa la clase WakeWordEngine para detectar la *wake word*, pasándole la ruta al modelo entrenado y con un umbral de confianza de 0,5. Este valor se ha elegido probando distintas opciones y eligiendo la que permitía detectar correctamente la *wake word* sin aceptar falsos positivos.

Una vez que se han inicializado todos los subsistemas, el constructor establece la conexión TCP con Unity a través de TcpClient, y define qué función se llamará cuando se reciban mensajes entrantes en el *socket*.

La clase Config implementa funciones relacionadas con el archivo de configuración, pero todavía no se han explicado en profundidad. Para empezar, la función `save` se encarga de volcar los datos de la clase Config a config.json, utilizando la función `json.dump` configurada para que se acepten carácteres con tildes y eñes. Esta función de volcado se llama desde la otra función de esta clase, `change_configuration`. Su responsabilidad es, pasándole un *tag* y un valor, actualizarlo tanto dentro de la clase como en el archivo config.json.

En cuanto a los atributos de config.json (ver anexo A.2), a excepción del micrófono y sample rate, los demás valores han sido fruto de decisiones que se han ido tomando durante la implementación. Dentro de `'classifier'`, el valor de `'threshold'` es 85, que representa el porcentaje de acierto del clasificador para que una transcripción se acepte como comando válido. Al igual que otros umbrales de confianza, este también se ha ajustado haciendo pruebas y buscando un límite donde la tasa de acierto es lo más alto posible. En el bloque `'whisper'` se define la configuración del modelo de Faster-Whisper. El `'model'` elegido es `'base'`, un modelo intermedio de Whisper que ofrece un balance entre precisión y velocidad de transcripción, lo cual es necesario porque el sistema transcribe en tiempo real. Para `'language'` se ha elegido el español, ya que todos los comandos se han definido en este idioma. Por otro lado, el `'device'` se ha configurado como `'cpu'` para evitar las dependencias de hardware de Faster-Whisper (para utilizar la GPU se necesita una GPU NVIDIA con las librerías cuBLAS y cuDNN instaladas [58]) y `'compute_type'` se ha fijado a `'int8'` porque es la precisión que se recomienda en la configuración para CPU. Para terminar, el bloque (`'unity'`) representa la conexión con Unity mediante un `'host'` (`'127.0.0.1'`) y un `'port'` (5005). Se ha elegido la IP local porque los dos procesos se ejecutan en la misma máquina, y se ha elegido un puerto que estaba disponible en el sistema.

Ya se ha explicado en detalle la construcción de la clase VoiceService y la función de las llamadas a otras clases, pero falta por detallar cómo funciona el flujo principal,

en este caso, la función `run`. Esta llama a `start_wake_word_listener`, que se encarga de lanzar el hilo de escucha de la *wake word* utilizando el método `wake_word_loop`, el cual se explica con detalle en el siguiente subapartado.

Captura de audio y detección de la *wake word*

Antes de detallar la función `wake_word_loop`, es importante tener un contexto más general de los sistemas relacionados con la detección de la *wake word* y la captura de audio. La única función de la clase WakeWordEngine (`process_frame`) tiene como responsabilidad procesar un audio y devolver si la *wake word* se identifica en ese audio o no. Por otro lado está el sistema de captura de audio, que se define e implementa en la clase Recorder. En este archivo se introducen dos atributos que se utilizan en numerosas funciones: `SAMPLE_RATE = 16000` para la frecuencia de muestreo que exige Faster-Whisper, y `NATIVE_SAMPLE_RATE = 48000` para la frecuencia del *hardware* de los visores Quest. En esta clase se definen dos funciones principales para dos modos de captura de audio, `record` y `open_stream`. Por un lado, el método `record` captura un clip de duración fija, y se llama cuando el sistema tiene que escuchar un comando a ejecutar. Sin embargo, `open_stream` abre el micrófono y escucha un bloque de 80 ms, y se utiliza para la detección de la *wake word*. Este tamaño de bloque se ha definido porque, por un lado, openWakeWord recomienda que los fragmentos de audio sean múltiplos de 80 ms y, por otro lado, los bloques más pequeños son más caros de procesar y los más grandes son más lentos en detectar la *wake word* [46].

Cabe destacar que mientras se graba un comando, se deja de escuchar la *wake word*.

Con el contexto más detallado, se va a explicar la función `wake_word_loop` de `VoiceService` (ver código 3.7). Su función principal es mantener una escucha activa para detectar en tiempo real si se ha pronunciado la *wake word*. Para ello, el bucle lee el audio en bloques de 80 ms. Como no todos los micrófonos graban en la misma frecuencia, el número de muestras leídas en cada bloque se calcula a partir de la frecuencia del micrófono conectado para que siempre se tomen 80 ms de audio. Antes de pasar cada bloque al modelo, se convierte a *float32*, y si el dispositivo no graba a 16 kHz, se remuestrea a esa frecuencia para que sea compatible con Faster-Whisper. Después se procesa cada frame de audio con la función `process_frame` de WakeWordEngine, la cual decide si contiene la *wake word* o no. En cada iteración también se comprueba si la activación se ha hecho desde Unity mediante el *hand pose*, por tanto, esta función gestiona ambos modos de activación. Cuando el modelo detecta la *wake word*, se calcula el RMS del *frame* y se compara con `MIN_ENERGY_RMS` antes de aceptar la detección, ya que puede ser un falso positivo. Si el RMS supera el valor definido, se pasa a la segunda capa de ejecución, encargada de la escucha y transcripción del comando (`execute_command_pipeline`).

```
def wake_word_loop(self):
        self._wake_stream = self.recorder.open_stream(device=self.
            device_index)
        device_sample_rate = self.recorder.get_device_sample_rate(self
            .device_index)
```

```
native_block_size = int(1280 * device_sample_rate / Recorder.
   SAMPLE_RATE)
MIN_ENERGY_RMS = 30

while self._listening:
    if self._manual_trigger_event.is_set():
        self._manual_trigger_event.clear()
        self.execute_command_pipeline()
        continue

    frame, _ = self._wake_stream.read(native_block_size)
    frame_flat = np.asarray(frame, dtype=np.float32).reshape
       (-1)
    if device_sample_rate != Recorder.SAMPLE_RATE:
        frame_flat = Recorder.resample_to_target(frame_flat)

    detected = self.wake_word.process_frame(frame_flat)

    if detected:
        rms = np.sqrt(np.mean(frame_flat.astype(np.float32) **
            2))
        if rms < MIN_ENERGY_RMS:
            continue
        self.execute_command_pipeline()
```

Extracto de código 3.7: Implementación simplificada de wake_word_loop

Transcripción y cálculo de similitud con RapidFuzz

La función principal encargada de la transcripción (y del cálculo de la similitud con RapidFuzz) es `execute_command_pipeline` (ver código 3.8). Cuando se empieza el proceso, la función avisa a Unity de ello para que este reproduzca el audio '¿Sí?' y así el usuario entienda que puede decir la acción a ejecutar. Después, se detiene el *stream* de la *wake word* para evitar que continúe intentando detectarla mientras el usuario pronuncia el comando. Como se ha mencionado en el apartado 3.3.3, el sistema intenta entender el comando un máximo de dos veces por cada vez que se activa. En cada iteración se graban 3 segundos de audio y se transcribe con la función `transcribe` de SpeechEngine, en la cual se pasa el audio al modelo Faster-Whisper y se devuelve la frase en un *string*.

Una vez que la transcripción está completa, el siguiente paso es clasificar el texto y buscar su comando correspondiente. Para ello, se llama a la función `classify`, que se encarga de comparar el texto con todos los comandos y guardar el que mejor puntuación haya obtenido. Además, se compara el porcentaje del comando más parecido con el umbral guardado en el apartado `classifier.threshold` de config.json. Si la puntuación obtenida con RapidFuzz es mayor que el umbral, se acepta el comando y se devuelve un objeto que contiene la frase original, la frase con la que se ha hecho la coincidencia, el comando de dicha frase y el porcentaje de similitud. En este caso, se aborta el *loop* que controla la cantidad de intentos, y se avisa a Unity de que debe ejecutar la función correspondiente al comando. Sin embargo, en el caso en que el comando no se encuentra, se avisa a Unity de ello para que reproduzca el audio respectivo.

El listado completo de las frases asociadas a cada comando puede consultarse en el anexo A.1.

```
def execute_command_pipeline(self):
    self.client.send_json({"type": "recording_started"})
    if self._wake_stream is not None:
        self._wake_stream.stop()
        self._wake_stream.close()

    for attempt in range(1, self.MAX_ATTEMPTS + 1):

        audio = self.recorder.record(seconds=3, device=self.
           device_index)

        text = self.engine.transcribe(audio)

        result = self.classifier.classify(text)

        if result.success:
            break

        if attempt < self.MAX_ATTEMPTS:
            self._play_audio_and_wait("no_entendido")
        else:
            self._play_audio_and_wait("no_encontrado")

    if result.success:
        unity_command = UnityCommand(command=result.command, text=
           result.original_text,score=result.score)
        self.client.send(unity_command)
    else:
        unity_command = UnityCommand(command="None", text=result.
           original_text, score=result.score)
        self.client.send(unity_command)
```

Extracto de código 3.8: Implementación simplificada de execute_command_pipeline

Comunicación TCP con Unity

Para conectar el sistema de Python al *socket* TCP se desarrolló una clase TcpClient. Este trabaja como cliente TCP que se conecta al servidor que abre Unity, y gestiona en dos direcciones el envío de mensajes salientes y la recepción continua de mensajes.

Cuando la aplicación se ejecuta, este intenta conectarse al *socket* abierto de Unity. Sin embargo, no hay garantía de que la aplicación de Unity esté escuchando cuando VoiceService se ejecuta, así que este reintenta hasta 30 veces con un segundo de espera entre intentos hasta conectarse. El tiempo específico de 30 segundos se eligió después de analizar el tiempo que tardaba Unity en cargarse e inicializar el servidor del *socket*, dejando un margen de seguridad para cubrir posibles arranques más lentos. En el caso de que no se conecte, la aplicación lanza un error y se termina.

El hilo dedicado a recibir mensajes de Unity se ejecuta en segundo plano porque

leer del *socket* es una operación bloqueante, es decir, si se hiciera en el hilo principal, el programa se quedaría esperando el siguiente mensaje de Unity en vez de escuchar la *wake word* o procesar comandos. Este hilo secundario se inicializa utilizando `Thread` de la librería `threading` con el atributo `daemon` en verdadero, como se ve en el código 3.9.

```
self.receive_thread = threading.Thread(
    target=self._receive_loop,
    daemon=True
)
```

Extracto de código 3.9: Llamada al constructor de la clase Thread para crear un hilo en segundo plano

Cabe mencionar que tanto en el envío como en la recepción de los mensajes, `TcpClient` no interpreta los mensajes, sino que se dedica a ser una capa intermedia entre el *socket* y la clase VoiceService.

La interpretación de los mensajes recibidos se hace en la función `handle_unity_message` en la clase VoiceService. Esta función se encarga de analizar el mensaje recibido y de ejecutar su acción correspondiente. Para ver qué tipo de mensaje se ha recibido, se lee el atributo 'type' del JSON y se compara con las opciones disponibles. Por ejemplo, si el mensaje que llega desde Unity tiene el tipo 'get_microphones', Python obtiene la lista de micrófonos disponibles y busca el seleccionado mirando la configuración. Luego, manda esa información a Unity mediante el *socket*. Esta parte de la función se puede ver en el código 3.10.

```
def handle_unity_message(self, message):
    try:
        data = json.loads(message.lstrip("\ufeff"))
        message_type = data.get("type")

        ...

        elif message_type == "get_microphones":
            devices = Recorder.get_input_devices() #Obtener los
                microfonos disponibles
            selected_index = -1

            for i, device in enumerate(devices):
                if device["name"] == self.config.microphone: #
                    Encontrar el seleccionado
                    selected_index = i
                    break

            self.client.send_json({"type": "microphones", "devices":
                devices, "selected_index": selected_index}) #enviar el
                 JSON a Unity

        ...

    except Exception as e:
        print(f"Error procesando mensaje de Unity: {e}")
```

Extracto de código 3.10: Gestión del mensaje de Unity para obtener los micrófonos disponibles

3.6.3. Unity XR

El proyecto de Unity se creó utilizando el VR Template que ofrece Unity Editor, con la versión 6000.3.8f1. Una vez creado, se continuó con la instalación de paquetes necesarios para el desarrollo y con la preparación del entorno.

Contexto técnico

Dentro de OpenXR se han utilizado OpenXR Plugin, Unity OpenXR Meta y XR Plugin Management configurados para la plataforma de destino PC (Windows), ya que la aplicación se conecta con el visor a través de Meta Quest Link, como se ha comentado en la sección 3.1.1. Para la interacción este proyecto ha utilizado XR Interaction Toolkit, XR Hands e Input System. El *passthrough* se ha implementado utilizando AR Foundation, ya que permite activarlo mediante el componente AR Camera Manager que se ha añadido a la cámara principal del XR Origin. En cuanto al *render* y elementos visuales, el proyecto utiliza Universal Render Pipeline (URP), incluido por defecto en la plantilla VR de Unity. Por otro lado, para los elementos de UI del panel de configuración (*scroll* de la pantalla, *dropdowns*, etc.) se ha utilizado el sistema uGUI.

Cabe mencionar que, para que la duplicación de pantalla con StreamPantalla funcionase correctamente, se cambió el Graphics API del proyecto a Direct3D 11 en los ajustes de Unity, ya que el *plugin* nativo es compatible con el dispositivo Direct3D del motor (ver el apartado 3.6.1).

Organización general

El componente más importante de la aplicación de Unity es el XR Origin Hands (XR Rig), añadido a la escena desde la carpeta de 'samples' de XR Interaction Toolkit. Este *prefab* listo para usar enlaza el visor y las interacciones a la cámara y elementos necesarios para integrar al usuario en la escena, en otras palabras, este es su avatar en la experiencia. Uno de los subcomponentes que tiene es la cámara (Main Camera), que es en el que se incluyó el AR Camera Manager para activar el *passthrough*.

Otro cambio que se ha hecho en el XR Origin Hands ha sido en RightHandQuestVisual y LeftHandQuestVisual, donde se ha añadido un *hand pose* para activar el sistema de comandos manualmente. Para crear el *hand pose*, primero se creó un *hand shape* con los valores de forma de cada dedo. En este caso, al querer detectar la mano abierta, se seleccionó la forma Full Curl con Target a 0 para cada dedo. Una vez definido el *hand shape*, se creó el *hand pose*, donde el valor de su atributo *Hand Shape* era el recién creado. En este *asset* también se definió que la dirección de la palma de la mano debe mirar a la cámara. Para ello se añadió una condición donde el eje *Palm Direction* de la mano debía alinearse con la dirección hacia la cabeza (*Hand to Head*) con una tolerancia angular de 45º. Ya con el *hand pose* terminado, se añadió el componente Static Hand Gesture a RightHandQuestVisual y LeftHandQuestVisual con el *hand pose*, la referencia a la función a ejecutar cuando este se detecta (`VoiceStarter.StartRecording`) y el tiempo mínimo que hay que mantener la pose para que la función se active (0,2 segundos). Este tiempo se fijó probando valores hasta

encontrar un equilibrio entre evitar activaciones accidentales y evitar que el operario tenga que mantener la pose mucho tiempo, lo cual podría interferir con el control del ROV.

Para controlar los elementos generales del proyecto como cambios de paneles o la gestión de mensajes recibidos desde el *socket* TCP, en este proyecto se han incluido dos Game Objects vacíos: Managers y VozManager.

Managers tiene como componentes dos *scripts* propios, llamados CommandManager y StateManager. Por un lado, StateManager se ocupa del cambio entre paneles de la pantalla. Para ello se definen las variables privadas (`[SerializeField] private`) de tipo `GameObject` para la pantalla de inicio, la de configuración y la de duplicación de pantalla. Una vez asignadas desde el inspector, estas pueden ser activadas y desactivadas desde el *script*. La experiencia empieza con la pantalla de inicio activa, y después de diez segundos (valor elegido de forma aproximada dejando un margen suficiente para que Unity termine de inicializarse) esta pantalla desaparece y se visualiza la de configuración. Además, en ese momento se avisa a Python, a través del *socket* TCP, de que desactive la gestión de comandos por voz para asegurar, antes de continuar, que el usuario elige un micrófono. Una vez el usuario pulsa el botón de Aceptar de dicha pantalla, se llama a la función `AceptarConfiguracion` de este mismo *script*, cuya responsabilidad es desactivar la pantalla actual, activar la pantalla de la duplicación y volver a activar la gestión de voz. Cabe mencionar que mientras la pantalla de configuración esté visible, no es posible moverla utilizando las manos, ya que las interacciones pueden colisionar con las del UI. Por otro lado, el *script* CommandManager es responsable de ejecutar las acciones relacionadas con cada comando. Su función `Execute` recibe un comando de tipo VoiceCommands (enumerador con los nombres de todos los comandos disponibles) y, con un *switch*, define la función de Unity a ejecutar para cada caso.

Por otro lado, al objeto VozManager tiene el *script* VoiceReceiver, que se encarga de abrir el *socket* TCP como servidor, y de enviar y recibir mensajes. Para empezar, en su función `Start` se crea un hilo en segundo plano donde inicializa el servidor con el método `StartServer`. Este se encarga de crear un objeto de tipo TcpListener para abrir el *socket* y aceptar los clientes que se quieran conectar a él. Además, crea un objeto *writer* de tipo `StreamWriter` que se utiliza para volcar tanto los mensajes recibidos como los enviados. Por otro lado, en la función `Update`, que se ejecuta en cada actualización de *frame*, se gestiona la recepción de los mensajes del *socket*. Aquí se comprueban los casos específicos de los mensajes de tipo `play_audio` (reproduce el audio con el nombre que se ha pasado desde VoiceService), `recording_started` (se recibe cuando la grabación del comando empieza para reproducir el audio que dice '¿Sí?'), `microphones` (recibe la lista de micrófonos disponibles, para rellenar el *dropdown* de la configuración) y `configuration` (se recibe la configuración seleccionada). Además, esta función también es la responsable de llamar a `Execute` de CommandManager. Por otro lado, este *script* contiene funciones para enviar mensajes a VoiceService, como por ejemplo `RequestConfiguration` para obtener la configuración, o `StartRecording` para activar el gestor de voz de los comandos. Además, esta clase también se encarga de reproducir los audios adecuados en cada momento.

Los elementos visuales de la escena se pueden dividir en dos bloques principales:

`Pantalla SM` y `Pantalla SM Comandos`. Ambos elementos son GameObjects vacíos con los siguientes elementos: Rigidbody, XR Grab Interactable, XR General Grab Transformer, Lazy Follow y Box Collider. En el Rigidbody las características más llamativas son que la gravedad está desactivada, que `Is Kinematic` es *false* (haciendo que el panel se pueda mover), que el valor de `Linear Damping` es muy alto (para amortiguar la velocidad residual y para que no siga moviéndose al soltarlo), y que el valor de `Angular Damping` es muy bajo (para que la rotación cuando la pantalla se mueve sea más libre). En cuanto al XR Grab Interactable, `Movement Type: Instantaneous` hace que el movimiento se aplique directamente a la posición del objeto y no por física o fuerzas, dando la sensación de seguimiento inmediato a la mano. Además, `Track Position`, `Smooth Position Amount: 5` y `Tighten Position: 0,068` hacen que la posición siga a la mano, pero de manera suavizada y no tan rígida. `Track Rotation`, está desactivado para que el panel no gire con la rotación de la mano, y `Throw On Detach` desactivado hace que al soltar el panel no se aplique ninguna velocidad de lanzamiento. Por último, `Select Mode: Multiple` hace que se pueda interactuar con el elemento con ambas manos a la vez. XR General Grab Transformer permite al usuario girar la pantalla con ambas manos, y Lazy Follow hace que al mover la pantalla por el espacio esta rote para que mire al usuario de frente.

Dentro de Pantalla SM encontramos los tres paneles (*quads*) mencionados anteriormente: Pantalla Inicio, Panel Configuración y Pantalla PC.

Para empezar, Pantalla Inicio es un *quad* con el logo de la empresa EONSEA.

Panel Configuración, en cambio, es un panel obtenido de la plantilla de VR de Unity, que ya venía preparado con un *canvas* con *scroll* y *auto adjust* configurados, además de elementos como *dropdowns* y botones listos para utilizar. El único cambio visual aplicado a este panel ha ido eliminar los elementos que no eran necesarios. En este caso, el contenido que tiene este panel está compuesto por dos *dropdowns*, uno para elegir el micrófono y el otro para elegir el idioma (ahora mismo el sistema solo está preparado para el español, por tanto, este desplegable solo tendrá esa opción). El desplegable de los micrófonos tiene asignado un *script* (DropdownOptionsManager) responsable de obtener los micrófonos desde VoiceService y de actualizar dinámicamente su lista de opciones. Además, también tiene el archivo OptionChanged, que se encarga de guardar la configuración seleccionada en config.json de VoiceService. En el caso del desplegable del idioma no hace falta el DropdownOptionsManager y solo se mantiene el *script* de OptionChanged, ya que las opciones de idiomas disponibles son una lista estática que se ha definido a través de la interfaz. Por último, este panel de configuración tiene un botón de aceptar cuyo su trabajo será llamar a la función `AceptarConfiguracion` de StateManager.

El tercer elemento principal de Pantalla SM es Pantalla PC, que se encarga de retransmitir la duplicación de la pantalla del PC conectado. A este *quad* se le asignó un material sin textura, ya que el *script* ScreenCaptureDisplay se encarga de crear uno y utilizar el *plugin* nativo StreamPantalla para asignárselo. Este *script* es el encargado de la comunicación con el *plugin* y es el punto en el que StreamPantalla (ver sección 3.6.1) se integra en la escena. En la función `Start` de ScreenCaptureDisplay, se inicializa la duplicación de la pantalla, se crea la textura de tipo RenderTexture y se asigna al material del panel. Si se necesita reiniciar el sistema por cierre de sesión, cambio de tamaño,

etc., se gestiona en la función `Update`, donde se llama a `RestartDesktopDuplication` si `NeedsDesktopDuplication` devuelve un valor distinto a 0. Cuando este Game Object se destruye, en la función `OnDestroy` se liberan los recursos relacionados con la duplicación.

Por último, falta mencionar que dentro de Pantalla SM Comandos se encuentra un panel parecido a los anteriores, pero con una lista de comandos que se ha diseñado en un editor externo a Unity (Canva) y exportado como PNG. Se ha creado un material con esa imagen y asignado al *quad* de la pantalla. Como mención especial, esta pantalla se muestra en la parte derecha de la pantalla principal, y aparece cuando el usuario diga el comando correspondiente.

3.6.4. Empaquetado y despliegue

El entregable del proyecto completo es una carpeta con los archivos necesarios. Por un lado, el ejecutable del proyecto de Unity se construye utilizando su proceso de *build*. El resultado es un archivo `.exe` preparado para ejecutarse. Por otro lado, para el proyecto VoiceService también se ha construido su `.exe` correspondiente utilizando PyInstaller, una herramienta diseñada para empaquetar aplicaciones de Python con todas sus dependencias. Sin embargo, para que funcione correctamente, se han añadido los archivos commands.json, config.json y la carpeta de models (donde se guarda el modelo de la *wake word*) en la misma carpeta. Además, esto tiene la ventaja de que cualquiera que ejecute este proyecto tiene acceso a la configuración y a la lista de comandos, y puede cambiar el modelo de *wake word* por uno con otra frase o palabra.

Por último, para facilitar la ejecución de la aplicación, se ha creado un archivo `.bat` que pone en marcha los dos ejecutables.

Para que el proyecto funcione correctamente, hay que cumplir unos requisitos técnicos. El ordenador debe tener Microsoft Visual C++ Redistributable x64 instalado para que el plugin StreamPantalla funcione. Por otro lado, la conexión entre el visor al ordenador se hace mediante Meta Quest Link. Además, antes de ejecutar la aplicación, es recomendable comprobar en la configuración del sistema que el micrófono del visor está activado, para que este pueda captar el audio.

3.6.5. Obtención de datos para los resultados

Para poder cuantificar tiempos y tasas de acierto de la aplicación, se ha implementado un sistema de recogida de datos tanto en la parte de Python como en Unity. Los datos que se obtienen en los archivos se utilizan para analizar el rendimiento y la robustez de la aplicación en el capítulo 4. El desarrollo de este apartado se hizo de forma iterativa, es decir, se implementó primero un sistema que guardara los tiempos de procesamiento de *wake word* y comandos. Después, se quiso investigar el tiempo total del *pipeline* incluyendo la escucha del comando, y desde ahí surgió la idea de calcular también la latencia del *socket* TCP. Por último, se quiso cuantificar el rendimiento de la aplicación y calcular su uso de CPU y memoria para ambos procesos, y FPS en el caso de Unity.

Algunos de los archivos de este sistema de registro se generan en VoiceService, mientras que otros se crean en Unity, dependiendo de los datos que se están recogiendo. Por ejemplo, los datos sobre el rendimiento del sistema de voz se generan y guardan en Python, y los datos relacionados con la transmisión de datos por el *socket* TCP y los de rendimiento de Unity, en Unity. Los cambios de código para añadir los *logs* se han hecho en voice_service.py. En Unity, en cambio, se ha creado un archivo ExperimentLogger.cs que se encarga de rellenar los datos y guardarlos en sus archivos correspondientes. Todos los archivos (tanto los de Python como los de Unity) se guardan en la carpeta Documentos/TFM_Experimentos. En la tabla 3.1 se resume qué mide cada archivo, quién lo escribe (Python o Unity) y con qué frecuencia.

Archivo	**Qué mide**	**Quién**	**Frecuencia**
`procesamiento_wakeword.csv`	Tiempo que tarda el algoritmo de *wake word* en decidir	Python	Cada *frame* de audio analizado
`procesamiento_comando.csv`	Tiempo que necesita Faster-Whisper + el clasificador en reconocer un comando	Python	Por cada intento de reconocimiento
`tiempo_total_pipeline.csv`	Tiempo total del *pipeline*, desde la activación hasta que se envía la respuesta a Unity	Python	Por cada activación
`recursos_python.csv`	Uso de CPU y memoria de Python	Python	Cada segundo
`latencia_tcp.csv`	Latencia de comunicación entre Python y Unity en ambas direcciones	Unity	Por cada mensaje TCP
`recursos_unity.csv`	Uso de CPU y memoria de Unity	Unity	Cada segundo
`fps.csv`	*Frames* por segundo de la aplicación	Unity	Cada segundo

Cuadro 3.1: Resumen de los archivos de registro generados durante la ejecución

Para crear estos archivos con sus correspondientes cabeceras, se han implementado las funciones `asegurar_cabecera_csv` en voice_service.py y `AsegurarCabecera` en ExperimentLogger.cs. La meta de estas funciones es crear el fichero con su correspondiente cabecera en el caso de que no exista en el directorio. En el caso de que el archivo exista, este no se vuelve a crear y los nuevos datos se acumulan en la parte de abajo. Esto permite hacer diferentes sesiones de pruebas sin necesidad de fusionar manualmente los datos ni perder los datos anteriores.

En las siguientes líneas se explicará la implementación de la recogida de datos de cada archivo de la tabla 3.1.

procesamiento_wakeword.csv

La meta de este archivo es recoger los tiempos que le hacen falta al código de *wake word* para decidir en cada *frame* de audio, si la *wake word* se ha dicho o no. Las columnas de este archivo son `'timestamp_unix'`, `'tiempo_procesamiento_ms'` y `'resultado'` (encontrado / no encontrado). Para obtener esos datos, se ha implementado la función `log_procesamiento_wakeword` en voice_service.py. Esta función obtiene como parámetro el tiempo en ms que ha necesitado el sistema, y si la *wake word* se ha detectado o no. En la función se calcula el tiempo actual para la columna `'timestamp_unix'` y formatea el parámetro `'detectado'` a un string. Al final, todos esos datos se escriben en el CSV.

Para calcular el tiempo que necesita el sistema desde que se escucha el *frame* de audio hasta que se procesa, se han utilizado dos variables de tiempo. Una se llena justo después de escuchar el *frame* de audio, mientras que la otra se calcula después de que el modelo de *wake word* haga la predicción y calcule el resultado (detectado o no detectado). El tiempo entre esos dos instantes se calcula restando los tiempos y convirtiéndo el resultado a milisegundos.

Para terminar, se llama a la función `log_procesamiento_wakeword` y se guardan los valores calculados.

procesamiento_comando.csv

Este archivo guarda el tiempo que tardan Faster-Whisper y la clasificación con RapidFuzz en reconocer (o no) un comando. Tiene una línea por cada intento y los datos que se guardan son el timestamp (`'timestamp_unix'`), si es el primer o segundo intento (`'intento'`), el tiempo de procesamiento (`'tiempo_procesamiento_ms'`), el resultado (`'resultado'`), el comando detectado (`'comando'`), el porcentaje de similitud devuelto por RapidFuzz (`'score'`), la frase que ha transcrito Faster-Whisper (`'frase_transcrita'`) y si se ha activado mediante *wake word* o *hand pose* (`'modo_activacion'`).

Para obtener estos datos, se ha editado el código dentro de la función `execute_command_pipeline`, precisamente dentro del *loop* que controla cada intento de reconocimiento del comando. Además, se ha creado la función `log_procesamiento_comando`, que obtiene como parámetro el índice del intento (1 o 2), el tiempo de procesamiento del comando, un objeto de tipo `ClassificationResult` (`'original_text'`, `'matched_phrase'`, `'command'`, `'score'`) y el modo de activación. Para el índice de intento, se utiliza la variable de la estructura `for` directamente. Sin embargo, para calcular el tiempo de procesamiento se obtiene el tiempo justo después de grabar el audio y antes de transcribirlo y clasificarlo, y se resta y se convierte a milisegundos. Para el objeto de tipo `ClassificationResult`, simplemente se pasa el resultado de la función `classify` de la clase CommandClassifier. Por último, para obtener el modo de activación se añadió un parámetro 'modo' a la función `execute_command_pipeline`, así cuando se llama a dicha función desde la activación por *wake word* se da el valor `modo="wake_word"`, y en el caso contrario `modo="manual"`. Como último paso, la función `log_procesamiento_comando` calcula el tiempo actual, convierte la variable del

resultado 'success' de booleano a string (encontrado/no encontrado) y escribe los datos en el archivo.

tiempo_total_pipeline.csv

La función de este archivo es calcular el tiempo total que necesita el sistema para escuchar, transcribir y clasificar el comando, es decir, calcula el tiempo desde que se activa el sistema de voz (a través de *wake word* o *hand pose*) hasta que el mensaje con el comando correspondiente se envía a Unity. Es importante destacar que este no mide el tiempo que el sistema necesita para mandar el mensaje por el canal de TCP.

Las columnas de este archivo son `'timestamp_unix'`, `'tiempo_total_ms'`, `'intentos'` (cantidad de intentos requeridos), `'resultado'` (encontrado / no encontrado) y `'modo_activacion'`.

Este código también se ha desarrollado en el lado de Python, y se ha mantenido la estructura de los demás archivos explicados hasta ahora. Por un lado está la función `log_tiempo_total`, que obtiene como parámetro el tiempo en milisegundos, la cantidad de intentos utilizados, si el comando se ha encontrado y el modo de activación que se ha utilizado. Para calcular el tiempo, se resta el tiempo de inicio del *pipeline* (registrado justo después de la activación) al tiempo del instante en que el mensaje se envía a Unity. Para la cantidad de intentos, se accede directamente a la variable del `for` (aunque la llamada de `log_tiempo_total` esté fuera del for, Python permite acceder a dicha variable para saber a qué valor ha llegado en el *loop*). Para el valor del parámetro de si el comando se ha encontrado o no, se accede directamente a la variable success del resultado de la función `classify`, como se ha hecho para el archivo procesamiento_comando.csv. Para terminar, el modo se obtiene de la misma manera, desde el parámetro de `execute_command_pipeline`. Por último, la función `log_tiempo_total` se encarga de convertir el valor del resultado a 'encontrado' o 'no encontrado' y guarda todos los datos en el archivo.

recursos_python.csv

En el archivo que recoge los recursos utilizados por Python se guardan los valores de `'timestamp_unix'`, `'cpu_percent'` y `'memoria_mb'`. Para obtener estos datos dentro del proceso de Python se ha utilizado un hilo independiente. Esto es necesario porque el hilo principal encargado de la escucha de la *wake word* también ejecuta de forma síncrona y bloqueante todo el *pipeline* de reconocimiento de comandos. Si la obtención de recursos se hiciera en ese mismo hilo, se quedaría bloqueado y las mediciones no serían fiables. Por tanto, utilizar otro hilo garantiza que las muestras de CPU y memoria se recojan sin interrupciones, de forma independiente y que los valores sean correctos y fiables.

Para esta parte de la implementación se han creado 3 funciones nuevas: `start_monitor_recursos` (para crear el hilo), `monitor_recursos_loop` (para obtener los datos continuamente) y `log_recursos` (para escribir el archivo CSV). La función

`monitor_recursos_loop` se llama desde `start_monitor_recursos`, para que empiece a ejecutarse en el momento en el que se crea el hilo. `monitor_recursos_loop` utiliza la librería Psutil, que permite obtener información de procesos activos y utilización del sistema (CPU, memoria, discos, etc.) [70]. Mientras que el proceso está activo, cada segundo se obtiene el porcentaje de CPU que está usando el proceso, además de su uso de memoria en MB. Una vez se obtienen esos datos de un instante, se guardan en el archivo llamando a `log_recursos`.

latencia_ipc.csv

Este archivo guarda el tiempo que necesitan los mensajes para trasladarse desde Python hasta Unity por el *socket* TCP, y tiene las columnas `'tarea'` (identificador de texto del tipo de mensaje enviado), `'timestamp_unix'`, `'dirección'` (Unity a Python o viceversa) y `'latencia_ms'` (el tiempo que ha necesitado el mensaje para llegar de un extremo a otro). El cálculo de estos tiempos se hace mediante los dos procesos, no solo desde Python o desde Unity. Sin embargo, el archivo se escribe en Unity, ya que tener dos procesos escribiendo en el mismo archivo podría traer problemas.

Para implementar esta parte se añadió un atributo a todos los mensajes que se envían tanto desde Unity como desde Python: `timestamp_envio`. Este atributo tiene el momento en el que el mensaje se envía, y gracias a eso, el proceso que recibe el mensaje puede calcular cuánto ha tardado en llegar al otro extremo. La medición del tiempo transcurrido de los mensajes de Unity a Python se hace dentro de la función `handle_unity_message`, la función que se utiliza para gestionar los mensajes entrantes del *socket* (ver el código 3.11). Al inicio de la función se lee el valor del timestamp del JSON entrante, se calcula el tiempo transcurrido, y se llama a la función `log_latencia_ipc` para que envíe un mensaje a Unity con *type*='log_latencia_ipc', dirección='unity_a_python', el tipo de mensaje y la latencia.

```
def handle_unity_message(self, message):
    try:
        data = json.loads(message.lstrip("\ufeff"))
        message_type = data.get("type")
        # Log latencia
        timestamp_envio = data.get("timestamp_envio") #Leer el valor
            de timestamp enviado desde Unity
        if timestamp_envio is not None:
            latencia_ms = (time.time() - timestamp_envio) * 1000 #
                Calcular latencia
            self.log_latencia_ipc("unity_a_python", message_type,
                latencia_ms)

    ...
```

Extracto de código 3.11: Gestión del mensaje de Unity para obtener los micrófonos disponibles

Por otro lado, una vez el mensaje llega a Unity, este lo gestiona en el archivo VoiceReceiver.cs, en la función `Update`, que se encarga de obtener los mensajes procedentes de Python. Por un lado, la función genera una línea en el log de la latencia

para todos los mensajes recibidos, sea cual sea su tipo. Para ello, se calcula la latencia y se llama a la función `LogLatenciaIPC` de la clase ExperimentLogger. Por ejemplo, si el mensaje enviado contiene el *type* 'log_latencia_ipc', se crea una línea en el archivo haciendo referencia a la transmisión de dicho mensaje desde Python a Unity. Por otro lado, solo si el *type* es 'log_latencia_ipc', se hace un log del comando que se había enviado anteriormente de Unity a Python y se había gestionado en Python como se ha visto en 3.11. Esto se puede ver en el código 3.12.

```
private void Update()
{
    while (messageQueue.TryDequeue(out string message))
    {
        MensajeConTimestamp meta = JsonUtility.FromJson<MensajeConTimestamp>(
            message);
        if (meta != null && meta.timestamp_envio > 0)
        {
            double ahora = (DateTime.UtcNow - new DateTime(1970, 1, 1)).
                TotalSeconds; //Obtener timestamp actual (restar 1/1/1970 porque
                el calculo de tiempo en Python se hace desde esa fecha)

            double latenciaMs = (ahora - meta.timestamp_envio) * 1000.0; //Obtener
                 latencia
            if (ExperimentLogger.Instance != null)
                ExperimentLogger.Instance.LogLatenciaIPC("python_a_unity", meta.
                    type, latenciaMs); //Hacer el log del mensaje recibido
        }

        ...

        //LOG latencia IPC
        LogLatenciaMessage logLatenciaMessage = JsonUtility.FromJson<
            LogLatenciaMessage>(message);
        if (logLatenciaMessage.type == "log_latencia_ipc")
        {
            if (double.TryParse(logLatenciaMessage.latencia_ms, System.
                Globalization.NumberStyles.Float,
                System.Globalization.CultureInfo.InvariantCulture, out double
                    latenciaMs))
            {
                ExperimentLogger.Instance.LogLatenciaIPC(logLatenciaMessage.
                    direccion, logLatenciaMessage.tipo_mensaje, latenciaMs); //
                    Hacer log del mensaje mandado anteriormente de Unity a Python
            }
            continue;
        }

        ...
```

Extracto de código 3.12: Gestión de mensajes obtenidos en Unity relacionados con el log de la latencia.

Por último, se debe comentar que la función `LogLatenciaIPC` de ExperimentLogger se encarga de guardar los datos en el archivo CSV.

recursos_unity.csv

La implementación de la obtención de los recursos de Unity es diferente a la utilizada en el proceso de Python. Para ello, primero se ha creado un *script* llamado RecursosUnity.cs, que se encargará de acceder a los DLL de Windows para obtener los datos de la CPU y de la memoria. Para poder acceder a dichos recursos, se ha utilizado el DLL kernel32.dll, y especialmente las funciones `GetCurrentProcess` y

`GetProcessTimes`. La función `GetProcessTimes` devuelve, dado un identificador del proceso, el momento en el que se creó el proceso (`lpCreationTime`), el momento en el que terminó (`lpExitTime`), el tiempo de CPU que el programa ha pasado haciendo acciones del sistema operativo (`lpKernelTime`) y el tiempo de CPU que el programa ha estado ejecutando su propio código (`lpUserTime`). Para calcular ese identificador se utiliza `GetCurrentProcess`. Se puede ver el *import* de dichas funciones en el código 3.13.

```
public class RecursosUnity : MonoBehaviour
{
    [DllImport("kernel32.dll")]
    private static extern IntPtr GetCurrentProcess();

    [DllImport("kernel32.dll", SetLastError = true)]
    private static extern bool GetProcessTimes(
        IntPtr hProcess,
        out long lpCreationTime, //momento en el que se creo el proceso
        out long lpExitTime, //momento en el que termino el proceso
        out long lpKernelTime, //tiempo usado en el kernel (Tiempo que el programa ha
            pasado haciendo cosas que requieren servicios del sistema operativo)
        out long lpUserTime); //tiempo usado por user (tiempo de CPU que el programa
            ha estado ejecutando su propio codigo)

    ...
```

Extracto de código 3.13: Importación de las funciones de Kernel32 en RecursosUnity.cs.

El proceso para calcular el uso de CPU y memoria se basa en comparar los valores actuales con los obtenidos aproximadamente un segundo antes. Para ello se implementó la función `Update`, que hace los cálculos de recursos aproximadamente una vez por segundo.

El primer paso consiste en calcular el tiempo total de CPU utilizado por Unity con la función `GetProcessTimes`. Para hacerlo, se implementó la función `ObtenerTiempoCpu`, que obtiene los tiempos `lpKernelTime` y `lpUserTime` y los suma. El resultado representa el tiempo total de CPU consumido por el proceso, y su unidad es 'Tick', lo que es igual a unidades de 100 nanosegundos. Por tanto, más tarde al calcular la diferencia entre la medición actual y la anterior, se divide entre 10000 para convertirlo a milisegundos.

El siguiente paso fue implementar la función `Update`, en la que se obtiene el tiempo de CPU actual y se calcula la diferencia respecto a la medición anterior. Además, en esta función también se calcula el tiempo real transcurrido entre las dos mediciones. Con estos valores se calcula el porcentaje de uso de CPU, dividiendo el tiempo de CPU utilizado entre el tiempo real transcurrido multiplicado por el número de procesadores disponibles. Para terminar, el resultado se multiplica por 100 para convertirlo a porcentaje.

Por otro lado, se calcula la memoria utilizada por Unity en megabytes con la función `GetTotalAllocatedMemoryLong` de su clase Profiler. Esta devuelve el resultado en bytes, por tanto, se convierte a megabytes.

Para terminar, se llama a `LogRecursos` de ExperimentLogger para crear el log en el archivo recursos_unity.csv, y se actualizan las variables `cpuAnteriorTicks`, `tiempoAnterior` y `timer` para la siguiente iteración. Para ver la implementación completa de la función `Update` ver el código 3.14.

```
private void Update()
{
    timer += Time.unscaledDeltaTime;
    if (timer >= 1f) //Por cada segundo aproximadamente
    {
        long cpuActualTicks = ObtenerTiempoCpu();
        DateTime tiempoActual = DateTime.UtcNow;

        double cpuUsadoMs = (cpuActualTicks - cpuAnteriorTicks) / 10000.0; //Convertir
             Ticks (100 ns) a ms
        double tiempoTranscurridoMs = (tiempoActual - tiempoAnterior).
            TotalMilliseconds;
        double cpuPercent = (cpuUsadoMs / (tiempoTranscurridoMs * Environment.
            ProcessorCount)) * 100.0; //Obtener porcentaje de CPU

        double memoriaMb = Profiler.GetTotalAllocatedMemoryLong() / (1024.0 * 1024.0);
             //Obtener memoria en MB

        if (ExperimentLogger.Instance != null)
            ExperimentLogger.Instance.LogRecursos(cpuPercent, memoriaMb); //Log de
                recursos de Unity

        cpuAnteriorTicks = cpuActualTicks;
        tiempoAnterior = tiempoActual;
        timer = 0f;
    }
}
```

Extracto de código 3.14: Función Update de RecursosUnity para calcular el uso de CPU y memoria.

Falta mencionar que las columnas del archivo recursos_unity.csv son `'timestamp_unix'`, `'cpu_percent'` y `'memoria_mb'`, como en el proceso de Python.

fps.csv

La última medición que se hizo fue la de los fotogramas por segundo de Unity. Para calcularlo se creó un nuevo *script* llamado FPSCounter.cs, cuya función es calcular los FPS y guardar el dato.

Cada vez que se llama a la función `Update`, se suma una unidad a la variable *frames* y el tiempo en segundos desde el último *frame* (*deltaTime*) a la variable `'timer'`. Cuando el *timer* llega a 1 segundo aproximadamente, se calculan los FPS dividiendo los *frames* por el tiempo transcurrido y se llama a la función `LogFPS` de ExperimentLogger.

Las columnas de fps.csv son `'timestamp_unix'` y `'FPS'`.

4. Resultados

4.1. Introducción

Los resultados numéricos son importantes en el desarrollo de una aplicación, especialmente en una de realidades extendidas. En el proceso de preparación y desarrollo de la aplicación se ha tenido presente la importancia del correcto rendimiento y experiencia de usuario, y las decisiones de diseño se han basado en estos factores. Sin embargo, es necesario cuantificar los valores de rendimiento, tiempos de ejecución, etc., para confirmar que una aplicación tiene un buen rendimiento o no.

En este proyecto se han recogido datos de FPS, los tiempos totales de procesamiento de comandos, la latencia del *socket* TCP, etc. Algunos de los archivos de datos se han creado a partir de momentos específicos de la ejecución, mientras que otros archivos miden datos generales como *frames* por segundo, o recursos de CPU y memoria utilizados por Python y Unity. El siguiente esquema clasifica y coloca dichos datos en su momento de ejecución correspondiente.

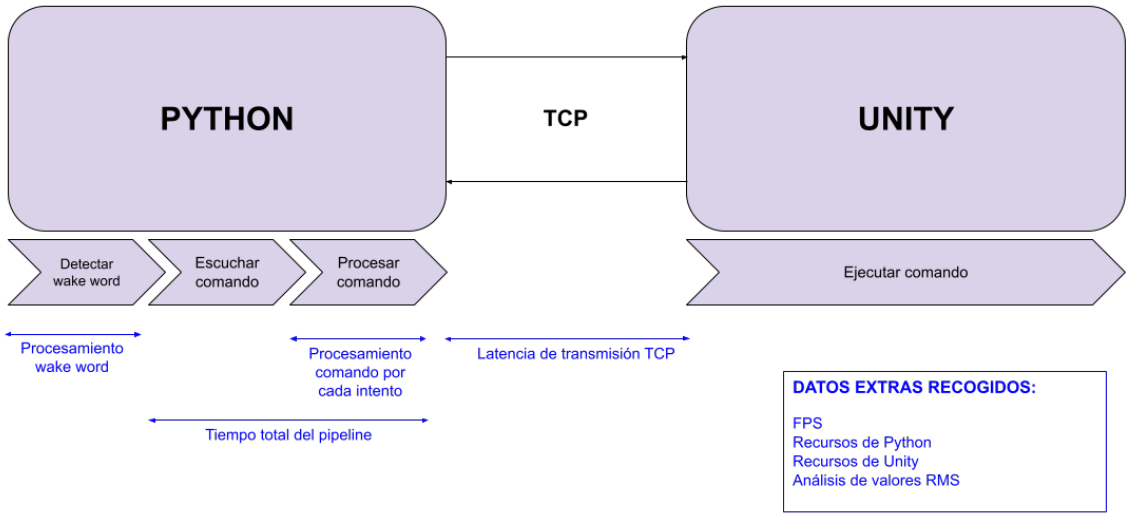


Figura 4.1: Esquema de detección de resultados posicionados en la ejecución.

El experimento en concreto ha consistido en 4 ejecuciones repetidas de una lista de 13 comandos de voz, utilizando la *wake word* o el *hand pose* como modo de activación. Para evitar que el orden de ejecución influyera en los resultados, cada repetición se ha dividido en dos bloques, uno por cada modo de activación, alternando en cada repetición cuál de los dos modos se utiliza primero y variando el orden de los comandos dentro de cada bloque. En total se han hecho 4 sesiones distintas, obteniendo 118 intentos de procesamiento de comandos en total. Los archivos se han ido generando, añadiendo a un libro de Excel, y eliminando de la carpeta de resultados entre sesiones. Como resultado, se han obtenido 4 CSV por cada tipo de dato a guardar, y después se

han unificado los 4 archivos por cada tipo de dato, diferenciando cada sesión con una nueva columna llamada `Sesión` para poder identificar los datos de cada sesión. Las frases pronunciadas en cada sesión se encuentran en el anexo A.0.3.

En este apartado se presentarán y interpretarán los valores recogidos desde la implementación descrita en la sección 3.6.5. Cabe mencionar que todos estos valores se han obtenido desde ejecuciones concretas llevadas a cabo por la autora de este proyecto, y que no se han recogido datos de usuarios.

4.2. Tiempo total del *pipeline* de reconocimiento de comandos

Como ya se ha mencionado en 3.6.5, este archivo guarda los tiempos que el proceso de Python ha necesitado desde que se detecta la *wake word* o *hand pose* hasta que se envía el comando a Unity (ver figura 4.1).

El dato más significativo que presenta este archivo es que la media del tiempo total para los casos de un intento es de 5565,84 ms. No se han tenido en cuenta los casos de dos intentos por su escasez de aparición y porque su valor ronda el doble, lo cual supondría un desvío en el cálculo de la media.

Con los datos que se han obtenido, se ha analizado la diferencia del tiempo del *pipeline* según el tipo de activación (*wake word* o *hand pose*). Como el tiempo se empieza a contar después de dicha activación (y el código siguiente se comparte para los dos modos), lo esperable es que no haya diferencia relevante. La tabla 4.1 y en el gráfico 4.2 muestran que la diferencia entre ambos es mínima, lo cual confirma la hipótesis.

Modo	**Media (ms)**	**n**	**Máx. (ms)**
Manual	5637,92	55	7879,799
wake word	5491,04	53	7865,183

Cuadro 4.1: Estadísticas del tiempo total del *pipeline* según el modo de activación.

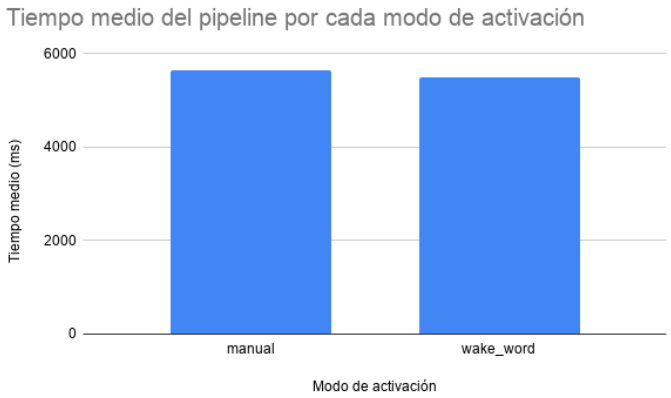


Figura 4.2: Representación visual de los tiempos medios del *pipeline* por modo de activación.

El haber obtenido datos de cuatro sesiones repetidas permite evaluar si el comportamiento es estable entre las sesiones, y si el resultado es puntual o generalizable. Para ello, se han recogido los tiempos medios para cada sesión en la tabla 4.2.

Sesión	**Media (ms)**
1	7331,13
2	4937,13
3	4911,98
4	4902,91

Cuadro 4.2: Tiempo medio del *pipeline* por sesión.

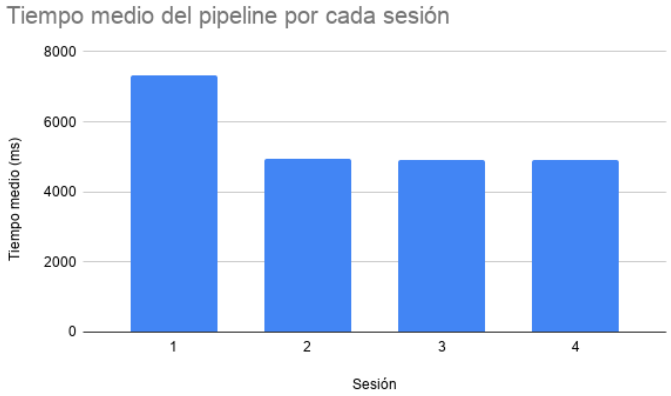


Figura 4.3: Representación visual de los tiempos medios del *pipeline* por sesión.

Se puede ver que para la sesión 1 el tiempo medio es casi un 50 % mayor que el de las demás sesiones. Sin embargo, se ve una consistencia en los tiempos de las sesiones 2, 3 y 4. Aunque no se puede confirmar, una de las posibles razones del tiempo añadido de la primera sesión es que el sistema necesita cargar recursos que luego utiliza, y

al haberse ejecutado la primera sesión justo después de encender el ordenador, puede que haya necesitado cargar todos los recursos. Entre estos recursos están el caché del sistema operativo, la carga de los modelos en la memoria, etc.

Aun así, cuatro sesiones no pueden aislar la causa, ya que sería necesario, por ejemplo, repetir sesiones después de encender el ordenador, para ver si el patrón se repite. Además, harían falta numerosas pruebas para llegar a una conclusión clara.

4.3. Precisión de reconocimiento de comandos

Como ya se ha detallado en la sección 3.6.5, el archivo procesamiento_comando guarda los datos de procesamiento de cada comando procesado.

Es importante mencionar que para facilitar la interpretación de los datos se han añadido las columnas 'frase_real' (la frase que se ha dicho realmente en la prueba), 'acierto' (valor booleano que indica si el comando ejecutado es el que se ha pedido realmente. Los valores posibles son verdadero, falso y vacío para los casos donde 'resultado' es no_encontrado), 'frase_real_normalizada' (la frase real normalizada al estilo de Faster-Whisper - sin tildes ni eñes), y 'transcripcion_exacta' (un booleano que indica si la frase real normalizada y la frase transcrita coinciden). Estas columnas se añadieron para analizar y separar los errores por transcripción y los errores de clasificación.

Sin embargo, el primer dato que se ha obtenido ha sido la precisión de acierto general, que ha sido de 91,89 % para los casos donde el comando se ha encontrado. De los 118 intentos de interpretación, 102 han sido aciertos (86,44 %), 9 han sido errores (7,63 %) y 7 no se han interpretado (5,93 %).

Parte del análisis de los datos ha sido investigar si los errores de acierto vienen de la transcripción o de la clasificación. Para ello se han comparado las frases transcritas exactamente y si se ha acertado o no. En la tabla 4.3 se puede ver que cuando Faster-Whisper transcribe la frase exactamente, el clasificador acierta el 100 % de las veces. Además, cuando la transcripción tiene algún error, el clasificador sigue acertando en el 80,43 % de los casos. Esto demuestra que el sistema de clasificación es tolerante a pequeños errores de transcripción y que la mayoría de los fallos vienen del sistema de Faster-Whisper.

Transcripción exacta	**No acertado**	**Acertado**
FALSE	9	37
TRUE	0	65

Cuadro 4.3: Tabla comparativa entre la transcripción exacta y el índice de acierto.

4.4. Tiempo de escucha y procesamiento del comando

Uno de los datos que recoge el archivo procesamiento_comando.csv que se menciona en 3.6.5 es el tiempo que el sistema ha necesitado para transcribir y clasificar el audio. El tiempo promedio para procesar en los casos donde el comando se ha encontrado es de 401,58 ms, y si lo comparamos con los 5565,84 ms que necesita el *pipeline* entero para la escucha, transcripción y clasificación del comando, se puede deducir que, para este análisis, de media el sistema necesita 5164,26 ms para activar el micrófono y escuchar el comando. En el gráfico 4.4 se pueden ver los porcentajes del tiempo total que supone cada fase.

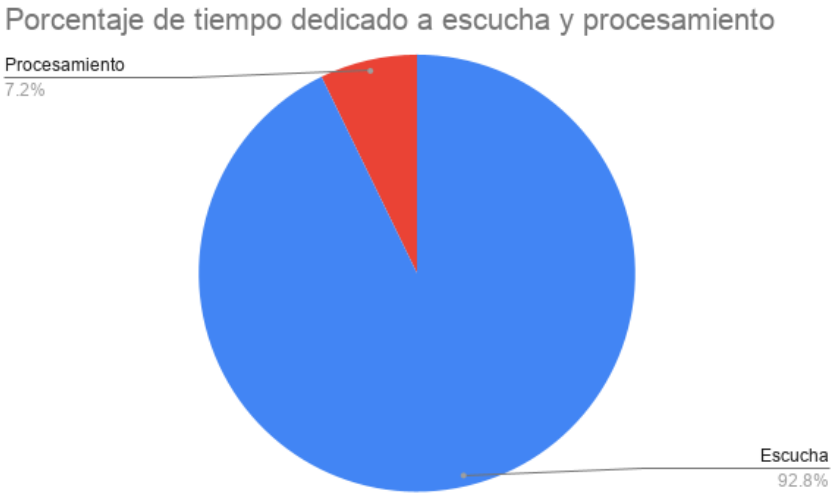


Figura 4.4: Porcentajes del tiempo de cada fase.

4.5. Latencia de comunicación entre Python y Unity

Los datos de la latencia expresan el tiempo que los mensajes necesitan para llegar de un extremo a otro. Su definición de columnas se ha mencionado en 3.6.5.

La latencia media general es de 5,71 ms. Sin embargo, este tiempo varía significativamente dependiendo del tipo de mensaje que se esté enviando. Como se puede ver en el gráfico 4.5, los mensajes con más latencia media son aquellos que mandan información en el mensaje. Por ejemplo, el tipo de mensaje 'microphones' envía el listado de los micrófonos disponibles, para que se rellene el desplegable de la configuración. Otro ejemplo es 'configuration', que se encarga de enviar el JSON de configuración desde Python a Unity.

Un caso curioso que se puede ver en el gráfico es 'log_latencia_ipc', que necesita más tiempo de transporte que lo esperado. Resulta llamativo porque este tipo de mensaje envía el tipo ('log_latencia_ipc'), la dirección ('unity_a_python' o 'python_a_unity'), el tipo de mensaje (el tipo del mensaje que está procesando) y la latencia solamente,

y no envía datos extensos ni complejos. Este caso no afecta al resto de análisis, pero resulta un caso curioso digno de investigar.

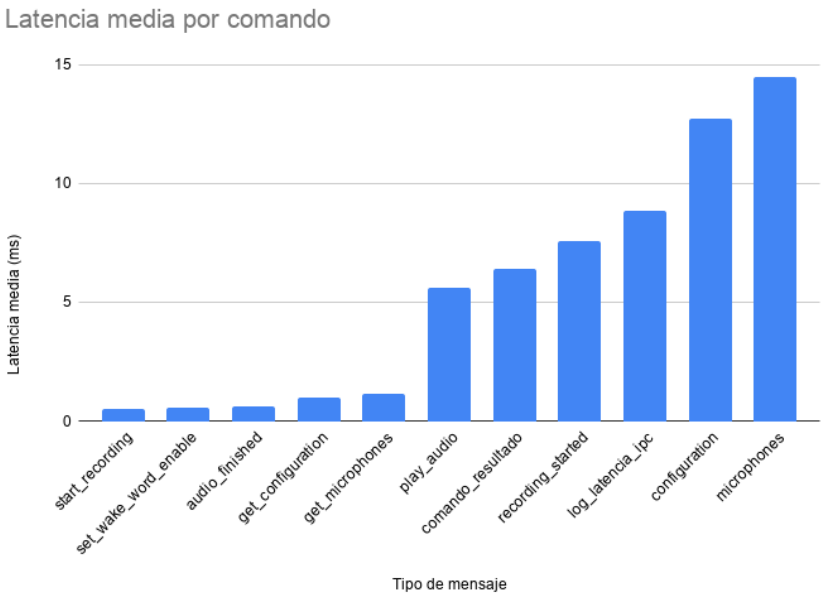


Figura 4.5: Latencia media por comandos

Otro aspecto a analizar ha sido la consistencia de la latencia entre sesiones. Como se puede ver en la tabla 4.4 y en el gráfico 4.6, las sesiones 1 y 2 tienen tiempos muy parecidos entre ellos, como las sesiones 3 y 4. Sin embargo, la diferencia entre las sesiones 2 y 3 es de 0,9 ms, un valor muy pequeño para tener relevancia práctica, y por lo que no puede atribuirse a una causa concreta.

Sesión	**Latencia media (ms)**
1	5,38
2	5,31
3	6,21
4	6,09

Cuadro 4.4: Latencia media por sesión.

4.6. Consumo de recursos

Algunas decisiones de diseño en la aplicación se han tomado para mantener un porcentaje de uso de CPU y de memoria lo más bajo posible. Por tanto, uno de los apartados más importantes de los resultados era analizar los datos obtenidos durante las pruebas y recogerlos en los archivos descritos en las secciones 3.6.5 y 3.6.5. Un detalle a mencionar es que al haber utilizado sistemas diferentes para la obtención de

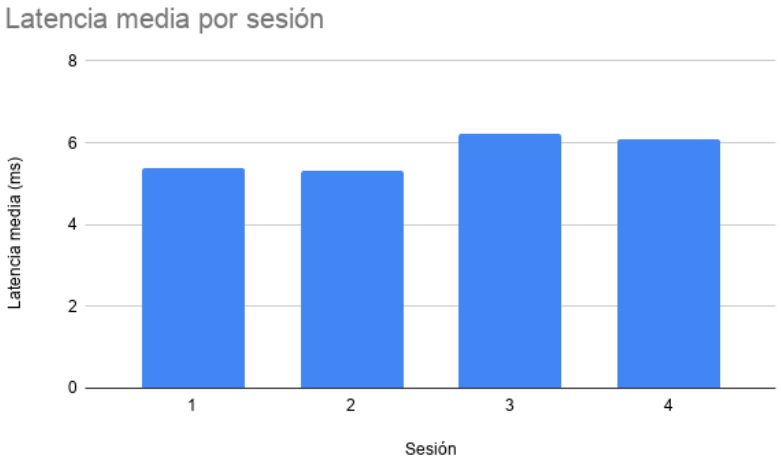


Figura 4.6: Latencia media por sesión

dichos datos (psutil en Python y kernel32.dll en Unity), los valores conseguidos no son comparables entre los dos procesos. Por tanto, en este apartado se compararán los procesos de manera independiente.

Para empezar, Python presenta un uso medio de CPU de 0,805 %, con un máximo de 16,27 % en uno de los casos. Además, la memoria media es de 370,479 MB, con un máximo de 483,66 MB y un mínimo de 342,1 MB.

La evolución temporal de dichos valores permite analizar si el consumo ha sido estable durante cada sesión. Para ello se han desarrollado los gráficos del consumo de CPU 4.7 y los del uso de memoria 4.8.

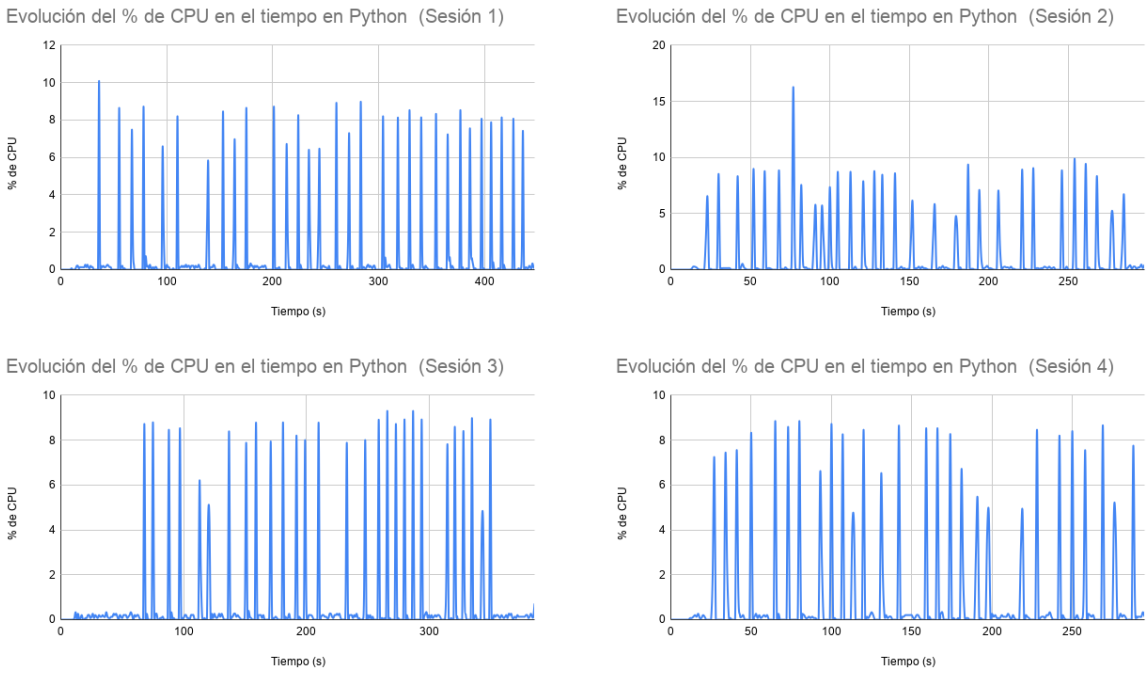


Figura 4.7: Consumo temporal de CPU en Python para cada sesión.

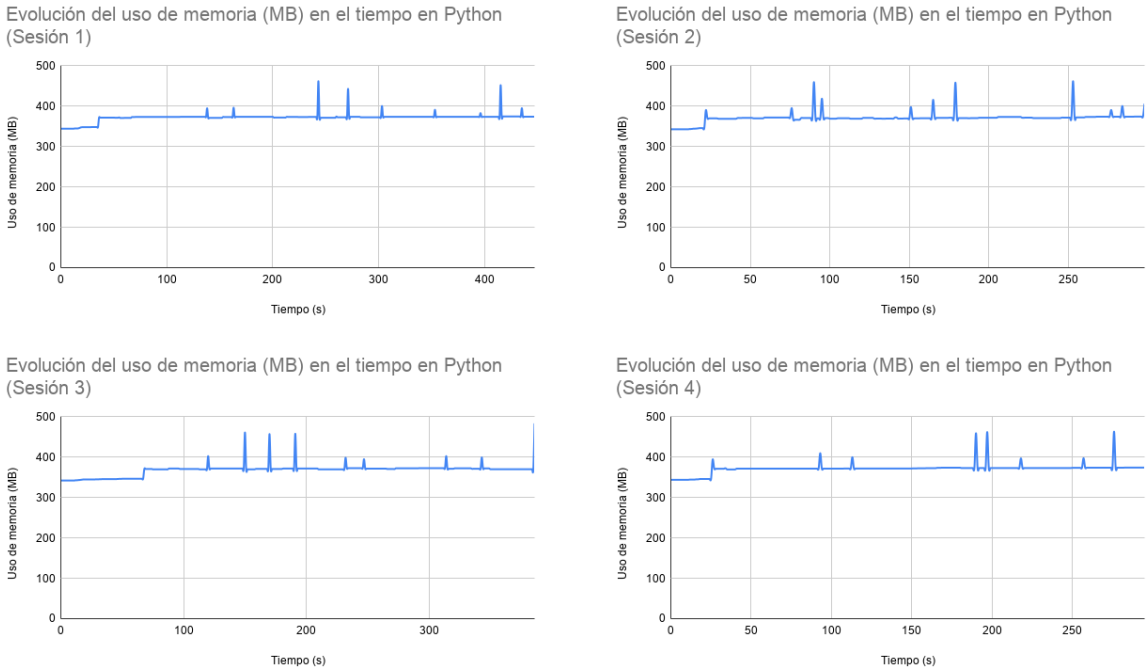


Figura 4.8: Consumo temporal de memoria en Python para cada sesión.

La interpretación de estos gráficos demuestra, por un lado, que el uso base de la CPU está en un 0 %, con picos puntuales que coinciden con las activaciones del sistema de comando. Estos picos se mantienen entre el 6 % y el 10 % en las cuatro sesiones, aunque hay un caso en la sesión 2 que sube a un 16,4 %. Esto se puede dar por una carga puntual del sistema o un caso con reintentos. En cuanto a la memoria, el patrón es muy parecido en las cuatro sesiones: cuando la aplicación se pone en marcha hay una subida rápida que pasa de unos 340 MB a 365 MB. Esto se debe a la carga de modelos, la reserva de *buffers*, etc., que se dan al inicio del proceso. Algo positivo de la memoria es que en los cuatro casos se mantiene un consumo generalmente constante después del arranque, lo cual indica que las reservas temporales de memoria se liberan correctamente. Los picos puntuales pueden atribuirse a reservas temporales de memoria, pero lo importante es que no son acumulativos.

Por otro lado, Unity presenta un uso de CPU medio de 1,50 %, con un máximo de 2,95 %. En cuanto a la memoria, la media está en 173,25 MB, con un máximo de 174,67 MB y un mínimo de 167,98 MB.

La evolución temporal de CPU y memoria se puede ver en los gráficos 4.9 y 4.10

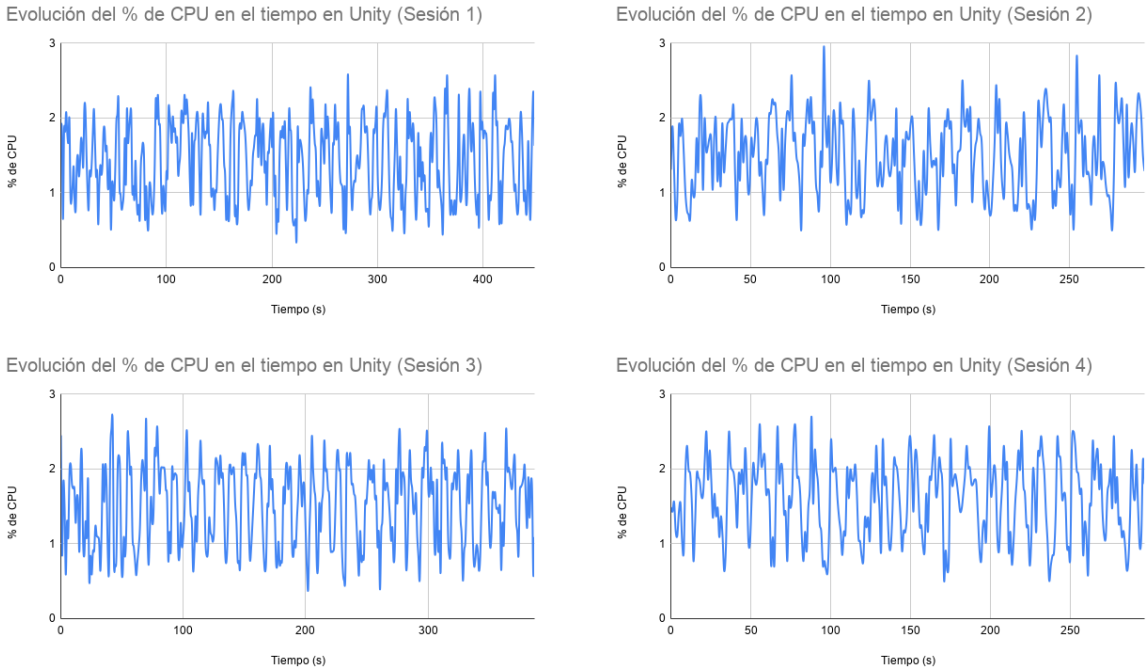


Figura 4.9: Consumo temporal de CPU en Unity para cada sesión.

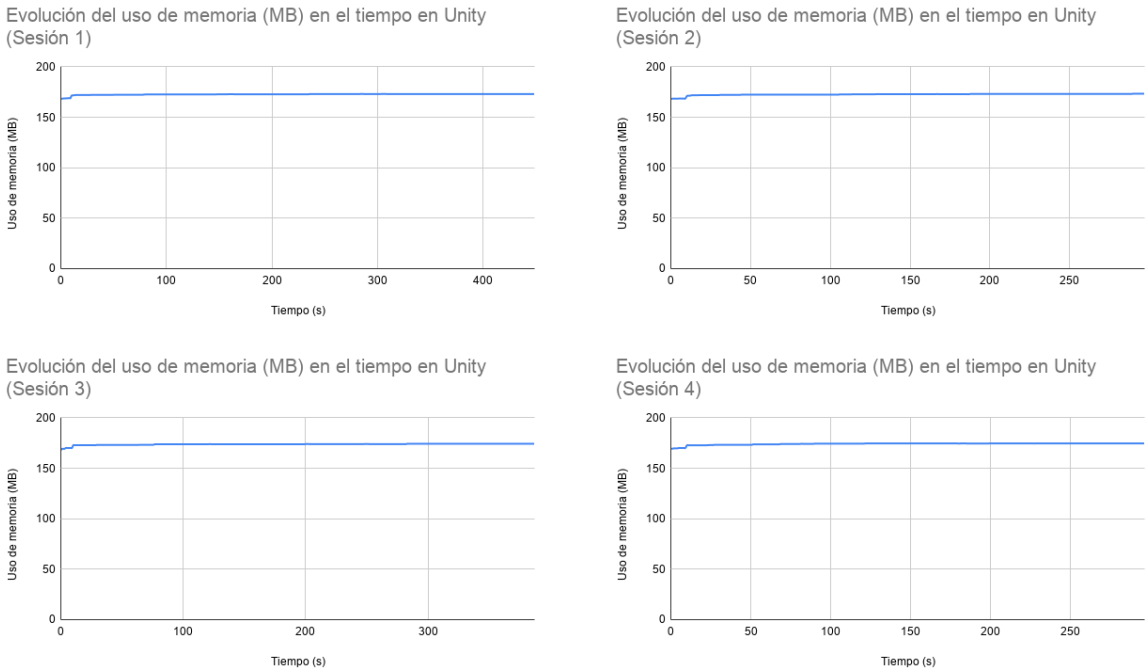


Figura 4.10: Consumo temporal de memoria en Unity para cada sesión.

En los gráficos de CPU de Unity no se ve una línea base con picos puntuales como pasaba en Python. En su lugar hay una oscilación continua y constante a lo largo de la sesión, moviéndose entre 0,5 % y 3 %. Esta diferencia tiene sentido, ya que la carga mayor de trabajo de Python ocurre cuando hay que procesar un comando, mientras

que Unity tiene que renderizar *frames*, calcular físicas, etc., continuamente. En Unity no se encuentra una correlación clara entre la activación del sistema de comandos y los picos visuales, por tanto, se puede confirmar que el uso de CPU está más ligado a procesos del motor (renderizado, etc.). Además, el rango y la forma de la oscilación son parecidos en las cuatro sesiones, lo cual representa la robustez del sistema.

En cuanto a la memoria, las cuatro sesiones muestran el mismo patrón, donde la memoria sube en el arranque (de 165 MB a 175 MB) y se mantiene estable durante el resto de la ejecución sin picos ni una subida constante.

4.7. Rendimiento - FPS

Por último, falta analizar si se mantiene un *framerate* estable y suficiente para una experiencia de VR cómoda, y si el sistema de voz afecta al rendimiento. En este apartado se analizarán los datos recogidos en el archivo fps.csv (ver sección 3.6.5).

Como dato general, la media de FPS total es de 71,96, con un mínimo de 61,3 en el momento de arranque y un máximo de 72,1.

En los gráficos 4.11 se observa que el patrón se mantiene para las cuatro sesiones, donde el *framerate* arranca en 61-65 FPS, y asciende y se estabiliza en los 72 FPS. Con esta visualización se demuestra que el *framerate* mínimo no es por una caída puntual, sino por el proceso de arranque, y que los FPS se mantienen medianamente estables en 72 durante la ejecución. Sin embargo, se pueden observar unas pequeñas caídas que se pueden descartar, ya que se recuperan instantáneamente y bajan a 69 FPS en el peor de los casos.

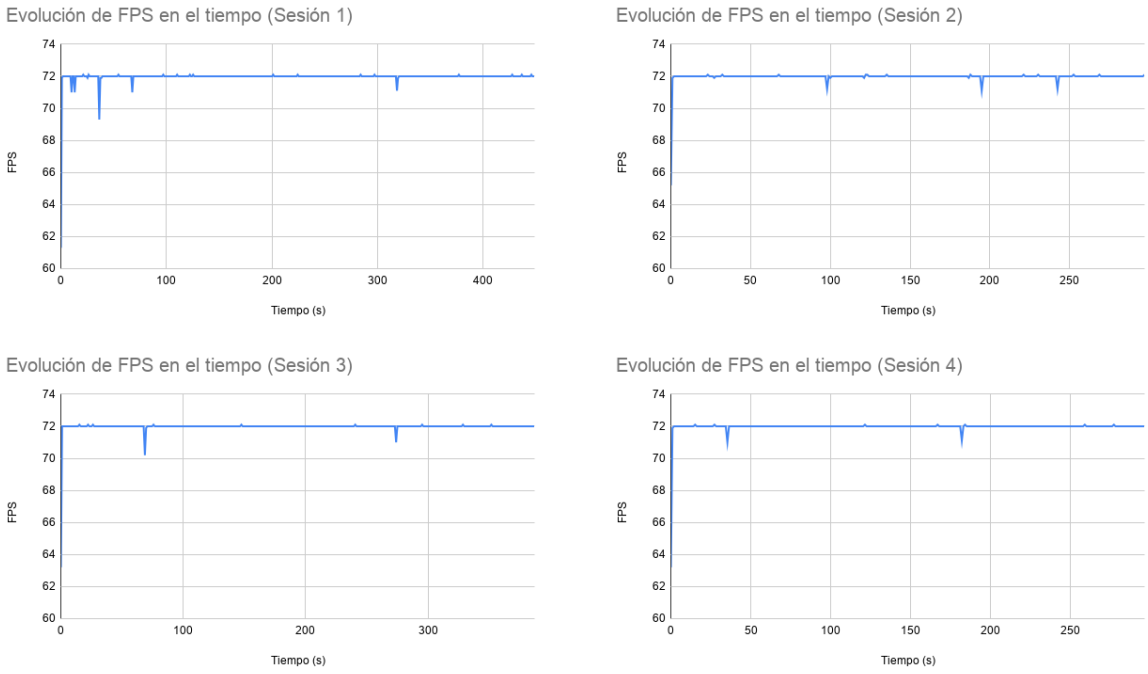


Figura 4.11: Evolución de FPS en el tiempo.

En cuanto a la consistencia entre sesiones, la tabla 4.5 muestra que la media del *framerate* en cada sesión es prácticamente idéntica para todas las sesiones.

Sesión	**Media de FPS**
1	71,96
2	71,97
3	71,97
4	71,97

Cuadro 4.5: Media de FPS por sesión.

4.8. Limitaciones de las pruebas

La limitación más importante de estas pruebas es que todas han sido ejecutadas por la autora del proyecto, por tanto, los resultados reflejan el comportamiento del sistema bajo las condiciones concretas de la autora (acento, entonación, *hardware*, etc.), por lo que no representan los datos exactos del sistema. Además, el tamaño de la muestra (4 sesiones, 118 activaciones en total) es limitado, otra razón por la que los resultados no se pueden generalizar.

En cuanto a la latencia, es importante subrayar que los datos no incluyen la latencia de una red, ya que los procesos se han ejecutado en la misma máquina, y, por tanto, los datos no son extrapolables a arquitecturas con máquinas distintas. Además, al haber ejecutado todas las pruebas en el mismo ordenador y visor, los resultados de consumo de recursos y de FPS no se pueden generalizar a otros equipos.

Sin embargo, estos datos permiten tener una visión general del comportamiento de la aplicación en un sistema de *hardware* específico con los procesos ejecutándose localmente.

5. Conclusiones

5.1. Conclusiones

La meta de este Trabajo de Fin de Máster ha sido implementar y evaluar una aplicación de realidad mixta para los visores Meta Quest que permite a un operario de ROV visualizar de forma inmersiva la información del vehículo e interactuar con dicha información mediante comandos de voz. En este proyecto se ha incorporado un sistema de interacción por voz, y se ha diseñado e implementado un proceso de evaluación que permite cuantificar su comportamiento, como se planteaba en los objetivos del capítulo 1.

En cuanto a la visualización, se ha conseguido crear una experiencia de XR que permite mostrar el contenido de Cockpit, manteniendo un rendimiento y consumo de recursos estables como se ha demostrado en el capítulo 4. En cuanto a la interacción de voz, se ha conseguido implementar un proyecto de Python que gestiona todo el sistema de voz, el cual que se puede activar tanto mediante un gesto manual como una *wake word* y es capaz de escuchar, transcribir y clasificar los comandos.

Para evaluar el sistema se han ejecutado cuatro sesiones de prueba con 118 intentos de procesamiento de comandos en total. Esta fase ha permitido cuantificar el comportamiento de cada proceso así como su comunicación. Entre los resultados más relevantes destaca que la media del tiempo total para el reconocimiento de comandos ha sido de 5565,84 ms, que incluye los 3 segundos de escucha del comando. en el apartado de Trabajo Futuro se presenta una solución para reducir este tiempo (ver 5.2). Por otro lado, la latencia de comunicación entre procesos ha sido de 5,71 ms de media, que es considerablemente menor que el resto del *pipeline*. Por último, los resultados apuntan a que la transcripción es la principal fuente de error, ya que se ha podido observar que cuando la transcripción del comando es exacta, la precisión de acierto es del 100 %, y 80,4 % cuando no lo es.

Sin embargo, el proyecto presenta varias limitaciones. La limitación principal es que la aplicación no tiene conexión directa con Cockpit, por lo que no permite ocultar o enseñar valores concretos del software a través de los comandos de voz. Como solución se ha implementado la duplicación de pantalla que permite visualizar Cockpit si este está visible en la pantalla del ordenador conectado al visor. Asimismo, el *passthrough* no se puede activar en la versión compilada por falta de desarrollo por parte de Meta OpenXR, aunque se prevé que esta funcionalidad llegue en el futuro. En cuanto a la evaluación de la aplicación, la limitación más relevante es que las pruebas se han realizado únicamente por la autora y sin la participación de usuarios externos, por lo que los resultados no se pueden generalizar y no se ha podido hacer un análisis de la experiencia de usuario.

En conclusión, en este trabajo se ha diseñado, implementado y evaluado una aplicación de realidad mixta funcional, se ha cuantificado su rendimiento y comportamiento, y se han identificado y explicado sus aspectos a mejorar en el apartado 5.2.

5.2. Trabajo Futuro

La aplicación presenta varias limitaciones, así como mejoras que no se han podido implementar por falta de tiempo. En este apartado se explicarán las propuestas de trabajos futuros.

Ahora mismo, el sistema siempre graba 3 segundos de audio para obtener el comando del usuario, por tanto, se propone sustituir la duración de captura fija por una solución dinámica donde la aplicación pare de escuchar cuando el usuario deja de hablar. Además, se propone investigar alternativas al modelo de reconocimiento de voz actual para mejorar la transcripción de dichos audios, y así mejorar la tasa de acierto.

En segundo lugar, se plantea adaptar el sistema de detección de audio a diferentes volúmenes, acentos y entonaciones. Para el caso del volumen, se propone incorporar una fase inicial de calibración en la que se mide y guarda el volumen de voz del usuario, de forma que, al decir un comando, se ajuste el volumen del audio captado para que el sistema no se vea especialmente afectado por voces de volumen bajo.

Relacionado con la propuesta inicial del proyecto, se sugiere implementar la visualización en tiempo real de máscaras de anomalías mediante modelos de visión artificial, y también integrar directamente Cockpit en la aplicación para que el operario pueda ocultar y mostrar datos específicos del software.

Además, se recomienda investigar la latencia cuando los procesos se ejecutan en diferentes máquinas y conectadas por la red.

Por último, se propone mejorar la interfaz de usuario mediante una fase de introducción en el que el operario reciba todas las instrucciones de uso. Además, un trabajo futuro importante es solucionar la limitación actual del *passthrough* en la versión compilada, que se podrá hacer cuando Meta OpenXR publique la actualización correspondiente.

6. Bibliografía

[1] Marcos De la Cruz, Gustavo Casañ, Pedro Sanz, and Raul Marín Prades. Preliminary work on a virtual reality interface for the guidance of underwater robots. *Robotics*, 9:81, 10 2020.

[2] Yuchong Zhang, Bastian Orthmann, Michael C. Welle, Jonne Van Haastregt, and Danica Kragic. Llm-driven augmented reality puppeteer: Controller-free voice-commanded robot teleoperation, 2025.

[3] GG-Labs. Linking dlls. https://unreal.gg-labs.com/wiki-archives/devops/linking-dlls, n.d.

[4] Epic Games. Unreal Engine Modules — Unreal Engine 5.8 Documentation — Epic Developer Community. https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-modules, n.d.

[5] Alan Zucconi. How to write native plugins for unity. https://www.alanzucconi.com/2015/10/11/how-to-write-native-plugins-for-unity/, October 2015.

[6] Unity Technologies. Unity - Manual: Native plug-ins. https://docs.unity3d.com/6000.3/Documentation/Manual/plug-ins-native.html, n.d.

[7] Circuit Stream. C# vs c++: Complete comparison between unity and unreal programming language. https://www.circuitstream.com/blog/c-vs-c-complete-comparison-between-unity-and-unreal-programming-language, May 2022.

[8] Santiago Berrezueta-Guzman and Stefan Wagner. Choosing the right engine in the virtual reality landscape. *IEEE Access*, 14:13972–13985, 2026.

[9] Oleksandra Sobchyshak, Santiago Berrezueta-Guzman, and Stefan Wagner. Pushing the boundaries of immersion and storytelling: A technical review of unreal engine. *Displays*, 91:103268, January 2026.

[10] Khronos Group. Openxr. https://www.khronos.org/openxr/#xr-fragmentation, n.d.

[11] Meta. Meta and OpenXR. https://developers.meta.com/horizon/blog/openxr-standard-quest-horizonos-unity-unreal-godot-developer-success/, n.d.

[12] Microsoft. Windows GDI - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/gdi/windows-gdi, n.d.

[13] Microsoft. Device Contexts - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/gdi/device-contexts, n.d.

[14] Microsoft. Bitmaps (Windows GDI) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/gdi/bitmaps, n.d.

[15] Microsoft. BitBlt function (wingdi.h) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/api/wingdi/nf-wingdi-bitblt, n.d.

[16] Microsoft. Desktop Duplication API - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/direct3ddxgi/desktop-dup-api, n.d.

[17] Microsoft. DXGI - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/direct3ddxgi/dx-graphics-dxgi, n.d.

[18] Microsoft. IDXGIOutputDuplication::AcquireNextFrame (dxgi1_2.h) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/api/dxgi1_2/nf-dxgi1_2-idxgioutputduplication-acquirenextframe, n.d.

[19] Microsoft. IDXGIOutput1::DuplicateOutput (dxgi1_2.h) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/api/dxgi1_2/nf-dxgi1_2-idxgioutput1-duplicateoutput, n.d.

[20] Microsoft. IDXGIOutputDuplication (dxgi1_2.h) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/api/dxgi1_2/nn-dxgi1_2-idxgioutputduplication, n.d.

[21] Microsoft. Screen capture - Windows apps. https://learn.microsoft.com/en-us/windows/apps/develop/media-authoring-processing/screen-capture, n.d.

[22] Microsoft. GraphicsCapturePicker Class (Windows.Graphics.Capture) - Windows apps. https://learn.microsoft.com/en-us/uwp/api/windows.graphics.capture.graphicscapturepicker?view=winrt-28000, n.d.

[23] Microsoft. GraphicsCaptureSession Class (Windows.Graphics.Capture) - Windows apps. https://learn.microsoft.com/en-us/uwp/api/windows.graphics.capture.graphicscapturesession?view=winrt-28000, n.d.

[24] Microsoft. Direct3D11CaptureFramePool Class (Windows.Graphics.Capture) - Windows apps. https://learn.microsoft.com/en-us/uwp/api/windows.graphics.capture.direct3d11captureframepool?view=winrt-28000, n.d.

[25] Microsoft. GraphicsCaptureSession.IsBorderRequired Property (Windows.Graphics.Capture) - Windows apps. https://learn.microsoft.com/en-us/uwp/api/windows.graphics.capture.graphicscapturesession.isborderrequired, n.d.

[26] Microsoft. ID3D11DeviceContext::CopyResource (d3d11.h) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/api/d3d11/nf-d3d11-id3d11devicecontext-copyresource, n.d.

[27] Unity Technologies. Unity - Manual: Introduction to native plug-in API. https://docs.unity3d.com/6000.3/Documentation/Manual/native-plugin-interface-introduction.html, n.d.

[28] Unity Technologies. Unity - Manual: Native plug-in API for graphics and rendering. https://docs.unity3d.com/Manual/low-level-native-plugin-rendering-extensions.html, n.d.

[29] Unity Technologies. Unity - Scripting API: Texture.GetNativeTexturePtr. https://docs.unity3d.com/ScriptReference/Texture.GetNativeTexturePtr.html, n.d.

[30] Microsoft. Surface sharing between Windows graphics APIs - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/direct3darticles/surface-sharing-between-windows-graphics-apis, n.d.

[31] Microsoft. IDXGIKeyedMutex (dxgi.h) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/api/dxgi/nn-dxgi-idxgikeyedmutex, n.d.

[32] Microsoft. IDXGIKeyedMutex::AcquireSync (dxgi.h) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/api/dxgi/nf-dxgi-idxgikeyedmutex-acquiresync, n.d.

[33] Microsoft. IDXGIKeyedMutex::ReleaseSync (dxgi.h) - Win32 apps. https://learn.microsoft.com/en-us/windows/win32/api/dxgi/nf-dxgi-idxgikeyedmutex-releasesync, n.d.

[34] Virtual Desktop. Home — Virtual Desktop. https://www.vrdesktop.net/, n.d.

[35] Meta. Configurar y conectar meta horizon link y air link — ayuda de quest (es). https://www.meta.com/es-es/help/quest/509273027107091/, 2026.

[36] Meta. Requisitos de los ordenadores windows para usar meta horizon link — ayuda de quest (es). https://www.meta.com/es-es/help/quest/140991407990979/, 2026.

[37] GitHub. GitHub - alvr-org/ALVR: Stream VR games from your PC to your headset via Wi-Fi. https://github.com/alvr-org/ALVR, n.d.

[38] GitHub. How ALVR works. https://github.com/alvr-org/ALVR/wiki/How-ALVR-works, n.d.

[39] Bigscreen. Remote Desktop · Bigscreen. https://bigscreenvr.com/remotedesktop/, n.d.

[40] Immersed. Immersed. https://immersed.com/, n.d.

[41] Minhua Wu. Alexa scientists present two new techniques that improve wake word performance. https://www.amazon.science/blog/alexa-scientists-present-two-new-techniques-that-improve-wake-word-performance, April 2018.

[42] Ming Sun, Varun Nagaraja, Björn Hoffmeister, and Shiv Vitaladevuni. Model shrinking for embedded keyword spotting. In *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, pages 369–374, 2015.

[43] Biel Tura, Santiago Escuder, Ferran Diego, Carlos Segura, and Jordi Luque. Efficient keyword spotting by capturing long-range interactions with temporal lambda networks, 2021.

[44] Fernando López, Jordi Luque, Carlos Segura, and Pablo Gómez. Robust wake-up word detection by two-stage multi-resolution ensembles, 2023.

[45] Picovoice. Complete guide to wake word detection. https://picovoice.ai/blog/complete-guide-to-wake-word/, October 2025.

[46] David Scripka. openwakeword. https://github.com/dscripka/openWakeWord, 2024.

[47] Yanjiao Chen, Yijie Bai, Richard Mitev, Kaibo Wang, Ahmad-Reza Sadeghi, and Wenyuan Xu. Fakewake: Understanding and mitigating fake wake-up words of voice assistants, 2021.

[48] Aditya Sharadchandra Joshi. Self-trigger prevention. Patent US-12033631-B1.

[49] Marcia Combs, Casey Hazelwood, and Randall Joyce. Are you listening? - an observational wake word privacy study. *Organizational Cybersecurity Journal: Practice, Process & People*, 2(2):113–123, 06 2022.

[50] Pai Zhu, Dhruuv Agarwal, Jacob W Bartel, Kurt Partridge, Hyun Jin Park, and Quan Wang. Synth4kws: Synthesized speech for user defined keyword spotting in low resource environments. In *Synthetic Data's Transformative Role in Foundational Speech Models*, syndata4genai_2024, pages 11–15. ISCA, August 2024.

[51] Meta. Voice sdk overview. https://developers.meta.com/horizon/documentation/unity/voice-sdk-overview/.

[52] Meta. Developer data use policy. https://developers.meta.com/horizon/policy/data-use/.

[53] Meta Platform Technologies. Meta platform technologies sdk license agreement. https://developers.meta.com/horizon/licenses/oculussdk/.

[54] Jie Tang, Yong Ren, and Shaoshan Liu. Real-time robot localization, vision, and speech recognition on nvidia jetson tx1, 2017.

[55] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2022.

[56] N. D. Tantaroudas, A. J. McCracken, I. Karachalios, and E. Papatheou. Ai-based services for inclusive language learning in immersive xr environments: Speech translation, and sign language integration [version 2; peer review: 2 approved]. *Open Research Europe*, 6(72), 2026.

[57] Alpha Cephei. Vosk offline speech recognition api. https://alphacephei.com/vosk/.

[58] SYSTRAN. Faster-whisper. https://github.com/SYSTRAN/faster-whisper.

[59] RapidFuzz. Rapidfuzz. https://rapidfuzz.com/.

[60] RapidFuzz. rapidfuzz.fuzz. https://rapidfuzz.github.io/RapidFuzz/Usage/fuzz.html.

[61] Souvika Sarkar, Mohammad Fakhruddin Babar, Md Mahadi Hassan, Monowar Hasan, and Shubhra Kanti Karmaker Santu. Processing natural language on embedded devices: How well do transformer models perform? In *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*, ICPE '24, pages 211–222, New York, NY, USA, 2024. Association for Computing Machinery.

[62] Kalpa Gunaratna, Vijay Srinivasan, and Hongxia Jin. Explainable and accurate natural language understanding for voice assistants and beyond, 2023.

[63] P. Monteiro, G. Goncalves, H. Coelho, M. Melo, and M. Bessa. Hands-free interaction in immersive virtual reality: A systematic review. *IEEE Transactions on Visualization and Computer Graphics*, 27(5):2702–2713, 2021.

[64] A. Silberschatz, P.B. Galvin, and G. Gagne. *Operating System Concepts*. Wiley, 2018.

[65] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. Addison-Wesley Publishing Company, USA, 5th edition, 2009.

[66] Unity Technologies. Unity - Manual: VR frame timing. https://docs.unity3d.com/6000.3/Documentation/Manual/VRFrameTiming.html, n.d.

[67] Microsoft. Socket.Receive Método (System.Net.Sockets). https://learn.microsoft.com/es-es/dotnet/api/system.net.sockets.socket.receive?view=net-10.0, n.d.

[68] Unity Technologies. Unity - Manual: Asynchronous programming. https://docs.unity3d.com/6000.0/Documentation/Manual/AwaitSupport.html, n.d.

[69] GitHub. GitHub - alfiedennen/openwakeword-colab-2026: Bulletproof openWakeWord trainer Colab notebook (2026 edition). Run-all-and-walk-away. Train your own wake word in 75-90 min on Colab Pro. https://github.com/alfiedennen/openwakeword-colab-2026/tree/main, n.d.

[70] Giampaolo Rodola. psutil 7.2.2 documentation. https://psutil.readthedocs.io/stable/, n.d.

A. Anexos

A.0.1. Listado de comandos de voz disponibles

Extracto de código A.1: Listado de comandos de voz disponibles

```
{
    "IncreaseQuadSize": [
        "aumenta el tamaño",
        "aumentar el tamaño",
        "aumenta la pantalla",
        "aumentar la pantalla",
        "aumenta tamaño",
        "aumentar tamaño",
        "aumenta pantalla",
        "aumentar pantalla",
        "aumenta el tamaño de la pantalla",
        "aumentar el tamaño de la pantalla",


        "incrementa el tamaño",
        "incrementar el tamaño",
        "incrementa tamaño",
        "incrementar tamaño",
        "incrementa la pantalla",
        "incrementar la pantalla",
        "incrementa el tamaño de la pantalla",
        "incrementar el tamaño de la pantalla",


        "amplía la pantalla",
        "ampliar la pantalla",
        "amplía pantalla",
        "ampliar pantalla",
        "amplía el tamaño",
        "ampliar el tamaño",
        "amplía el tamaño de la pantalla",
        "ampliar el tamaño de la pantalla",


        "hacer la pantalla mas grande",
        "haz la pantalla mas grande",
        "hacer la pantalla mayor",
        "haz la pantalla mayor",
        "hacer el tamaño mayor",
        "haz el tamaño mayor",

        "pon la pantalla mas grande",
```

```
        "poner la pantalla mas grande",

        "agranda la pantalla",
        "agrandar la pantalla",
        "granda pantalla",
        "agrandar pantalla",
        "agranda el tamaño de la pantalla",
        "agrandar el tamaño de la pantalla"
    ],
    "DecreaseQuadSize": [
        "reduce el tamaño",
        "reducir el tamaño",
        "reduce la pantalla",
        "reducir la pantalla",
        "reduce tamaño",
        "reducir tamaño",
        "reduce pantalla",
        "reducir pantalla",
        "reduce el tamaño de la pantalla",
        "reducir el tamaño de la pantalla",

        "disminuye el tamaño",
        "disminuir el tamaño",
        "disminuye tamaño",
        "disminuir tamaño",
        "disminuye la pantalla",
        "disminuir la pantalla",
        "disminuye el tamaño de la pantalla",
        "disminuir el tamaño de la pantalla",


        "hacer la pantalla más pequeña",
        "haz la pantalla más pequeña",
        "hacer la pantalla menor",
        "haz la pantalla menor",
        "hacer el tamaño menor",
        "haz el tamaño menor",

        "pon la pantalla más pequeña",
        "poner la pantalla más pequeña",

        "achica la pantalla",
        "achicar la pantalla",
        "achica pantalla",
        "achicar pantalla",
        "achica el tamaño de la pantalla",
        "achicar el tamaño de la pantalla"

    ],
    "CenterScreen": [
```

```
93          "centra la pantalla",
94          "centrar la pantalla",
95          "centra pantalla",
96          "centrar pantalla",
97          "pon la pantalla en el centro",
98          "poner la pantalla en el centro",
99          "pon pantalla en el centro",
100         "poner pantalla en el centro",
101
102         "mueve la pantalla al centro",
103         "mover la pantalla al centro",
104         "mueve pantalla al centro",
105         "mover pantalla al centro",
106         "lleva la pantalla al centro",
107         "llevar la pantalla al centro",
108         "lleva pantalla al centro",
109         "llevar pantalla al centro",
110
111         "ponla en el centro",
112         "ponerla en el centro",
113         "muévela al centro",
114         "moverla al centro",
115
116         "alinear al centro",
117         "alinea al centro",
118         "alinear la pantalla al centro",
119         "alinea la pantalla al centro",
120         "alinear pantalla al centro",
121         "alinea pantalla al centro"
122
123     ],
124     "RaiseScreen": [
125         "sube la pantalla",
126         "subir la pantalla",
127         "sube pantalla",
128         "subir pantalla",
129         "sube la pantalla arriba",
130         "subir la pantalla arriba",
131         "sube pantalla arriba",
132         "subir pantalla arriba",
133         "sube la pantalla hacia arriba",
134         "subir la pantalla hacia arriba",
135
136         "mueve la pantalla arriba",
137         "mover la pantalla arriba",
138         "mueve pantalla arriba",
139         "mover pantalla arriba",
140         "mueve la pantalla hacia arriba",
141         "mover la pantalla hacia arriba",
142         "mueve pantalla hacia arriba",
```

```
143         "mover pantalla hacia arriba",
144 
145         "lleva la pantalla arriba",
146         "llevar la pantalla arriba",
147         "lleva pantalla arriba",
148         "llevar pantalla arriba",
149         "desplaza la pantalla arriba",
150         "desplazar la pantalla arriba",
151         "desplaza pantalla arriba",
152         "desplazar pantalla arriba",
153 
154 
155         "pon la pantalla más arriba",
156         "poner la pantalla más arriba",
157         "pon pantalla más arriba",
158         "poner pantalla más arriba",
159         "coloca la pantalla más arriba",
160         "colocar la pantalla más arriba",
161         "coloca pantalla más arriba",
162         "colocar pantalla más arriba",
163 
164         "desplaza la pantalla hacia arriba",
165         "desplazar la pantalla hacia arriba",
166         "desplaza pantalla hacia arriba",
167         "desplazar pantalla hacia arriba",
168         "mueve la pantalla hacia arriba",
169         "mover la pantalla hacia arriba",
170         "mueve pantalla hacia arriba",
171         "mover pantalla hacia arriba"
172 
173     ],
174     "LowerScreen": [
175         "baja la pantalla",
176         "bajar la pantalla",
177         "baja pantalla",
178         "bajar pantalla",
179         "baja la pantalla abajo",
180         "bajar la pantalla abajo",
181         "baja pantalla abajo",
182         "bajar pantalla abajo",
183         "baja la pantalla hacia abajo",
184         "bajar la pantalla hacia abajo",
185 
186         "mueve la pantalla abajo",
187         "mover la pantalla abajo",
188         "mueve pantalla abajo",
189         "mover pantalla abajo",
190         "mueve la pantalla hacia abajo",
191         "mover la pantalla hacia abajo",
192         "mueve pantalla hacia abajo",
```

```
193         "mover pantalla hacia abajo",
194 
195         "lleva la pantalla abajo",
196         "llevar la pantalla abajo",
197         "lleva pantalla abajo",
198         "llevar pantalla abajo",
199         "desplaza la pantalla abajo",
200         "desplazar la pantalla abajo",
201         "desplaza pantalla abajo",
202         "desplazar pantalla abajo",
203 
204         "pon la pantalla más abajo",
205         "poner la pantalla más abajo",
206         "pon pantalla más abajo",
207         "poner pantalla más abajo",
208         "coloca la pantalla más abajo",
209         "colocar la pantalla más abajo",
210         "coloca pantalla más abajo",
211         "colocar pantalla más abajo",
212 
213         "desplaza la pantalla hacia abajo",
214         "desplazar la pantalla hacia abajo",
215         "desplaza pantalla hacia abajo",
216         "desplazar pantalla hacia abajo",
217         "mueve la pantalla hacia abajo",
218         "mover la pantalla hacia abajo",
219         "mueve pantalla hacia abajo",
220         "mover pantalla hacia abajo"
221 
222     ],
223     "MoveScreenRight": [
224         "mueve la pantalla a la derecha",
225         "mover la pantalla a la derecha",
226         "mueve pantalla a la derecha",
227         "mover pantalla a la derecha",
228         "mueve la pantalla hacia la derecha",
229         "mover la pantalla hacia la derecha",
230         "mueve pantalla hacia la derecha",
231         "mover pantalla hacia la derecha",
232 
233         "lleva la pantalla a la derecha",
234         "llevar la pantalla a la derecha",
235         "lleva pantalla a la derecha",
236         "llevar pantalla a la derecha",
237         "lleva la pantalla hacia la derecha",
238         "llevar la pantalla hacia la derecha",
239         "lleva pantalla hacia la derecha",
240         "llevar pantalla hacia la derecha",
241 
242         "desplaza la pantalla a la derecha",
```

```
243         "desplazar la pantalla a la derecha",
244         "desplaza pantalla a la derecha",
245         "desplazar pantalla a la derecha",
246         "desplaza la pantalla hacia la derecha",
247         "desplazar la pantalla hacia la derecha",
248         "desplaza pantalla hacia la derecha",
249         "desplazar pantalla hacia la derecha",
250 
251         "pon la pantalla a la derecha",
252         "poner la pantalla a la derecha",
253         "pon pantalla a la derecha",
254         "poner pantalla a la derecha",
255         "coloca la pantalla a la derecha",
256         "colocar la pantalla a la derecha",
257         "coloca pantalla a la derecha",
258         "colocar pantalla a la derecha",
259 
260         "haz que la pantalla se mueva a la derecha",
261         "hacer que la pantalla se mueva a la derecha",
262         "haz que la pantalla vaya a la derecha",
263         "hacer que la pantalla vaya a la derecha"
264 
265     ],
266     "MoveScreenLeft": [
267         "mueve la pantalla a la izquierda",
268         "mover la pantalla a la izquierda",
269         "mueve pantalla a la izquierda",
270         "mover pantalla a la izquierda",
271         "mueve la pantalla hacia la izquierda",
272         "mover la pantalla hacia la izquierda",
273         "mueve pantalla hacia la izquierda",
274         "mover pantalla hacia la izquierda",
275 
276         "lleva la pantalla a la izquierda",
277         "llevar la pantalla a la izquierda",
278         "lleva pantalla a la izquierda",
279         "llevar pantalla a la izquierda",
280         "lleva la pantalla hacia la izquierda",
281         "llevar la pantalla hacia la izquierda",
282         "lleva pantalla hacia la izquierda",
283         "llevar pantalla hacia la izquierda",
284 
285         "desplaza la pantalla a la izquierda",
286         "desplazar la pantalla a la izquierda",
287         "desplaza pantalla a la izquierda",
288         "desplazar pantalla a la izquierda",
289         "desplaza la pantalla hacia la izquierda",
290         "desplazar la pantalla hacia la izquierda",
291         "desplaza pantalla hacia la izquierda",
292         "desplazar pantalla hacia la izquierda",
```

```
293
294         "pon la pantalla a la izquierda",
295         "poner la pantalla a la izquierda",
296         "pon pantalla a la izquierda",
297         "poner pantalla a la izquierda",
298         "coloca la pantalla a la izquierda",
299         "colocar la pantalla a la izquierda",
300         "coloca pantalla a la izquierda",
301         "colocar pantalla a la izquierda",
302
303         "haz que la pantalla se mueva a la izquierda",
304         "hacer que la pantalla se mueva a la izquierda",
305         "haz que la pantalla vaya a la izquierda",
306         "hacer que la pantalla vaya a la izquierda"
307     ],
308     "MoveScreenCloser": [
309         "acerca la pantalla",
310         "acercar la pantalla",
311         "acerca pantalla",
312         "acercar pantalla",
313
314         "acerca la imagen",
315         "acercar la imagen",
316         "acerca imagen",
317         "acercar imagen",
318         "acerca la vista",
319         "acercar la vista",
320         "acerca vista",
321         "acercar vista",
322
323         "pon la pantalla más cerca",
324         "poner la pantalla más cerca"
325
326     ],
327     "MoveScreenFurther": [
328         "aleja la pantalla",
329         "alejar la pantalla",
330         "aleja pantalla",
331         "alejar pantalla",
332
333         "aleja la imagen",
334         "alejar la imagen",
335         "aleja imagen",
336         "alejar imagen",
337         "aleja la vista",
338         "alejar la vista",
339         "aleja vista",
340         "alejar vista",
341
342         "pon la pantalla más lejos",
```

```
343         "poner la pantalla más lejos"
344
345     ],
346     "ActivatePassthrough": [
347         "ver el exterior",
348         "ver fuera",
349         "muestra el exterior",
350         "mostrar el exterior",
351         "muéstrame el exterior",
352
353         "ver el entorno",
354         "muéstrame el entorno",
355         "mostrar el entorno",
356         "ver mi entorno",
357
358         "ver alrededor",
359         "mira alrededor",
360         "mirar alrededor",
361         "ver alrededor mío",
362         "ver a mi alrededor",
363
364         "ver el mundo exterior",
365         "muestra lo que hay fuera",
366         "mostrar lo que hay fuera",
367         "enséñame lo que hay fuera",
368         "enseñar lo que hay fuera"
369
370     ],
371     "DeactivatePassthrough": [
372         "deja de mostrar el exterior",
373         "dejar de mostrar el exterior",
374         "oculta el exterior",
375         "ocultar el exterior",
376         "quítame el exterior",
377
378         "deja de mostrar el entorno",
379         "quítame el entorno",
380         "oculta el entorno",
381         "deja de mostrar mi entorno",
382
383         "deja de mostrar alrededor",
384         "deja de mirar alrededor",
385         "dejar de mirar alrededor",
386         "deja de mostrar lo que hay alrededor",
387         "deja de mostrar lo que hay a mi alrededor",
388
389         "deja de mostrar el mundo exterior",
390         "deja de mostrar lo que hay fuera",
391         "dejar de mostrar lo que hay fuera",
392         "deja de enseñarme lo que hay fuera",
```

```
        "dejar de enseñar lo que hay fuera"

    ],
    "OpenConfiguration": [
        "abrir la configuración",
        "abre la configuración",
        "abrir configuración",
        "abre configuración",

        "enseña la configuración",
        "enséñame la configuración",
        "enseña configuración",
        "enséñame configuración",

        "mostrar la configuración",
        "muéstrame la configuración",
        "mostrar configuración",
        "muéstrame configuración",


        "abrir los ajustes",
        "abre los ajustes",
        "abrir ajustes",
        "abre ajustes",

        "enseña los ajustes",
        "enséñame los ajustes",
        "enseña ajustes",
        "enséñame ajustes",

        "mostrar los ajustes",
        "muéstrame los ajustes",
        "mostrar ajustes",
        "muéstrame ajustes"
    ],
    "CloseConfiguration": [
        "cerrar la configuración",
        "cierra la configuración",
        "cerrar configuración",
        "cierra configuración",

        "oculta la configuración",
        "ocúltame la configuración",
        "oculta configuración",
        "ocúltame configuración",

        "dejar de mostrar la configuración",
        "deja de mostrar la configuración",
        "dejar de mostrar configuración",
        "deja de mostrar configuración",
```

```
443
444         "cerrar los ajustes",
445         "cierra los ajustes",
446         "cerrar ajustes",
447         "cierra ajustes",
448
449         "oculta los ajustes",
450         "ocúltame los ajustes",
451         "oculta ajustes",
452         "ocúltame ajustes",
453
454         "dejar de mostrar los ajustes",
455         "deja de mostrar los ajustes",
456         "dejar de mostrar ajustes",
457         "deja de mostrar ajustes"
458
459     ],
460     "OpenCommandList": [
461         "abrir los comandos",
462         "abre los comandos",
463         "abrir comandos",
464         "abre comandos",
465
466         "mostrar los comandos",
467         "muestra los comandos",
468         "mostrar comandos",
469         "muestra comandos",
470
471         "enseña los comandos",
472         "enséñame los comandos",
473         "enseña comandos",
474         "enséñame comandos",
475
476         "abrir la lista de comandos",
477         "abre la lista de comandos",
478         "abrir lista de comandos",
479         "abre lista de comandos",
480
481         "mostrar la lista de comandos",
482         "muestra la lista de comandos",
483         "mostrar lista de comandos",
484         "muestra lista de comandos",
485
486         "enseña la lista de comandos",
487         "enséñame la lista de comandos",
488         "enseña lista de comandos",
489         "enséñame lista de comandos",
490
491         "abrir el menú de comandos",
492         "abre el menú de comandos",
```

```
493         "abrir menú de comandos",
494         "abre menú de comandos",
495 
496         "mostrar el menú de comandos",
497         "muestra el menú de comandos",
498         "mostrar menú de comandos",
499         "muestra menú de comandos"
500     ],
501     "CloseCommandList": [
502         "cerrar los comandos",
503         "cierra los comandos",
504         "cerrar comandos",
505         "cierra comandos",
506 
507         "ocultar los comandos",
508         "oculta los comandos",
509         "ocultar comandos",
510         "oculta comandos",
511 
512         "dejar de mostrar los comandos",
513         "deja de mostrar los comandos",
514         "dejar de mostrar comandos",
515         "deja de mostrar comandos",
516 
517         "cerrar la lista de comandos",
518         "cierra la lista de comandos",
519         "cerrar lista de comandos",
520         "cierra lista de comandos",
521 
522         "ocultar la lista de comandos",
523         "oculta la lista de comandos",
524         "ocultar lista de comandos",
525         "oculta lista de comandos",
526 
527         "dejar de mostrar la lista de comandos",
528         "deja de mostrar la lista de comandos",
529         "dejar de mostrar lista de comandos",
530         "deja de mostrar lista de comandos",
531 
532         "cerrar el menú de comandos",
533         "cierra el menú de comandos",
534         "cerrar menú de comandos",
535         "cierra menú de comandos",
536 
537         "ocultar el menú de comandos",
538         "oculta el menú de comandos",
539         "ocultar menú de comandos",
540         "oculta menú de comandos",
541 
542         "dejar de mostrar el menú de comandos",
```

```
543             "deja de mostrar el menú de comandos",
544             "dejar de mostrar menú de comandos",
545             "deja de mostrar menú de comandos"
546
547         ]
548
549 }
```

A.0.2. Ejemplo del archivo de configuración config.json

Extracto de código A.2: Ejemplo del archivo de configuración config.json

```
1  {
2      "microphone": "Headset Microphone (Oculus Virtual Audio
           Device)",
3      "sample_rate": 16000,
4      "classifier": {
5          "threshold": 85
6      },
7      "whisper": {
8          "model": "base",
9          "language": "es",
10         "device": "cpu",
11         "compute_type": "int8"
12     },
13     "unity": {
14         "host": "127.0.0.1",
15         "port": 5005
16     }
17 }
```

A.0.3. Sesiónes para obtener los resultados

En cada sesión se dijeron las frases indicadas, en ese orden.

Sesión 1

Bloque A: Wake word

1. Centra la pantalla.
2. Mueve a la izquierda.
3. Aumenta el tamaño.
4. Abre los comandos.
5. Baja la pantalla.

6. Acerca la pantalla.
7. Abre la configuración.
8. Cierra la configuración.
9. Reduce el tamaño.
10. Mueve a la derecha.
11. Cierra los comandos.
12. Sube la pantalla.
13. Abre la configuración.
14. Aleja la pantalla.

Bloque B: Manual

1. Mueve a la derecha.
2. Reduce el tamaño.
3. Abre la configuración.
4. Centra la pantalla.
5. Abre los comandos.
6. Aumenta el tamaño.
7. Sube la pantalla.
8. Abre la configuración.
9. Aleja la pantalla.
10. Mueve a la izquierda.
11. Acerca la pantalla.
12. Abre los comandos.
13. Baja la pantalla.

Sesión 2

Bloque A: Manual

1. Acerca la pantalla.
2. Abre la configuración.
3. Sube la pantalla.
4. Aumenta el tamaño.
5. Mueve a la izquierda.

6. Abre los comandos.
7. Aleja la pantalla.
8. Baja la pantalla.
9. Cierra los comandos.
10. Reduce el tamaño.
11. Centra la pantalla.
12. Mueve a la derecha.
13. Abre la configuración.

Bloque B: Wake word

1. Baja la pantalla.
2. Abre la configuración.
3. Aumenta el tamaño.
4. Abre los comandos.
5. Cierra los comandos.
6. Centra la pantalla.
7. Mueve a la izquierda.
8. Reduce el tamaño.
9. Abre los comandos.
10. Sube la pantalla.
11. Acerca la pantalla.
12. Cierra los comandos.
13. Aleja la pantalla.
14. Mueve a la derecha.

Sesión 3

Bloque A: Wake word

1. Reduce el tamaño.
2. Abre los comandos.
3. Mueve a la izquierda.
4. Aleja la pantalla.
5. Sube la pantalla.

6. Abre la configuración.
7. Aumenta el tamaño.
8. Mueve a la derecha.
9. Cierra los comandos.
10. Acerca la pantalla.
11. Baja la pantalla.
12. Abre la configuración.
13. Centra la pantalla.

Bloque B: Manual

1. Abre la configuración.
2. Sube la pantalla.
3. Acerca la pantalla.
4. Abre los comandos.
5. Mueve a la derecha.
6. Aumenta el tamaño.
7. Centra la pantalla.
8. Mueve a la izquierda.
9. Cierra los comandos.
10. Aleja la pantalla.
11. Baja la pantalla.
12. Reduce el tamaño.
13. Abre los comandos.

Sesión 4

Bloque A: Manual

1. Aleja la pantalla.
2. Mueve a la derecha.
3. Abre los comandos.
4. Aumenta el tamaño.
5. Sube la pantalla.
6. Abre la configuración.

7. Acerca la pantalla.
8. Cierra los comandos.
9. Baja la pantalla.
10. Reduce el tamaño.
11. Abre la configuración.
12. Centra la pantalla.
13. Mueve a la izquierda.

Bloque B: Wake word

1. Mueve a la izquierda.
2. Centra la pantalla.
3. Reduce el tamaño.
4. Abre la configuración.
5. Baja la pantalla.
6. Cierra los comandos.
7. Acerca la pantalla.
8. Abre la configuración.
9. Sube la pantalla.
10. Aumenta el tamaño.
11. Abre los comandos.
12. Mueve a la derecha.
13. Aleja la pantalla.